

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS FLORIANÓPOLIS
CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Lucas Bueno Cesario

POLYGLOT IMPORT CSV: UMA SOLUÇÃO PARA IMPORTAÇÃO DE DOCUMENTOS CSV PARA MÚLTIPLOS BANCOS DE DADOS

Florianópolis
2026

Lucas Bueno Cesario

POLYGLOT IMPORT CSV: UMA SOLUÇÃO PARA IMPORTAÇÃO DE DOCUMENTOS CSV PARA MÚLTIPLOS BANCOS DE DADOS

Trabalho de Conclusão de Curso submetido ao Curso de Graduação em Ciência da Computação do Centro Tecnológico da Universidade Federal de Santa Catarina para obtenção do título de Bacharel em Ciência da Computação.
Orientador: Prof. Ronaldo dos Santos Mello, Dr.

Florianópolis
2026

Ficha catalográfica para trabalhos acadêmicos.

Cesario, Lucas Bueno
Polyglot Import CSV : Uma solução para importação de documentos CSV para múltiplos bancos de dados / Lucas Bueno Cesario ; orientador, Ronaldo dos Santos Mello, 2026.
49 p.

Trabalho de Conclusão de Curso (graduação) - Universidade Federal de Santa Catarina, Centro Tecnológico, Graduação em Ciências da Computação, Florianópolis, 2026.

Inclui referências.

1. Ciências da Computação. 2. Persistência poliglota. 3. Importação de dados CSV. 4. NoSQL. 5. Software Livre. I. Mello, Ronaldo dos Santos. II. Universidade Federal de Santa Catarina. Graduação em Ciências da Computação. III. Título.

Lucas Bueno Cesario

POLYGLOT IMPORT CSV: UMA SOLUÇÃO PARA IMPORTAÇÃO DE DOCUMENTOS CSV PARA MÚLTIPLOS BANCOS DE DADOS

Este Trabalho de Conclusão de Curso foi julgado adequado para obtenção do Título de "Bacharel em Ciência da Computação" e aprovado em sua forma final pelo Curso de Graduação em Ciências da Computação.

Florianópolis, _____ de _____ de 2026.

_____

Profa. Lúcia Helena Martins Pacheco, Dra.
Coordenadora do Curso

Banca Examinadora:

_____

Prof. Ronaldo dos Santos Mello, Dr.
Orientador
Universidade Federal de Santa Catarina

_____

Profa. Carina Friedrich Dorneles, Dra.
Avaliadora
Universidade Federal de Santa Catarina

_____

Prof. Jônata Tyska Carvalho, Dr.
Avaliador
Universidade Federal de Santa Catarina

“Sabendo que a provação da fé que vocês têm produz perseverança. Ora, a perseverança deve ter ação completa, para que vocês sejam perfeitos e íntegros, sem que lhes falte nada.”

Tiago, 1:3-4

RESUMO

A persistência poliglota é essencial na arquitetura contemporânea de bancos de dados, integrando diversas tecnologias de armazenamento em uma mesma aplicação. A ferramenta *Polyglot Import CSV* destaca-se como solução para otimizar a importação de dados CSV em diversos bancos de dados simultaneamente, superando desafios da persistência poliglota. Optou-se pelo formato CSV por ser amplamente utilizado na importação de dados pelos diversos SGBDs e por ser um formato simples e familiar tanto a humanos quanto a sistemas. Por meio de configurações declarativas em arquivos JSON, a ferramenta oferece suporte ao principal representante de código aberto dos bancos de dados relacionais — o PostgreSQL — e aos principais representantes dos bancos de dados NoSQL: Redis (chave-valor), MongoDB (documento), Apache Cassandra (colunar) e Neo4j (grafo). A importação admite dois modos de entrada — um arquivo CSV por entidade ou um único CSV combinado — e realiza a conversão nativa aos tipos de cada SGBD, com observabilidade e métricas por fase de execução. O desempenho apoia-se em dois recursos: a escrita em lote, com conversão vetorizada de tipos, e um modelo de execução em fluxo, de memória limitada, capaz de importar arquivos maiores que a memória disponível. Uma avaliação empírica com dados reais quantificou esses ganhos: a escrita em lote acelerou a importação em cerca de nove vezes em relação à gravação linha a linha e o modo de fluxo reduziu o pico de memória em mais de doze vezes no maior volume medido, mantendo-o constante em relação ao tamanho do arquivo. Não foram identificadas ferramentas equivalentes que realizem, a partir de arquivos CSV e de regras declarativas, a importação simultânea para múltiplos paradigmas de dados, o que evidencia a originalidade da proposta. A publicação do código aberto da ferramenta impulsiona avanços colaborativos na dinâmica arquitetura moderna de bancos de dados.

Palavras-chave: Persistência poliglota. Ferramenta *Polyglot Import CSV*. Importação de dados CSV. Avaliação de desempenho. JSON. SQL. NoSQL. Software Livre.

ABSTRACT

Polyglot persistence is essential in contemporary database architecture, integrating various storage technologies within the same application. The Polyglot Import CSV tool stands out as a solution for optimizing CSV data import into multiple databases simultaneously, overcoming polyglot persistence challenges. The CSV format was chosen because it is widely used for data import by various DBMSs and because it is a simple and familiar format for both humans and systems. Through declarative configurations in JSON files, the tool supports the leading open-source representative of relational databases — PostgreSQL — and the main representatives of NoSQL databases: Redis (key-value), MongoDB (document), Apache Cassandra (columnar), and Neo4j (graph). The import supports two input modes — one CSV file per entity or a single combined CSV — and performs native type casting to each DBMS, with observability and per-phase execution metrics. Performance rests on two features: batch writing, with vectorized type casting, and a streaming execution model, with bounded memory, able to import files larger than available memory. An empirical evaluation with real data quantified these gains: batch writing sped up the import by roughly nine times over row-by-row writing, and the streaming mode reduced peak memory by more than twelve times at the largest measured volume, keeping it constant with respect to file size. No equivalent tools were identified that perform, from CSV files and declarative rules, the simultaneous import to multiple data paradigms, which highlights the originality of the proposal. The open-source publication of the tool drives collaborative advances in the dynamic modern database architecture.

Keywords*: Polyglot persistence. Polyglot Import CSV tool. CSV data import. Performance evaluation. JSON. SQL. NoSQL. Free Software.*

LISTA DE ILUSTRAÇÕES

Figura 1 – Exemplo de aplicação da Persistência Poliglota em um e-commerce. 19
Figura 2 – Diferentes tipos de modelos de dados NoSQL. 20
Figura 3 – Modelo chave-valor: chaves apontando para valores arbitrários (Redis). 21
Figura 4 – Modelo colunar: chaves de linha e famílias de colunas (Cassandra). 22
Figura 5 – Modelo de documento: documentos autocontidos com campos variáveis (MongoDB). 23
Figura 6 – Modelo de grafo: nós e arestas tipadas com propriedades (Neo4j). . 24
Figura 7 – Visão geral da ferramenta: entradas, processamento e saídas. . . . 33
Figura 8 – Estrutura do JSON Schema de importação (`import_config`). 35
Figura 9 – Estrutura do JSON Schema de conexão (`dbms_config`). 47
Figura 10 – Fluxo de carregamento e validação das entradas da ferramenta. . . 50
Figura 11 – Algoritmo de execução da importação PolyglotImportCSV. 52
Figura 12 – Interface gráfica no estado ocioso: configuração multifonte escolhida, ícones de ajuda e comando montado, colorido, em modo somente leitura. 67
Figura 13 – Execução em andamento: formulário bloqueado, botão de interrupção e saída colorida da CLI chegando ao console em tempo real. 68
Figura 14 – Execução concluída com sucesso, com o tempo total e o arquivo de *log* da sessão na barra de estado. 69
Figura 15 – Erro detectado antes da execução: a configuração de importação viola o *JSON Schema*, a mensagem da CLI aparece no cartão de configuração e o botão de execução permanece desabilitado. 70
Figura 16 – Verificação dos SGBDs pela interface gráfica: MongoDB e Neo4j fora do ar e, abaixo da tabela de estados, os comandos que os iniciam. . 71
Figura 17 – Tempo total de importação por estratégia de escrita ($N = 10\,000$, materialização; eixo vertical em escala logarítmica). Campanha sem instrumentação de memória. 75
Figura 18 – *Throughput* de escrita por SGBD ($N = 100\,000$, multifonte, materialização, estratégia em lote). Campanha sem instrumentação de memória. 77
Figura 19 – Pico de memória por modelo de execução em função do volume (multifonte, escala logarítmica): a materialização cresce com o arquivo; o fluxo achata. 79
Figura 20 – Tempo total por modelo de execução em função do volume (multifonte): o sobrecusto do fluxo encolhe de $+61\%$ para $+10\%$ à medida que a escrita passa a dominar. 80

LISTA DE TABELAS

Tabela 1 – Tipos de evento do cenário de e-commerce. 35
Tabela 2 – Partes do arquivo de importação. 36
Tabela 3 – Campos de mapeamento de coluna (`columnSpec`). 40
Tabela 4 – Correspondência entre tipo inferido e tipo de destino na geração de DDL. 41
Tabela 5 – Partes do arquivo de conexão. 45
Tabela 6 – Campos do descritor `connection` por SGBD. 48
Tabela 7 – Estados reportados por `--check-dbms` com dois SGBDs fora do ar. . 55
Tabela 8 – Linhas da fonte `stock` (`ecommerce_stock.csv`, colunas de identificação resumidas). 56
Tabela 9 – Relação `inventory` materializada no PostgreSQL a partir da fonte `stock`. 57
Tabela 10 – Partição do usuário `user1` na tabela `user_activity_log` (Cassandra). 58
Tabela 11 – Materialização de uma linha da fonte `stock` nos diferentes SGBDs. . 60
Tabela 12 – Contagens reportadas pelo modo `--dry-run` no cenário e-commerce. 61
Tabela 13 – Imagens Docker dos SGBDs utilizadas na avaliação. 74
Tabela 14 – Tempo total e das fases dominantes por estratégia de escrita, em segundos ($N = 10\,000$, materialização). Campanha sem instrumentação de memória. 75
Tabela 15 – Tempo por fase de uma importação ($N = 100\,000$, multifonte, materialização), em segundos. Campanha sem instrumentação de memória. 76
Tabela 16 – Escrita por SGBD em uma importação ($N = 100\,000$, multifonte, materialização). Campanha sem instrumentação de memória. . . . 76
Tabela 17 – Multifonte *versus* combinado ($N = 100\,000$, estratégia em lote). Tempos e picos de memória provêm de campanhas distintas, conforme indicado. 77
Tabela 18 – Pico de memória por modelo de execução (N crescente, multifonte, estratégia em lote), em MB. Campanha com instrumentação de memória. 79
Tabela 19 – Tempo total por modelo de execução (N crescente, multifonte, estratégia em lote), em segundos. Campanha sem instrumentação de memória. 80
Tabela 20 – Pico de memória por estratégia de escrita ($N = 10\,000$, materialização), em MB. Campanha com instrumentação de memória. 81
Tabela 21 – Grupos de colunas do `ecommerce_join.csv` preenchidos por ação. . 99

Tabela 22 – Usuários do ecommerce_join.csv e seus dados de identificação e endereço. 100
Tabela 23 – Linhas action == stock do ecommerce_join.csv. 101
Tabela 24 – Linhas action == purchase do ecommerce_join.csv. 102
Tabela 25 – Linhas action == select_product do ecommerce_join.csv. 102
Tabela 26 – Linhas action == add_to_cart do ecommerce_join.csv. 103

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
API	*Application Programming Interface* (Interface de Programação de Aplicações)
BSON	*Binary JSON*
CI	*Continuous Integration* (Integração Contínua)
CLI	*Command Line Interface* (Interface de Linha de Comando)
CSV	*Comma-Separated Values* (Valores Separados por Vírgula)
DBMS	*Database Management System* (Sistema de Gerenciamento de Banco de Dados)
DDL	*Data Definition Language* (Linguagem de Definição de Dados)
GUI	*Graphical User Interface* (Interface Gráfica de Usuário)
JSON	*JavaScript Object Notation*
NoSQL	*Not Only SQL*
SGBD	Sistema de Gerenciamento de Banco de Dados
SGPD	Sistema de Gerenciamento Poliglota de Dados
SQL	*Structured Query Language* (Linguagem de Consulta Estruturada)
TCC	Trabalho de Conclusão de Curso
UFSC	Universidade Federal de Santa Catarina
XML	*Extensible Markup Language*
YCSB	*Yahoo! Cloud Serving Benchmark*

SUMÁRIO

1 INTRODUÇÃO . . . 14
1.1 Visão Geral . . . 14
1.2 Objetivos . . . 15
1.2.1 Objetivo Geral . . . 15
1.2.2 Objetivos Específicos . . . 15
1.3 Metodologia . . . 15
1.4 Estrutura do Trabalho . . . 16

2 FUNDAMENTAÇÃO TEÓRICA . . . 18
2.1 Persistência Poliglota . . . 18
2.2 Bancos de Dados NoSQL . . . 20
2.2.1 Chave-valor . . . 20
2.2.2 Colunar . . . 21
2.2.3 Documento . . . 22
2.2.4 Grafo . . . 23
2.3 Bancos de Dados Multimodelo e Arquiteturas de Persistência Poliglota . . . 24

3 TRABALHOS RELACIONADOS . . . 26
3.1 Importação de Arquivo CSV para um Único SGBD . . . 26
3.1.1 PostgreSQL . . . 26
3.1.2 Redis . . . 27
3.1.3 MongoDB . . . 28
3.1.4 Cassandra . . . 28
3.1.5 Neo4j . . . 29
3.2 Importação de Arquivo CSV para SGBD Multimodelo . . . 29
3.3 Considerações sobre os Trabalhos Relacionados . . . 30

4 A PROPOSTA DA FERRAMENTA POLYGLOTIMPORTCSV . . . 32
4.1 Visão Geral . . . 32
4.2 Formato de Configuração (JSON Schema) . . . 34
4.2.1 O Arquivo de Importação . . . 35
4.2.1.1 Raiz e bloco `sources` . . . 37
4.2.1.2 Blocos de SGBD . . . 38
4.2.1.3 Entidade . . . 39
4.2.1.4 Colunas . . . 40

4.2.2 Particularidades de Cada SGBD no Arquivo de Importação 41
4.2.3 O Arquivo de Conexão 45
4.2.3.1 Raiz e blocos de SGBD 46
4.2.3.2 Parâmetros de conexão 46
4.2.3.3 Inicialização dos SGBDs 48
4.2.4 Validação, Consistência entre Arquivos e Fluxo de Carregamento . . 49
4.3 Algoritmo de Execução da Importação 51
4.3.1 Verificação dos SGBDs 54
4.3.2 O Cenário de Exemplo e os Arquivos de Dados 55
4.3.3 A Importação em Cada SGBD 56
4.3.4 Exemplo de Uso e Evidência de Execução (Cenário E-commerce) . . 60
4.4 Observabilidade e Métricas 62
4.5 Estratégia de Escrita e Modelos de Execução 63
4.5.1 Evolução da Estratégia de Escrita 63
4.5.2 Modelos de Execução: Materialização e *Streaming* 64
4.6 Interface Gráfica 65

5 AVALIAÇÃO DE DESEMPENHO 72
5.1 Metodologia 72
5.1.1 Geração dos Dados 73
5.1.2 Protocolo de Medição 73
5.2 Configuração Experimental 74
5.3 Resultados 75
5.3.1 Estratégia de Escrita: Em Lote *versus* Ingênua (QP4) 75
5.3.2 Decomposição por Fase e Escrita por SGBD (QP1) 76
5.3.3 Modos de Importação: Multifonte *versus* Combinado (QP2) 77
5.3.4 Modelos de Execução: Memória e Tempo (QP5 e QP6) 78
5.3.5 Uma Observação sobre o Pico de Memória da Estratégia em Lote . 80
5.4 Análise e Discussão 81
5.4.1 Ameaças à Validade 82

6 ATIVIDADES FUTURAS 83

7 CONSIDERAÇÕES FINAIS 84

REFERÊNCIAS 86

APÊNDICES 89

APÊNDICE A – JSON SCHEMAS DE CONFIGURAÇÃO NA FORMA SERIALIZADA 90
A.1 Schema de Importação (import_config.schema.json) 90
A.2 Schema de Conexão (dbms_config.schema.json) 94

APÊNDICE B – O ARQUIVO DE DADOS ECOMMERCE_JOIN.CSV . . . 98
B.1 Linhas com action == stock 100
B.2 Linhas com action == purchase 101
B.3 Linhas com action == select_product 102
B.4 Linhas com action == add_to_cart 103

APÊNDICE C – ARQUIVOS DE CONFIGURAÇÃO DO CENÁRIO E-COMMERCE 104
C.1 Arquivo de Importação (import_config.json) 104
C.2 Arquivo de Importação no Modo Combinado (import_config_combined.json) 107
C.3 Arquivo de Conexão (dbms_config.json) 108

1 INTRODUÇÃO

1.1 VISÃO GERAL

Nas esferas industrial e acadêmica de bancos de dados, a concepção de que um único modelo de dados não atende a todas as necessidades tem conquistado aceitação, apontando para a perspectiva de persistência poliglota no futuro. Embora os sistemas de banco de dados relacionais devam manter sua predominância, espera-se uma convivência com diversas formas de sistemas, como os NoSQL (SADALAGE; FOWLER, 2013).

Algumas pesquisas têm sido feitas na área de persistência poliglota, como sugestões de modelos de dados unificados (CHILLóN et al., 2023), comparação de desempenho de bancos de dados multi-modelos contra abordagens de persistência poliglota (YE et al., 2023), revisões sistemáticas sobre modelagem poliglota de dados (SILVA; BORNIA; MELLO, 2024), tutoriais sobre o estado da arte e desafios abertos (KIEHN et al., 2022), e a adoção da persistência poliglota em Big Data (KHINE; WANG, 2019).

Dada essa necessidade crescente da adoção de persistência poliglota pelas aplicações, percebe-se a falta de uma ferramenta única de importação simultânea de dados para diversos SGBDs que tratem modelos de dados diferentes. Até onde se investigou (Capítulo 3), não foram identificadas ferramentas equivalentes: os importadores nativos atendem a um único SGBD por vez, e as soluções de migração ou de modelagem poliglota não realizam a materialização simultânea de um mesmo CSV em múltiplos paradigmas de dados — o que ressalta a originalidade desta proposta.

Assim sendo, este trabalho desenvolve a ferramenta *Polyglot Import CSV*, que importa dados armazenados em CSV para SGBDs relacionais e NoSQL. A configuração da ferramenta baseia-se em dois arquivos JSON: um de importação, mapeando entidades e relacionamentos em cada SGBD, considerado seu modelo de dados; outro de configuração de conexão dos SGBDs envolvidos. A escolha pelo formato JSON deve-se à sua simplicidade e ampla adoção; sua finalidade é detalhar, de forma declarativa, o mapeamento das colunas do CSV para os modelos de dados de cada SGBD, permitindo a validação prévia da importação.

CSV (*Comma-Separated Values*) é um arquivo de valores separados por vírgulas, frequentemente visualizado em Excel e outras planilhas eletrônicas. Pode haver outros tipos de valores como delimitadores, mas o mais comum é a vírgula. Muitos sistemas e processos hoje convertem seus dados para o formato CSV para saídas de

arquivo para outros sistemas, relatórios amigáveis para humanos, disponibilização de dados públicos, e outras necessidades. É um formato de arquivo padrão com o qual humanos e sistemas já estão familiarizados ao usar e manipular.

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Desenvolver a ferramenta *Polyglot Import CSV* que possibilitará a importação de dados em arquivos CSV para múltiplos SGBDs: PostgreSQL, Redis, MongoDB, Cassandra ou Neo4j.

1.2.2 Objetivos Específicos

- Projetar os arquivos de configuração JSON para que eles sigam sintaxes específicas, e flexíveis para lidar com vários modelos de dados, além de adicionar opções como filtros, escolha de chave primária e escolha do nome da coluna no banco de dados;
- Não permitir configurações incorretas, ou seja, se houver erro em algum lugar dos arquivos de configuração, a importação não deve começar até que sejam corrigidos (validação prévia da configuração);
- Avaliar o desempenho da ferramenta para diferentes volumes e modelos de dados destino;
- Disponibilizar a ferramenta de forma acessível ao usuário: por meio de uma interface de linha de comando (CLI) e de uma interface gráfica (GUI) que monta e executa o comando de importação de forma assistida, mantendo-o visível ao usuário.

1.3 METODOLOGIA

O desenvolvimento deste trabalho segue uma metodologia de pesquisa aplicada, adotando as seguintes etapas:

1. Estudo sobre persistência poliglota e seu estado da arte;
2. Estudo sobre os paradigmas não-relacionais que serão abrangidos pela solução: chave-valor, documento, grafo e colunar;
3. Estudo de decisão de linguagem de implementação que demonstra melhor produtividade para codificar a solução;

4. Projeto dos arquivos de configuração JSON;
5. Projeto da instrução principal de importação que utiliza os arquivos de configuração;
6. Implementação da solução “Polyglot Import CSV”;
7. Testes da solução “Polyglot Import CSV” implementada;
8. Avaliação de desempenho da solução para diferentes volumes e modelos de dados destino.

Na primeira etapa, foi estudado o que é a persistência poliglota, e a sua relevância no cenário atual.

Na segunda etapa, foram estudados os diversos modelos de dados não-relacionais: como cada modelo representa suas entidades; se há relacionamentos entre entidades no modelo; se é possível a existência de entidades aninhadas e multivaloradas; as limitações de cada modelo.

Na terceira etapa, foram estudadas as formas de implementar a solução usando as linguagens Java e Python e, pela simplicidade e produtividade, a linguagem Python foi escolhida.

Na quarta e quinta etapas, foram projetados os dois arquivos de configuração JSON — um para o mapeamento da importação (entidades, relacionamentos e colunas) e outro para a conexão dos SGBDs — bem como a instrução principal de importação que os utiliza.

As etapas seguintes trataram da implementação da solução proposta, “Polyglot Import CSV”, e dos testes da mesma com diferentes volumes de dados CSV e importações simples e complexas para diferentes modelos de dados destino. A última etapa consistiu na avaliação de desempenho da ferramenta, cujos resultados — *throughput* por SGBD, ganho da escrita em lote e economia de memória do modo de execução em fluxo — são apresentados no Capítulo 5.

1.4 ESTRUTURA DO TRABALHO

Este trabalho está estruturado em sete capítulos principais. O primeiro aborda os objetivos, a metodologia e o contexto que levaram à elaboração deste trabalho. O capítulo 2 revisa os fundamentos teóricos e o capítulo 3 apresenta os trabalhos relacionados. O capítulo 4 descreve a proposta da ferramenta *Polyglot Import CSV* — o formato de configuração, o algoritmo de importação, a observabilidade e os modelos de execução. O capítulo 5 apresenta a avaliação de desempenho da ferramenta, com

resultados reais. O capítulo 6 discute as atividades futuras e o capítulo 7 traz as considerações finais.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 PERSISTÊNCIA POLIGLOTA

Ford (2008) introduziu o conceito de “programação poliglota” para expressar a ideia de que aplicativos devem ser desenvolvidos utilizando uma combinação de linguagens, aproveitando o fato de que diferentes linguagens são mais adequadas para lidar com diferentes problemas. Uma das consequências notáveis da programação poliglota é a transição para a “persistência poliglota”, usando a mesma analogia: pode-se dizer que múltiplas tecnologias de bancos de dados podem ser utilizadas como armazenamento persistente para diferentes tipos de casos de uso. Embora ainda exista uma quantidade considerável de dados gerenciados por sistemas relacionais, a abordagem predominante passa a ser a reflexão sobre como se deseja manipular os dados antes de determinar qual tecnologia é a mais adequada para essa manipulação (FOWLER, 2011).

A persistência poliglota abrange uma variedade de formatos de dados, incluindo estruturados, semi-estruturados e não estruturados. Dados estruturados geralmente englobam formatos como relacional, objeto-relacional, e grafos de propriedades. Dados semi-estruturados incluem principalmente documentos JSON e XML. Por fim, dados não estruturados são representados por arquivos de texto, imagens e áudios, dentre outros (YE et al., 2023).

Um domínio em que a persistência poliglota pode ser aplicada é o de um e-commerce. Nesse cenário, diferentes categorias de dados possuem características e padrões de acesso distintos — volume de escrita, frequência de leitura, necessidade de consistência, estrutura dos registros — o que torna mais adequado o uso de um SGBD específico para cada uma delas. A Figura 1 ilustra esse cenário, mapeando cada categoria de dado do e-commerce ao SGBD mais adequado, conforme sugerido a seguir:

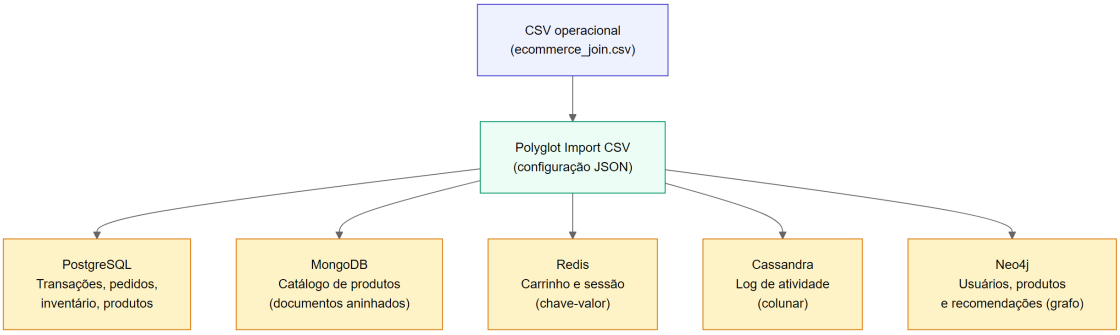


Figura 1 – Exemplo de aplicação da Persistência Poliglota em um e-commerce.
Fonte: Elaborado pelo autor (2026).

- **Dados Financeiros/Transacionais de Pagamento e Inventário de Produtos:** Pedidos, pagamentos e estoques exigem garantias transacionais fortes (ACID) para evitar inconsistências — por exemplo, debitar um pagamento sem registrar o pedido correspondente. Além disso, consultas analíticas sobre esses dados frequentemente combinam várias entidades por meio de junções. Bancos de dados relacionais como o PostgreSQL são a escolha natural para esse perfil, pois oferecem suporte nativo a transações com consistência forte, integridade referencial e uma linguagem de consulta expressiva (SQL).

- **Dados do Catálogo de Produtos:** O catálogo de um e-commerce tende a ter esquema variável: diferentes produtos possuem atributos distintos (um livro físico com ISBN e editora; um eletrodoméstico com voltagem e garantia). Além disso, o catálogo é lido com muito maior frequência do que escrito, e cada consulta normalmente recupera o produto completo de uma vez. Bancos de dados orientados a documentos como o MongoDB são otimizados exatamente para esse perfil: armazenam cada produto como um documento autocontido (favorecendo leituras de agregados complexos), suportam esquemas flexíveis e oferecem índices secundários e *pipelines* de agregação para fins de consulta.

- **Dados da Sessão e do Carrinho de Compras do Usuário:** Dados de sessão e carrinho são acessados a cada interação do usuário durante a navegação, exigindo latência mínima, e são também inerentemente temporários — um carrinho abandonado não precisa persistir indefinidamente. Bancos de dados chave-valor em memória como o Redis oferecem latência de microssegundos, expiração automática de chaves e são horizontalmente escaláveis para absorver picos de tráfego, tornando-se a escolha ideal para dados de natureza efêmera e de acesso intenso.

- **Dados de Atividade do Usuário:** Eventos de navegação (*clickstream*), visualizações de produto e histórico de ações são gerados em alto volume e alta

velocidade. Consultas típicas percorrem o histórico de um usuário em um intervalo de tempo, ou agregam eventos por tipo de ação. Bancos de dados colunares como o Apache Cassandra são projetados para ingestão massiva com particionamento por chave primária composta (por exemplo, `(user_id, timestamp)`), garantindo escritas rápidas e leituras eficientes por usuário ou período — padrão ideal para séries temporais de atividade.

- **Dados de Recomendação:** Sistemas de recomendação dependem de relacionamentos entre entidades: usuários que compraram os mesmos produtos, produtos frequentemente adquiridos em conjunto ou usuários com perfis semelhantes. Esses padrões são naturalmente modelados como grafos, onde nós representam usuários e produtos, e arestas representam interações (comprou, avaliou, visualizou, etc.). Bancos de dados orientados a grafos como o Neo4j permitem percorrer esses relacionamentos com eficiência por meio de consultas Cypher, tarefa que em um banco de dados relacional exigiria múltiplas junções e difícil otimização.

2.2 BANCOS DE DADOS NOSQL

Os bancos de dados NoSQL podem ser divididos em subcategorias com base em seus modelos de dados. Este trabalho utiliza a classificação de Hecht e Jablonski (2011), que categoriza os bancos de dados NoSQL em quatro tipos: chave-valor, colunares, documentos e grafos.

A Figura 2 ilustra esses modelos de dados, que são detalhados a seguir.



Figura 2 – Diferentes tipos de modelos de dados NoSQL.
Fonte: Elaborado pelo autor (2026).

2.2.1 Chave-valor

Os bancos de dados de chave-valor têm um modelo de dados simples baseado justamente em pares chave-valor, semelhante a um mapa associativo ou dicionário. A chave identifica exclusivamente o valor e é usada para armazenar e recuperar o valor no banco de dados, funcionando como uma chave primária. O valor pode ser usado para armazenar qualquer dado arbitrário, incluindo um número inteiro, uma *string*, um *array* ou um objeto, proporcionando uma representação de dados sem esquema conhecido.

Além disso, os bancos de dados de chave-valor são muito eficientes no armazenamento de dados distribuídos, mas não são adequados para cenários que requerem a definição de relacionamentos entre dados ou esquemas fortemente estruturados.

Qualquer funcionalidade que necessite desses requisitos deve ser implementada na aplicação cliente que interage com o banco de dados chave-valor.

Como os valores são opacos para os bancos de dados, esses não podem lidar com consultas e indexações a nível de dados, podendo realizar consultas apenas por meio de chaves. Um exemplo de SGBD chave-valor é o Redis, que mantém os dados tanto na memória como no disco. A Figura 3 ilustra o modelo com um exemplo de carrinho de compras e sessão de usuário, em que cada chave aponta para um valor opaco ao SGBD.

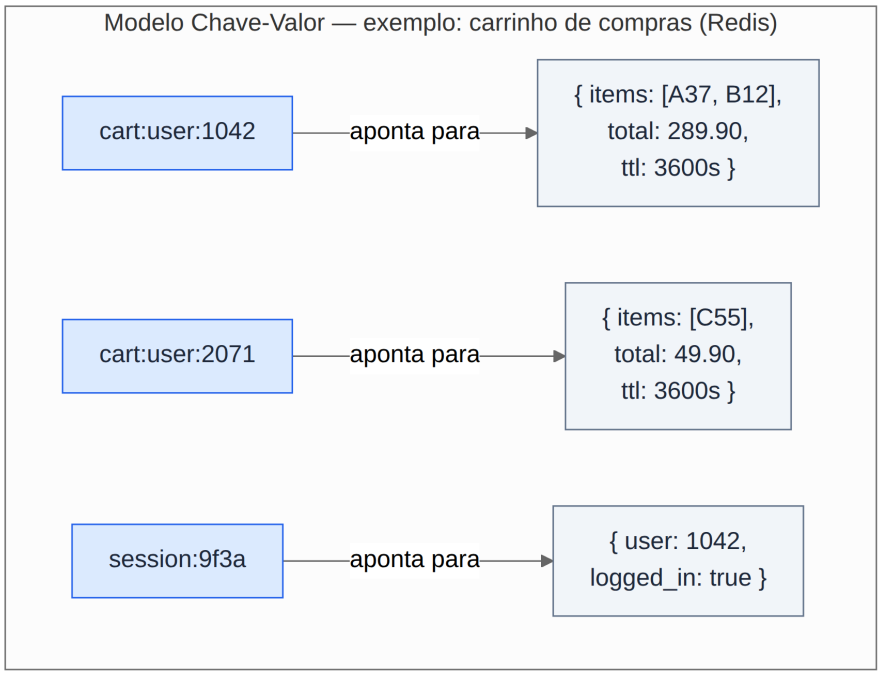


Figura 3 – Modelo chave-valor: chaves apontando para valores arbitrários (Redis).
Fonte: Elaborado pelo autor (2026).

2.2.2 Colunar

Em bancos de dados colunares, o conjunto de dados consiste em várias linhas, cada uma identificada por uma chave de linha única, semelhante a uma chave primária. Cada linha é composta por uma família (ou conjunto) de colunas, e diferentes linhas podem ter diferentes famílias de colunas. Semelhante aos bancos de dados de chave-valor, a chave de linha funciona como a chave, e o conjunto de famílias de colunas atua como o valor representado pela chave de linha. No entanto, colunas em uma família de colunas consistem em pares atributo-valor que podem ser indexadas. O SGBD Cassandra oferece a funcionalidade adicional de supercolunas, que são criadas agrupando várias colunas.

Normalmente, os dados pertencentes a uma linha são armazenados juntos no mesmo nó do servidor. No entanto, o Cassandra pode distribuir uma única linha

por vários nós de servidor usando chaves de partição compostas. Nos bancos de dados colunares, a configuração das famílias de colunas é tipicamente feita durante a inicialização. No entanto, a definição prévia de colunas não é necessária, oferecendo grande flexibilidade no armazenamento de qualquer tipo de dado.

De forma semelhante aos bancos de dados de chave-valor, qualquer lógica que exija relações deve ser implementada na aplicação cliente, ou seja, essa categoria de banco de dados também não têm ciência de relacionamentos entre dados e controle de integridade referencial. A Figura 4 apresenta um exemplo de *log* de atividade, em que cada chave de linha (`user_id`) agrupa uma família de colunas com eventos indexados por *timestamp*.

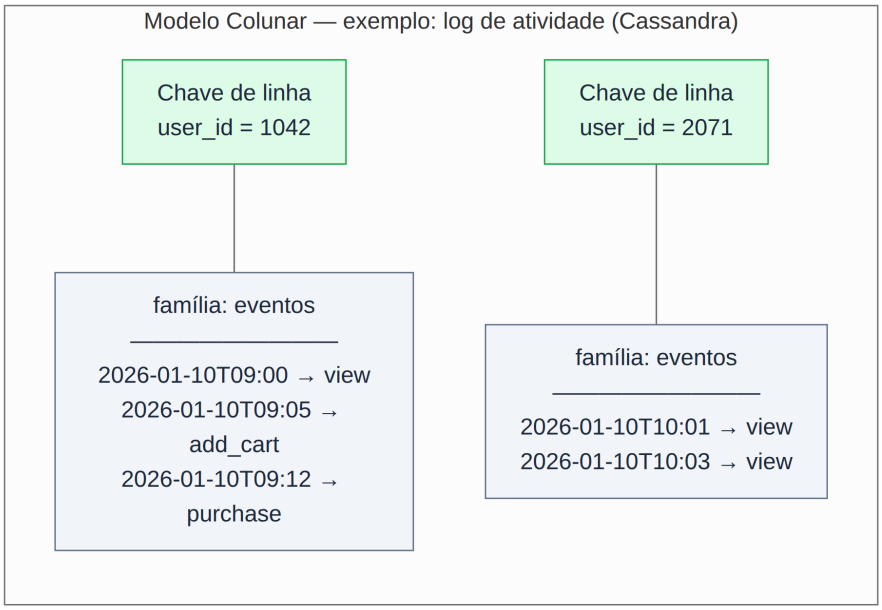


Figura 4 – Modelo colunar: chaves de linha e famílias de colunas (Cassandra).
Fonte: Elaborado pelo autor (2026).

2.2.3 Documento

Bancos de dados orientados a documento armazenam dados como documentos autocontidos, em geral em formato JSON ou BSON. Cada documento possui um identificador e um conjunto de campos que podem variar entre documentos de uma mesma coleção, favorecendo esquemas flexíveis e agregados de leitura (*read-optimized*). O SGBD MongoDB é um exemplo popular nesta categoria. Ele define coleções de documentos, índices secundários e *pipelines* de agregação, permitindo consultas ricas sem exigir junções (*joins*) no servidor. A Figura 5 ilustra uma coleção de catálogo de produtos, em que cada documento é autocontido e pode conter subdocumentos aninhados (como `category` e `stock`) e campos distintos entre si.

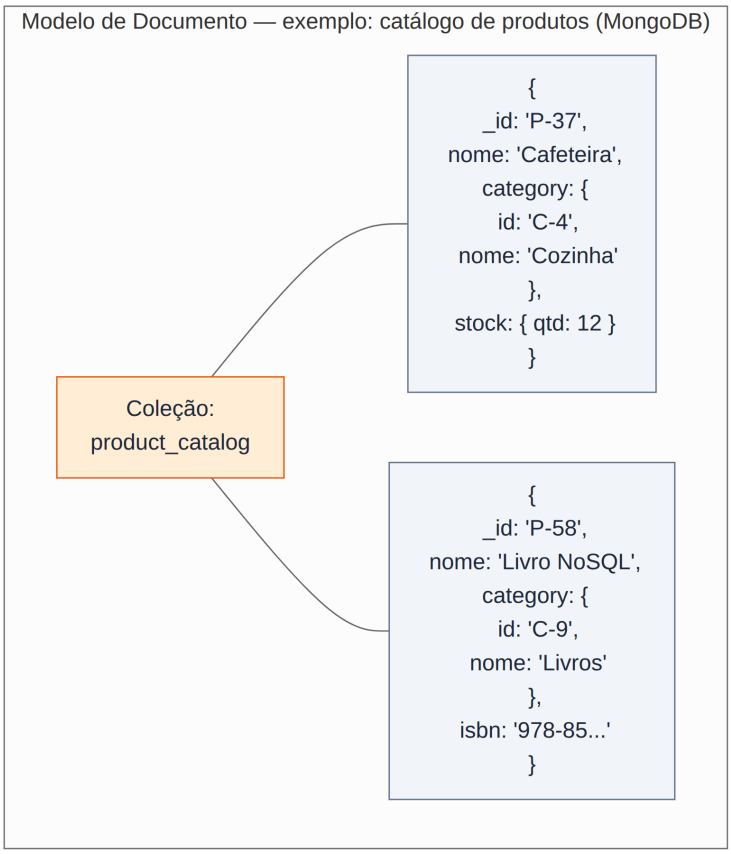


Figura 5 – Modelo de documento: documentos autocontidos com campos variáveis (MongoDB).
Fonte: Elaborado pelo autor (2026).

2.2.4 Grafo

Bancos de dados em grafo representam dados como **nós** (entidades) e **arestas** (relacionamentos tipados), com propriedades em ambos. Consultas exploram caminhos e vizinhanças de forma natural, o que é útil para recomendação, detecção de fraude e redes sociais. O Neo4j utiliza a linguagem Cypher para `MERGE`/`CREATE` de nós e relacionamentos.

Na ferramenta proposta, o destino grafo exige que o usuário declare explicitamente quais colunas do CSV identificam cada rótulo de nó e quais colunas alimentam propriedades do relacionamento (por exemplo, comprador $\rightarrow$ vendedor com atributos do pedido). A Figura 6 exemplifica um cenário de recomendação, em que nós `User` e `Product` são conectados por arestas tipadas (`PURCHASED`, `VIEWED`, `SIMILAR_TO`), algumas portando propriedades próprias.

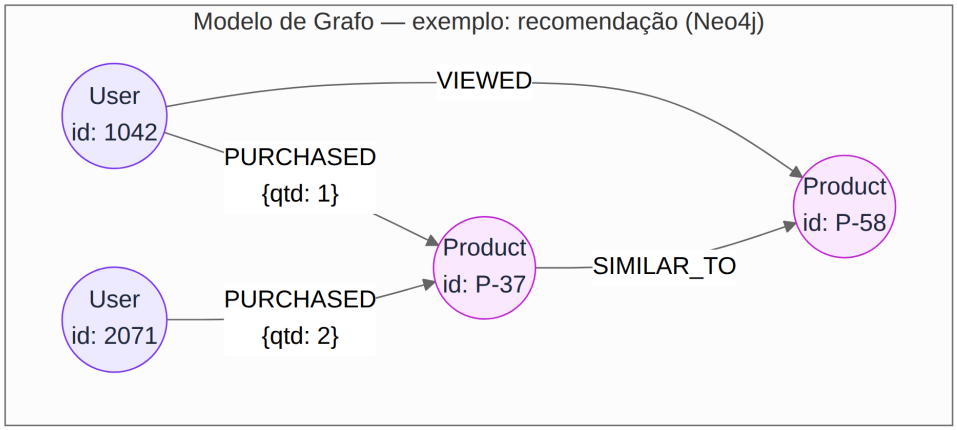


Figura 6 – Modelo de grafo: nós e arestas tipadas com propriedades (Neo4j).
Fonte: Elaborado pelo autor (2026).

2.3 BANCOS DE DADOS MULTIMODELO E ARQUITETURAS DE PERSISTÊNCIA POLIGLOTA

SGBDs multimodelo integram mais de um paradigma de dados no mesmo motor de gerência de dados (por exemplo, documento + chave-valor + grafo), oferecendo em geral uma interface de acesso unificada — como a linguagem AQL do SGBD ArangoDB, que combina filtros sobre estruturas de documento similares ao JSON e travessias em grafos em uma única abstração de modelo de dados que combina ambos os modelos NoSQL.

A persistência poliglota, por outro lado, combina vários SGBDs heterogêneos, cada um escolhido pela adequação a um caso de uso da aplicação (SADALAGE; FOWLER, 2013). Na literatura recente, essa ideia materializa-se como um SGPD (Sistema de Gerenciamento Poliglota de Dados): coordenação transparente entre tecnologias distintas, com requisitos de modelagem e expressividade de acesso (diferentes padrões de consulta) e capacidade de adaptação quando o *workload* evolui (KIEHN et al., 2022; GLAKE et al., 2022).

Tan et al. (2017) apresentam uma taxonomia útil para distinguir arquiteturas relacionadas: BD federado (fontes homogêneas, interface única), BD multistore (fontes heterogêneas, modelo canônico de consulta), BD *polystore* (fontes heterogêneas, múltiplas interfaces de acesso — exemplificadas por sistemas como o BigDAWG) e abordagens multimodelo (vários modelos no mesmo motor).

A escolha entre essas arquiteturas não é trivial. Roy-Hubara, Shoval e Sturm (2022) propõem critérios para selecionar quais modelos de dados usar em aplicações de persistência poliglota, destacando o *trade-off* entre especialização por SGBD e simplicidade operacional de um motor multimodelo.

O *Polyglot Import CSV* adota explicitamente a linha da persistência poliglota ao invés de multimodelo, conforme detalhado no capítulo 4. O usuário direciona entidades e filtros para PostgreSQL, Redis, MongoDB, Cassandra e Neo4j conforme o padrão de representação e acesso desejados.

3 TRABALHOS RELACIONADOS

Este capítulo apresenta e discute ferramentas oferecidas na indústria por SGBDs e também propostas na literatura acadêmica relacionadas com a importação de dados por SGBDs heterogêneos. A preferência está em dados no formato CSV que devem ser importados por alguma tecnologia de banco de dados, que é o foco deste trabalho.

3.1 IMPORTAÇÃO DE ARQUIVO CSV PARA UM ÚNICO SGBD

Esta seção investiga mecanismos de importação de dados presentes em SGBDs populares na indústria que são considerados neste trabalho.

3.1.1 PostgreSQL

No SGBD PostgreSQL você utiliza o comando `COPY` para realizar importações de dados CSV (The PostgreSQL Global Development Group, 2026). É necessário especificar a tabela com os nomes das colunas após a cláusula `COPY`. A ordem das colunas deve ser a mesma que aquelas no arquivo CSV, conforme ilustra o exemplo a seguir:

```
COPY nome_da_tabela(col1, col2, col3)
FROM 'C:/caminho_do_arquivo/dados_de_amostra.csv'
DELIMITER ','
CSV HEADER;
```

Caso o arquivo CSV contenha todas as colunas da tabela, você não precisa especificá-las explicitamente:

```
COPY nome_da_tabela
FROM 'C:/caminho_do_arquivo/dados_de_amostra.csv'
DELIMITER ','
CSV HEADER;
```

Você também deve inserir o caminho do arquivo CSV após a palavra-chave `FROM`. Como o formato do arquivo é CSV, você precisa especificar `DELIMITER`, bem como a cláusula `CSV`. A cláusula `HEADER` deve ser especificada para indicar que o arquivo CSV contém um cabeçalho. Quando o comando `COPY` importa dados, ele ignora o cabeçalho do arquivo.

O arquivo CSV deve ser lido diretamente pelo servidor PostgreSQL, não pela aplicação cliente. Portanto, ele deve ser acessível pela máquina do servidor PostgreSQL. Além disso, é necessário ter permissão de superusuário para executar com sucesso o comando `COPY`.

3.1.2 Redis

A maneira preferida de importar uma massa de dados para o Redis é gerar um arquivo de texto contendo o protocolo Redis, em formato bruto, para chamar os comandos necessários para inserir os dados requeridos (Redis Ltd., 2026).

Por exemplo, se é necessário gerar um grande conjunto de dados com bilhões de chaves no formato "chaveN $\rightarrow$ ValorN", deve ser criado um arquivo contendo os seguintes comandos:

```
SET Chave0 Valor0
SET Chave1 Valor1
SET ChaveN ValorN
```

Uma vez criado esse arquivo, a ação restante é alimentá-lo no Redis. No passado, a maneira de fazer isso era usar o netcat com o seguinte comando:

```
(cat dados.txt; sleep 10) | nc localhost 6379
```

Nas versões 2.6 ou posteriores do Redis, o utilitário `redis-cli` suporta um novo modo chamado modo *pipe* que foi projetado para realizar o carregamento em massa. O comando a ser executado neste caso é o seguinte:

```
cat dados.txt | redis-cli --pipe
```

O utilitário `redis-cli` imprime um pequeno resumo. A saída típica é semelhante a esta:

```
All data transferred. Waiting for the last reply...
Last reply received from server.
errors: 0, replies: 1000000
```

A linha de contagem ao final é a informação mais importante do resumo: `errors` indica quantos comandos foram rejeitados pelo servidor, e `replies` indica o total de respostas recebidas. Se `errors: 0`, todos os comandos foram processados com sucesso. Um valor maior que zero em `errors` significa que ao menos um comando falhou — seja por sintaxe inválida no arquivo, por tipo de dado incompatível com a chave existente, ou por qualquer outra razão que o Redis tenha rejeitado.

Vale notar que o `--pipe` não exibe cada resposta individualmente (como um `+OK` por `SET`), o que é intencional: imprimir bilhões de respostas destruiria o ganho de *performance* do modo *pipe* e inundaria o terminal. Em vez disso, o `redis-cli` lê e contabiliza todas as respostas silenciosamente conforme chegam, entregando apenas o agregado final.

Por fim, uma verificação útil: o valor de `replies` deve ser igual ao número de comandos enviados. Uma divergência entre os dois pode indicar que o arquivo de entrada estava truncado ou malformado, mesmo que `errors` apareça como zero.

3.1.3 MongoDB

No SGBD MongoDB, o processo de importação de dados em formato CSV é realizado através do utilitário `mongoimport` (MongoDB, Inc., 2026). Sua sintaxe básica exige que se informe o banco de dados de destino através do parâmetro `--db`, seguido do nome da coleção que receberá os dados, especificado em `--collection`. Caso esse último parâmetro seja omitido, o nome da coleção será automaticamente inferido a partir do nome do arquivo CSV fornecido.

```
mongoimport --db nome_do_db --collection nome_da_colecao \
  --type csv --headerline --ignoreBlanks \
  --file caminho/do/arquivo.csv
```

O tipo do arquivo de entrada deve ser declarado explicitamente através do parâmetro `--type`, que neste caso recebe o valor `csv`. Para que o `mongoimport` reconheça os nomes dos campos a partir do próprio arquivo, utiliza-se o parâmetro `--headerline`, que instrui o utilitário a tratar o conteúdo da primeira linha do CSV como o cabeçalho dos campos. Opcionalmente, o parâmetro `--ignoreBlanks` pode ser utilizado para que valores em branco presentes no arquivo sejam ignorados durante a importação, evitando que campos vazios sejam persistidos na coleção. Por fim, o caminho do arquivo CSV a ser importado é fornecido através do parâmetro `--file`.

3.1.4 Cassandra

A importação de dados CSV para o Cassandra utiliza o comando `sstableloader` e envolve vários passos (The Apache Software Foundation, 2026). Inicialmente, é necessário se certificar que o arquivo CSV está no formato adequado para o Cassandra. Isso geralmente envolve garantir que os tipos de dados e as colunas correspondem ao esquema da tabela no Cassandra.

O `sstableloader` requer que os dados estejam em um formato específico chamado SSTable. Para converter o CSV em SSTables, você pode usar o comando `cqlsh` fornecido pelo Cassandra. Um exemplo de uso é o seguinte:

```
cqlsh -e "COPY keyspace_name.table_name TO 'output_directory';"
```

Na sequência, é possível usar o `sstableloader` para carregar os dados. A sintaxe do comando é a seguinte:

```
sstableloader -d <hostname> -u <username> -pw <password> \
  output_directory/keyspace_name/table_name
```

- `<hostname>`: O endereço do nó Cassandra.
- `<username>`: O nome de usuário, se a autenticação estiver habilitada.
- `<password>`: A senha correspondente.

Este comando move os dados do diretório de saída para o Cassandra.

3.1.5 Neo4j

Usar `neo4j-admin` para importar grandes conjuntos de dados no Neo4j envolve alguns passos (Neo4j, Inc., 2026). Esta ferramenta de linha de comando é projetada para importações em massa eficientes.

Certifique-se de que seus arquivos CSV estejam estruturados corretamente e sigam o formato necessário para nós e relacionamentos. Crie um banco de dados não povoado:

```
neo4j-admin create-db --database=nome_do_seu_banco_de_dados \
  --from=diretorio_com_arquivos_csv
```

Substitua `nome_do_seu_banco_de_dados` pelo nome desejado para o novo banco de dados. A opção `--from` especifica o diretório onde estão localizados seus arquivos CSV. Execute o comando de importação com `neo4j-admin`. O comando exato depende do seu modelo de dados, mas pode se parecer com isto:

```
neo4j-admin import --database=nome_do_seu_banco_de_dados \
  --mode=csv \
  --nodes:Rotulo1 nos.csv \
  --nodes:Rotulo2 nos2.csv \
  --relationships:TIPO_RELACIONAMENTO relacionamentos.csv
```

Substitua *Rotulo1*, *Rotulo2*, *nos.csv*, *nos2.csv*, *TIPO_RELACIONAMENTO* e *relacionamentos.csv* pelos seus rótulos específicos e nomes de arquivo.

3.2 IMPORTAÇÃO DE ARQUIVO CSV PARA SGBD MULTIMODELO

Ferramentas comerciais e de código aberto tratam de **migração** ou **modelagem** entre tecnologias heterogêneas, embora raramente com o foco deste TCC (um único CSV largo + regras declarativas + cinco paradigmas).

- **ArangoDB** é um SGBD multimodelo nativo (documento/chave-valor/grafo) com `arangoimport` para CSV; a importação é para **um** motor multimodelo, não para cinco SGBDs independentes.
- **`mongoimport`** / **`COPY`** / **`redis-cli --pipe`** / **`cqlsh`** / **`neo4j-admin`** (seção 3.1) cobrem importação **para um único** produto por vez.
- **dbcrossbar** (DBCROSSBAR. . . , 2024) copia dados tabulares entre PostgreSQL, BigQuery, armazenamento em nuvem e CSV, incluindo conversão de esquemas; a ênfase é *pipeline* de migração, não mapeamento semântico de entidades aninhadas com filtros por valor.

- **Hackolade** (Hackolade, 2024) oferece *polyglot data modeling* com engenharia de esquema para dezenas de alvos; trata-se de **modelagem visual** e geração de artefatos, e não de um utilitário CLI único para importar um CSV operacional com validação prévia de filtros.

3.3 CONSIDERAÇÕES SOBRE OS TRABALHOS RELACIONADOS

A literatura sobre **metamodelos unificados** para SQL e NoSQL (por exemplo, U-Schema (CANDEL; RUIZ; GARCíA-MOLINA, 2022)) e sobre **evolução de esquema** em ambientes heterogêneos (CHILLóN et al., 2023; CHILLóN et al., 2024) informa decisões de validação e de representação de entidades no JSON de configuração, mas não substitui uma ferramenta de *bulk import* orientada a CSV.

Surveys recentes convergem em desafios abertos da gestão poliglota de dados: migração automatizada entre *stores* quando o *workload* muda, planejamento de consultas entre sistemas, gestão de esquemas multi-modelo e preservação das propriedades não funcionais de cada SGBD (KIEHN et al., 2022; GLAKE et al., 2022). Roy-Hubara, Shoval e Sturm (2022) tratam da **seleção de SGBDs** por fragmento de aplicação; Silva, Bornia e Mello (2024) mapeiam o estado da arte em **modelagem poliglota**. Nenhum desses trabalhos implementa, contudo, um utilitário declarativo que receba um CSV operacional único e o materialize simultaneamente em cinco paradigmas distintos com validação estática prévia.

Abordagens recentes reforçam essa tendência. Pereira, Guerra e Rosa (2025) propõem um *framework* de persistência poliglota **transparente**, com uma API unificada e extensível a diferentes tipos de banco, enquanto Sachdeva e Vasava (2024) exploram a combinação de bancos **multimodelo** e persistência poliglota para acomodar a variedade dos dados. Ambos os trabalhos atuam, porém, na **camada de acesso da aplicação** — oferecendo uma interface única para ler e gravar em múltiplos SGBDs em tempo de execução —, e não na **carga inicial declarativa** a partir de um CSV operacional, que é o foco da *Polyglot Import CSV*.

Sistemas *polystore* como o BigDAWG (DUGGAN et al., 2015) e *middlewares* de portabilidade em nuvem (KAUR; SHARMA; KAHLON, 2020) tratam de **consultas federadas** e movimentação de dados em ecossistemas complexos. Tan et al. (2017) classificam essas soluções (federado, multistore, polystore, poliglota) e avaliam transparência, autonomia e heterogeneidade — eixos nos quais importadores nativos (seção 3.1) e ferramentas de migração (seção 3.2) também divergem, mas sem convergir para um *pipeline* único de ingestão.

Ferramentas como **dbcrossbar** (DBCROSSBAR. . . , 2024) e **Hackolade** (Hackolade, 2024) aproximam-se do domínio de integração de dados, porém com foco

em *pipelines* de migração ou modelagem visual, respectivamente — não em regras declarativas sobre entidades aninhadas, filtros por valor (`each`) e validação por JSON Schema antes de qualquer conexão com SGBDs.

Lacuna identificada: a combinação de (i) um CSV largo como fonte operacional, (ii) configuração JSON validada estaticamente, (iii) materialização distinta por paradigma (relacional, chave-valor, documento, colunar, grafo) e (iv) modo *dry-run* para revisão prévia. O *Polyglot Import CSV* ocupa essa lacuna como ferramenta de **bootstrap de persistência poliglota** — carga inicial e reprodutível de dados — enquanto *middlewares* e *polystores* assumem dados já residentes ou consultas em tempo de execução. Trabalhos futuros podem integrar a ferramenta a mecanismos de migração adaptativa descritos por Kiehn et al. (2022) e Glake et al. (2022).

4 A PROPOSTA DA FERRAMENTA POLYGLOTIMPORTCSV

4.1 VISÃO GERAL

A ferramenta **PolyglotImportCSV** (pacote Python `polyglot-import-csv`) importa dados em formato CSV para vários SGBDs em uma única execução. Conforme ilustra a Figura 7, a ferramenta recebe como entrada os **arquivos de dados** (um ou mais CSV) e **dois arquivos de configuração** em formato JSON — o arquivo de **importação**, que declara quais CSVs alimentam a carga e mapeia as suas colunas para as entidades e relacionamentos desejados em cada SGBD, e o arquivo de **conexão**, que declara quais SGBDs estão disponíveis e como alcançá-los. O processamento é disparado por um comando de linha de comando (CLI), que primeiro **valida** as entradas — cada arquivo de configuração é conferido contra um **JSON Schema** embutido e, em seguida, contra os próprios CSVs — e só então executa a importação. A **saída** são os dados persistidos em cada SGBD presente na configuração, dentre os suportados: PostgreSQL, MongoDB, Cassandra, Redis e Neo4j. A ferramenta é um projeto de código aberto, hospedado no GitHub sob o nome `polyglot-import-csv`[1]; todos os caminhos de arquivo citados neste trabalho, como `data/ecommerce/` ou `docker-compose.yml`, são relativos à raiz desse repositório.

[1] Disponível em <https://github.com/lucasbc92/polyglot-import-csv>. Executáveis para Windows e Linux, que dispensam a instalação do Python, estão disponíveis na página de *releases* do projeto.

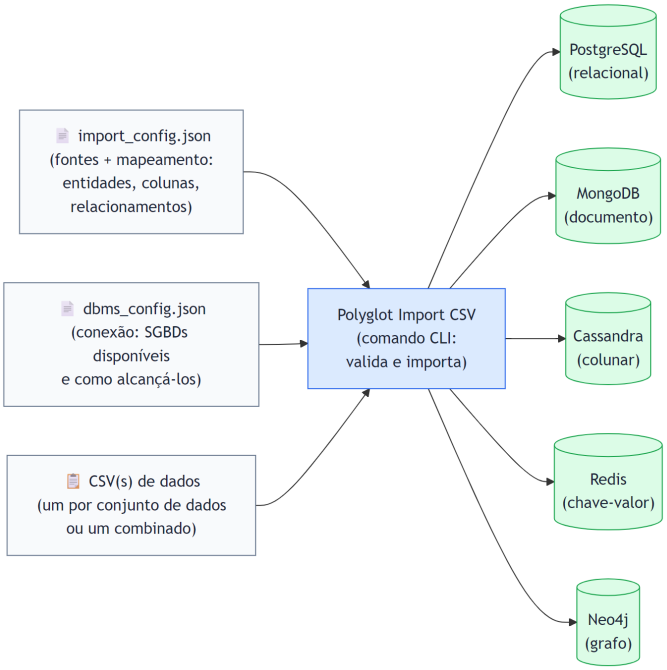


Figura 7 – Visão geral da ferramenta: entradas, processamento e saídas.
Fonte: Elaborado pelo autor (2026).

Diferentemente de uma importação amarrada a um único arquivo, os dados de entrada são declarados em um bloco `sources` dentro do arquivo de importação, o que habilita **dois modos de uso**: no modo **multifonte**, cada entidade lê de um CSV próprio (um arquivo por conjunto de dados); no modo **combinado**, um único CSV largo carrega todos os conjuntos de dados, distinguidos por uma coluna de origem. Ambos os modos e o formato completo dos dois arquivos de configuração são detalhados na seção 4.2; as validações realizadas antes da importação, na seção 4.2.4; o algoritmo de importação em si, na seção 4.3; e a camada de observabilidade e métricas, na seção 4.4. O comando principal da CLI é o seguinte:

```
python -m polyglotimportcsv \
  --config caminho/import_config.json \
  [--dbms-config caminho/dbms_config.json] \
  [--source NOME=CAMINHO] \
  [--dry-run] \
  [--check-dbms] \
  [--create-schema / --no-create-schema] \
  [--only postgres,redis] \
  [--execution stream|materialize] \
  [--log-level INFO] \
  [--sample N | --show-data | --no-data] \
  [--benchmark]
```

Cada parte do comando tem o seguinte papel. A opção `--config` é a única **obrigatória** e indica o caminho do arquivo JSON de importação, que contém, em seu bloco `sources`, os caminhos dos CSVs a carregar. As demais opções, entre colchetes, são opcionais. A opção `--dbms-config` indica o caminho do arquivo JSON de conexão dos SGBDs; quando não é informada, a ferramenta usa o arquivo `dbms_config.json`

do mesmo diretório do arquivo de importação. A opção `--source NOME=CAMINHO` **sobrescreve**, em tempo de execução, o caminho de uma fonte declarada no bloco `sources` sem editar o arquivo de configuração (por exemplo, para apontar a mesma configuração a um conjunto de dados maior); pode ser repetida uma vez por fonte.

As outras opções agrupam-se em três conjuntos. O primeiro controla **o que** e **onde** importar: `--dry-run` executa somente as validações e a preparação dos dados e reporta quantos registros **seriam** importados em cada destino, sem conectar a nenhum SGBD, o que é útil para revisar o mapeamento antes da carga real; `--check-dbms` apenas verifica se os SGBDs de destino respondem e encerra sem importar, como detalha a Seção 4.3.1; `--create-schema` e `--no-create-schema` controlam se a ferramenta cria as estruturas de armazenamento necessárias (tabelas no PostgreSQL, *keyspace* e tabelas no Cassandra) antes de gravar os dados, sendo a criação ativada por padrão; e `--only` restringe a execução a um subconjunto dos SGBDs configurados (por exemplo, `--only postgres,redis`). O segundo define **como** importar: `--execution` escolhe o modelo de execução (`stream`, o padrão, ou `materialize`), descrito na Seção 4.5.2. O terceiro conjunto governa a **observabilidade** da execução — `--log-level` ajusta o nível de detalhe das mensagens no terminal; `--sample N` exibe as primeiras `N` linhas de cada entidade (50, por padrão), enquanto `--show-data` exibe todas as linhas e `--no-data`, nenhuma; e `--benchmark` registra métricas de desempenho por fase —; essas opções são descritas em detalhe na seção 4.4.

4.2 FORMATO DE CONFIGURAÇÃO (JSON SCHEMA)

A configuração da ferramenta é dividida em **dois arquivos JSON**. O arquivo de **importação** (`import_config.json`) declara as fontes de dados e o **mapeamento** das suas colunas para entidades e relacionamentos em cada SGBD; o arquivo de **conexão** (`dbms_config.json`) declara quais SGBDs estão disponíveis e como alcançá-los. A separação atende a duas preocupações distintas: o mapeamento muda conforme o domínio dos dados, enquanto a conexão muda conforme o ambiente (desenvolvimento, testes, produção). Cada arquivo é validado por um **JSON Schema** próprio, embutido na ferramenta em `src/polyglotimportcsv/schemas/` e reproduzido no Apêndice A.

Os exemplos desta seção vêm do cenário de e-commerce apresentado na Seção 2.1 (Figura 1), cujos arquivos acompanham o repositório da ferramenta em `data/ecommerce/`. Os dados desse cenário registram os eventos de uma loja virtual, de quatro tipos, resumidos na Tabela 1. Todo evento registra o instante em que ocorreu e o usuário envolvido; a tabela indica os demais dados de cada tipo.

Tabela 1 – Tipos de evento do cenário de e-commerce.

Tipo	Evento	Demais dados registrados
`stock`	reposição de estoque	produto, categoria, quantidade disponível e preço
`purchase`	compra	pedido, produto, contraparte da venda, pagamento e avaliação
`select_product`	seleção de produto	produto selecionado
`add_to_cart`	adição ao carrinho	carrinho, produto e quantidade

Fonte: Elaborado pelo autor (2026).

Cada tipo de evento tem o seu próprio CSV, com oito linhas: `ecommerce_stock.csv`, `ecommerce_purchase.csv`, `ecommerce_select_product.csv` e `ecommerce_add_to_cart.csv`. Os mesmos eventos estão também reunidos em um único arquivo, `ecommerce_join.csv`, cuja primeira coluna (`action`) indica o tipo de cada linha. O Apêndice B reproduz esses dados em forma tabular.

As subseções seguintes descrevem cada arquivo da raiz até as folhas do seu schema: a Seção 4.2.1 apresenta o arquivo de importação; a Seção 4.2.2, o que esse arquivo tem de específico em cada SGBD; a Seção 4.2.3, o arquivo de conexão; e a Seção 4.2.4, as validações aplicadas aos dois antes da importação.

4.2.1 O Arquivo de Importação

A Figura 8 mostra a estrutura do schema do arquivo de importação, da raiz (à esquerda) até as folhas (à direita); os campos marcados com asterisco são obrigatórios. A Tabela 2 resume a finalidade de cada parte e indica quais podem ser omitidas.

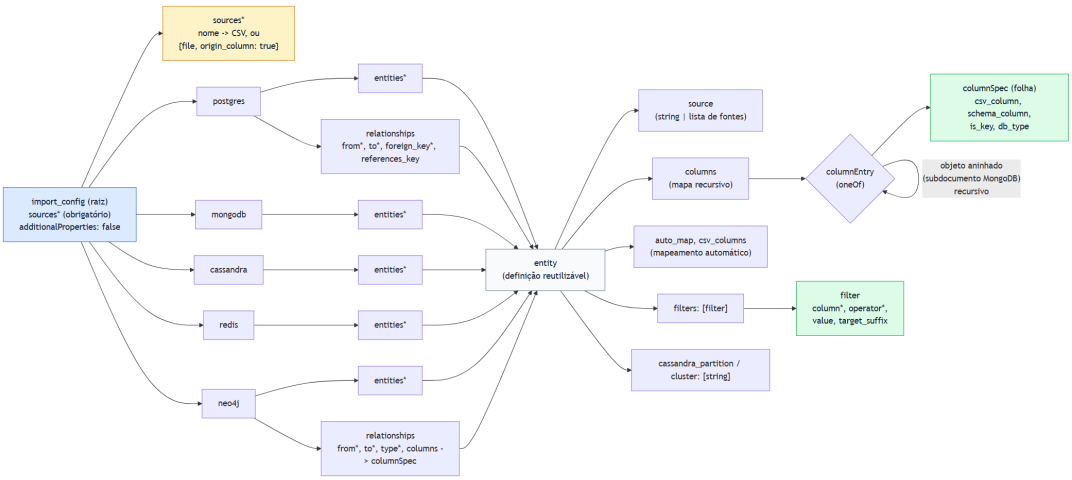


Figura 8 – Estrutura do JSON Schema de importação (`import_config`).

Fonte: Elaborado pelo autor (2026).

Tabela 2 – Partes do arquivo de importação.

Parte	**Onde**	**Finalidade**	**Obrigatória?**
`sources`	raiz	nomear os CSVs de entrada	sim
`postgres`, `mongodb`, `cassandra`, `redis`, `neo4j`	raiz	mapear dados para um SGBD	não; só os SGBDs de destino
`entities`	bloco de SGBD	declarar os alvos (tabelas, coleções, nós...)	sim
`relationships`	bloco PostgreSQL ou Neo4j	declarar chaves estrangeiras ou arestas	não
`source`	entidade	indicar a fonte lida	não; padrão: a fonte de mesmo nome
`columns`	entidade	mapear as colunas	não; se omitido, todas as colunas da fonte
`auto_map`, `csv_columns`	entidade	controlar o mapeamento automático	não
`filters`	entidade	selecionar linhas	não
`cassandra_partition`, `cassandra_cluster`	entidade do Cassandra	definir a chave primária	não; sem a partição, a gravação falha
`csv_column`, `schema_column`, `is_key`, `db_type`	coluna	ligar uma coluna do CSV a um atributo	não

Fonte: Elaborado pelo autor (2026).

A Listagem 4.1 mostra um trecho do arquivo de importação do cenário que percorre essa estrutura da raiz até as folhas: o bloco `sources`, o bloco do PostgreSQL, uma de suas entidades e duas de suas colunas. As reticências (...) marcam o conteúdo omitido; o arquivo completo está no Apêndice C.

```
"sources": {
  "stock":    "ecommerce_stock.csv",
  "purchase": "ecommerce_purchase.csv",
  ...
},
```

```
"postgres": {
  "entities": {
    "inventory": {
      "source": "stock",
      "columns": {
        "product_id": { "is_key": true, "db_type": "BIGINT" },
        ...
        "price":      { "db_type": "NUMERIC(14,2)" }
      }
    },
    ...
  }
},
...
```

Listagem 4.1 – Trecho do arquivo de importação, da raiz até as colunas.

Lido de cima para baixo, o trecho diz que: o bloco `sources` nomeia os arquivos de dados; o bloco `postgres` indica que parte dos dados vai para o PostgreSQL; a entidade `inventory` será uma tabela, alimentada pela fonte `stock`; e cada entrada de `columns` cria uma coluna dessa tabela. As subseções a seguir explicam cada um desses níveis, nessa ordem.

4.2.1.1 Raiz e bloco `sources`

A raiz do arquivo admite apenas o bloco `sources`, obrigatório, e um bloco por SGBD de destino. Qualquer outra chave é rejeitada (`additionalProperties: false`), de modo que um erro de digitação como `postgress` é apontado na validação em vez de ser ignorado.

O bloco `sources` dá um **nome** a cada arquivo CSV de entrada, e as entidades referem-se às fontes por esse nome. Há duas formas de declarar as fontes, que correspondem aos dois modos de importação. No modo **multifonte**, cada nome aponta diretamente para um CSV, como na Listagem 4.2: uma fonte por tipo de evento.

```
"sources": {
  "stock":          "ecommerce_stock.csv",
  "purchase":       "ecommerce_purchase.csv",
  "select_product": "ecommerce_select_product.csv",
  "add_to_cart":    "ecommerce_add_to_cart.csv"
}
```

Listagem 4.2 – Bloco `sources` no modo multifonte (um CSV por conjunto de dados).

No modo **combinado**, uma única fonte aponta para o CSV que reúne todos os eventos, e `"origin_column": true` indica que a primeira coluna desse arquivo identifica a origem de cada linha, como na Listagem 4.3.

```
"sources": {
  "ecommerce": { "file": "ecommerce_join.csv", "origin_column": true }
```

```
}
```

Listagem 4.3 – Bloco `sources` no modo combinado (um CSV com coluna de origem).

Ao carregar uma fonte combinada, a ferramenta separa as linhas pelo valor da primeira coluna e registra, além da própria fonte, uma fonte para cada valor distinto. Assim, os nomes `stock`, `purchase`, `select_product` e `add_to_cart` ficam disponíveis como no modo multifonte, e o restante do arquivo de importação é idêntico nos dois modos. Em ambos, cada linha recebe ainda uma **pseudo-coluna** `_source` com a sua origem — o nome da fonte, no modo multifonte, ou o valor da coluna de origem, no modo combinado —, que pode ser mapeada como qualquer outra coluna (um exemplo aparece na Listagem 4.10).

Nos dois modos, a **primeira linha de todo CSV é o seu cabeçalho**: é dela que vêm os nomes das colunas, aos quais o mapeamento se refere. O cabeçalho é, portanto, **obrigatório**, e a ferramenta não tem como distingui-lo de uma linha de dados. Em um CSV sem cabeçalho, a primeira linha de dados seria tomada como nomes de coluna e não seria importada; as colunas declaradas no mapeamento não seriam encontradas, e a execução terminaria com erro na validação (Seção 4.2.4). No modo combinado, a primeira coluna do cabeçalho é a coluna de origem.

4.2.1.2 Blocos de SGBD

Cada bloco de SGBD (`postgres`, `mongodb`, `cassandra`, `redis` ou `neo4j`) descreve o que será gravado naquele SGBD; só aparecem os blocos dos SGBDs que se deseja alimentar. A Listagem 4.4 mostra o esqueleto do bloco do PostgreSQL no cenário, com o conteúdo de cada entrada omitido.

```
"postgres": {
  "entities": {
    "categories": { ... },
    "products":   { ... },
    "inventory":  { ... },
    "orders":     { ... }
  },
  "relationships": {
    "product_category": { ... },
    "order_product":    { ... }
  }
}
```

Listagem 4.4 – Esqueleto do bloco `postgres` do arquivo de importação.

O mapa `entities`, obrigatório, contém as entidades a criar, cada uma identificada pelo nome que terá no SGBD — aqui, quatro tabelas. O mapa `relationships`, opcional, existe apenas nos blocos do PostgreSQL, onde declara chaves estrangeiras, e do Neo4j, onde declara arestas; ambos são detalhados na Seção 4.2.2.

4.2.1.3 Entidade

Uma entidade representa um alvo no SGBD: uma tabela no PostgreSQL ou no Cassandra, uma coleção no MongoDB, um conjunto de chaves no Redis ou um rótulo de nó no Neo4j. Todos os SGBDs usam a mesma definição de entidade, exemplificada na Listagem 4.5 pela entidade `inventory` do PostgreSQL.

```
"inventory": {
  "source": "stock",
  "columns": {
    "product_id":         { "is_key": true, "db_type": "BIGINT" },
    "quantity_available": { "db_type": "BIGINT" },
    "last_restock_date":  { "db_type": "TIMESTAMPTZ" },
    "price":              { "db_type": "NUMERIC(14,2)" }
  }
}
```

Listagem 4.5 – Entidade `inventory` do arquivo de importação.

Todos os campos de uma entidade são opcionais:

- `source` indica a fonte da qual a entidade lê. Pode ser o nome de uma fonte, como na listagem; uma lista de nomes (`"source": ["stock", "purchase"]`), caso em que as linhas das fontes são unidas em uma só entidade, com valores vazios nas colunas que faltam em alguma delas; ou ser omitido, caso em que a entidade lê da fonte que tem o seu próprio nome.
- `columns` lista as colunas da entidade (Seção 4.2.1.4). Se for omitido, a entidade recebe **todas** as colunas da fonte, com os nomes do CSV e tipos inferidos dos próprios dados; é o **mapeamento automático**, que `auto_map` e `csv_columns` ajustam, como descreve a mesma seção.
- `filters` restringe as linhas da fonte que participam da entidade, como descrito a seguir.
- `cassandra_partition` e `cassandra_cluster` definem a chave primária no Cassandra (Seção 4.2.2).

No cenário, cada tipo de evento já vem de uma fonte própria, e nenhuma entidade precisa de filtros. A Listagem 4.6 mostra um filtro ilustrativo, que não faz parte do cenário: aplicado a uma entidade que lê da fonte `purchase`, ele manteria apenas as compras pagas com cartão de crédito.

```
"filters": [
  { "column": "payment_method", "operator": "==", "value": "credit_card" }
]
```

Listagem 4.6 – Filtro ilustrativo sobre a coluna `payment_method`.

Cada filtro exige a coluna (`column`) e o operador (`operator`), que pode ser `==`, `!=`, `>`, `<`, `>=`, `<=`, `in`, `not_in` ou `each`; o valor comparado (`value`) também é exigido, exceto com `each`, e é uma lista com `in` e `not_in`. O operador `each` não restringe as linhas, e sim divide a entidade: cada valor distinto e não vazio da coluna gera um alvo próprio, cujo nome é o da entidade seguido de um sufixo derivado desse valor.

4.2.1.4 Colunas

O mapa `columns` associa cada atributo da entidade a uma coluna do CSV. A Listagem 4.7 mostra três entradas: as duas primeiras pertencem à entidade `user_activity_log` do Cassandra (Listagem 4.10); a terceira é ilustrativa.

```
"columns": {
  "product_id": {},
  "timestamp":  { "schema_column": "event_time" },
  "preco":      { "csv_column": "price" }
}
```

Listagem 4.7 – Três formas de mapear uma coluna.

A chave de cada entrada dá, por padrão, dois nomes ao mesmo tempo: o da coluna lida no CSV e o do atributo gravado no SGBD. Na primeira entrada, os dois nomes são iguais à chave (`product_id`), e o objeto vazio basta. Na segunda, a coluna `timestamp` do CSV é gravada com outro nome, `event_time`, indicado em `schema_column`. Na terceira, o atributo se chama `preco` no SGBD, mas é lido da coluna `price` do CSV, indicada em `csv_column`; esse campo também aceita a posição da coluna de dados, a partir de 1. A Tabela 3 resume os quatro campos que uma entrada pode ter.

Tabela 3 – Campos de mapeamento de coluna (`columnSpec`).

Campo	**Tipo**	**Obrigatório**	**Descrição**
`csv_column`	*string* ou inteiro (≥ 1)	não	Cabeçalho CSV ou posição da coluna de dados (base 1). Padrão: a chave.
`schema_column`	*string*	não	Nome do atributo no SGBD destino. Padrão: a chave.
`is_key`	booleano	não	Chave primária, chave de `MERGE` (Neo4j) ou chave Redis.
`db_type`	*string*	não	Tipo SQL/CQL para geração de DDL.

Fonte: Elaborado pelo autor (2026).

No lugar de uma entrada simples, `columns` pode conter um novo mapa de colunas. Esse **aninhamento** só é aceito no MongoDB, onde produz um subdocumento (Listagem 4.9).

Mapeamento automático. Quando `columns` é omitido, a entidade recebe todas as colunas da sua fonte, com os nomes do CSV. Com `"auto_map": true`, as colunas declaradas em `columns` somam-se às automáticas e prevalecem sobre elas: na Listagem 4.11, a entidade `shopping_cart` declara apenas a coluna-chave e recebe as demais automaticamente. Nos dois casos, `csv_columns` limita o mapeamento automático às colunas indicadas, por nome, por posição (a partir de 1) ou por intervalo, como `"1-5"`.

Inferência de tipos. Todos os valores são lidos inicialmente como texto, preservando o conteúdo original do arquivo. Em seguida, a ferramenta classifica cada coluna segundo o tipo predominante entre os seus valores — inteiro, real, data/hora, booleano, texto ou vazia — e converte os valores para o tipo nativo correspondente. Essa classificação também define o tipo de destino (`db_type`) usado na geração de DDL quando a coluna é auto-mapeada, segundo a correspondência da Tabela 4. Para uma coluna declarada sem `db_type`, o PostgreSQL usa `TEXT`, enquanto o Cassandra usa o tipo inferido.

Tabela 4 – Correspondência entre tipo inferido e tipo de destino na geração de DDL.

Tipo inferido	**Tipo de destino (`db_type`)**
inteiro (*integer*)	`BIGINT`
real (*float*)	`NUMERIC`
data/hora (*datetime*)	`TIMESTAMPTZ`
booleano (*boolean*)	`BOOLEAN`
texto (*string*) ou vazia	`TEXT`

Fonte: Elaborado pelo autor (2026).

4.2.2 Particularidades de Cada SGBD no Arquivo de Importação

Os cinco blocos de SGBD usam a mesma definição de entidade, mas cada SGBD acrescenta regras próprias, decorrentes do seu modelo de dados. Esta subseção apresenta essas regras, cada uma com o trecho correspondente do arquivo de importação do cenário; o arquivo completo está no Apêndice C.

PostgreSQL. As entidades são tabelas e só admitem colunas simples, sem aninhamento. O campo `db_type` de cada coluna define o tipo usado na criação da tabela (`CREATE TABLE`) quando `--create-schema` está ativo, e as colunas com `is_key`

formam a chave primária. O mapa `relationships` declara chaves estrangeiras: `from` é a tabela que contém a chave, `to` é a tabela referenciada, `foreign_key` é a coluna de `from` e `references_key` é a coluna de `to` (por padrão, a de mesmo nome). A Listagem 4.8 mostra a entidade `products` e a chave estrangeira que a liga às categorias.

```
"postgres": {
  "entities": {
    "products": {
      "source": "stock",
      "columns": {
        "product_id":   { "is_key": true, "db_type": "BIGINT" },
        "product_name": { "db_type": "TEXT" },
        ...
        "category_id":  { "db_type": "BIGINT" },
        ...
      }
    },
    ...
  },
  "relationships": {
    "product_category": {
      "from": "products",
      "to":   "categories",
      "foreign_key":    "category_id",
      "references_key": "category_id"
    },
    ...
  }
}
```

Listagem 4.8 – Fragmento de importação PostgreSQL com chave estrangeira.

As reticências omitem três colunas de `products` (variante, marca e preço), as outras três entidades (`categories`, `inventory` e `orders`) e a segunda chave estrangeira (`order_product`, de pedidos para produtos).

MongoDB. As entidades são coleções, e o MongoDB é o único SGBD que aceita **aninhamento** em `columns`: uma chave cujo valor é um novo mapa de colunas produz um subdocumento, sem nenhum campo adicional. Na Listagem 4.9, as chaves `category` e `stock` não são colunas do CSV, e sim agrupam outras colunas em dois subdocumentos de cada documento `product_catalog`.

```
"mongodb": {
  "entities": {
    "product_catalog": {
      "source": "stock",
      "columns": {
        "product_id":   {},
        "product_name": {},
        ...
        "category": {
          "category_id": {},
          "category_name": {}
```

```
        },
        "stock": {
          "quantity_available": {},
          "last_restock_date": {}
        }
      }
    }
  }
}
```

Listagem 4.9 – Fragmento de importação MongoDB com subdocumentos.

As reticências omitem as demais colunas cadastrais do produto (variante, marca, descrição, imagem e preço), mapeadas da mesma forma que `product_id` e `product_name`.

Apache Cassandra. As entidades são tabelas com colunas simples, e a chave primária é definida por dois campos próprios: `cassandra_partition`, a lista de colunas da chave de partição, e `cassandra_cluster`, as colunas de agrupamento. A partição é obrigatória na prática, pois a linguagem CQL requer ao menos uma coluna de partição na chave primária; a sua ausência, porém, não é apontada na validação nem pelo `--dry-run`, e só interrompe a importação no momento da gravação no Cassandra. No Cassandra, `is_key` não participa da chave primária. A Listagem 4.10 mostra uma entidade que **une** as quatro fontes em uma tabela de eventos.

```
"cassandra": {
  "entities": {
    "user_activity_log": {
      "source": ["stock", "purchase", "select_product", "add_to_cart"],
      "columns": {
        "user_id":             {},
        "timestamp":           { "schema_column": "event_time" },
        "_source":             { "schema_column": "event_type" },
        "product_id":          {},
        "order_number":        {},
        "selected_product_id": {},
        "shopping_cart_id":    {}
      },
      "cassandra_partition": ["user_id"],
      "cassandra_cluster":   ["timestamp"]
    }
  }
}
```

Listagem 4.10 – Fragmento de importação Cassandra com união de fontes e chave composta.

A pseudo-coluna `_source`, que guarda a fonte de cada linha, é gravada na coluna `event_type`, distinguindo os tipos de evento na tabela; a chave primária é formada pelo usuário (partição) e pelo instante do evento (agrupamento).

Redis. Cada entidade gera um conjunto de pares chave-valor e deve ter **exa-**

tamente uma coluna com `is_key`: o valor dessa coluna, tal como aparece no CSV, torna-se a chave Redis, e os demais campos mapeados compõem o valor, armazenado em JSON. Na Listagem 4.11, `shopping_cart` declara apenas a chave e usa `auto_map` para receber as demais colunas da fonte `add_to_cart`; `user_session` usa como chave o identificador do usuário e renomeia `timestamp` para `last_seen`, registrando a última navegação de cada usuário.

```
"redis": {
  "entities": {
    "shopping_cart": {
      "source": "add_to_cart",
      "auto_map": true,
      "columns": {
        "shopping_cart_id": { "is_key": true }
      }
    },
    "user_session": {
      "source": "select_product",
      "columns": {
        "user_id":    { "is_key": true },
        "user_name":  {},
        "user_email": {},
        "timestamp":  { "schema_column": "last_seen" }
      }
    }
  }
}
```

Listagem 4.11 – Fragmento de importação Redis (e-commerce), com mapeamento automático.

Neo4j. Cada entidade é um rótulo (*label*) de nó e deve ter exatamente uma coluna com `is_key`, usada para identificar o nó na operação `MERGE` da linguagem Cypher, que só cria o nó se ele ainda não existir. O mapa `relationships` declara arestas: `from` e `to` são os rótulos de origem e de destino, `type` é o tipo da aresta e `columns`, opcional, lista as propriedades da aresta. Na Listagem 4.12, o nó `User` lê da fonte `purchase` (os compradores) e o nó `Product`, da fonte `stock`; a aresta `PURCHASED` liga cada comprador ao produto comprado, com quantidade, preço e nota da avaliação.

```
"neo4j": {
  "entities": {
    "User": {
      "source": "purchase",
      "columns": {
        "user_id":    { "is_key": true },
        "user_name":  {},
        "user_email": {}
      }
    },
    "Product": {
      "source": "stock",
      "columns": {
        "product_id":    { "is_key": true },
```

```
        "product_name":  {},
        "product_brand": {}
      }
    }
  },
  "relationships": {
    "PURCHASED": {
      "from": "User",
      "to":   "Product",
      "type": "PURCHASED",
      "columns": {
        "order_number": { "is_key": true },
        "quantity":     {},
        "price":        {},
        "rating":       {}
      }
    }
  }
}
```

Listagem 4.12 – Fragmento de importação Neo4j (nós e aresta).

4.2.3 O Arquivo de Conexão

O arquivo de conexão diz *onde* alcançar cada SGBD, sem nenhuma informação de mapeamento. A Figura 9 mostra a estrutura do seu schema, também com os campos obrigatórios marcados com asterisco: a raiz, um bloco por SGBD e, dentro de cada bloco, os parâmetros de acesso. A Tabela 5 resume a finalidade de cada parte.

Tabela 5 – Partes do arquivo de conexão.

Parte	**Onde**	**Finalidade**	**Obrigatória?**
`version`	raiz	versão do formato do arquivo (atualmente, 1)	sim
`postgres`, `mongodb`, `cassandra`, `redis`, `neo4j`	raiz	dar acesso a um SGBD	não; ao menos os SGBDs de destino
`connection`	bloco de SGBD	parâmetros de acesso	sim
`schema`	bloco PostgreSQL	schema de destino das tabelas	não; padrão: `public`
`start`	bloco de SGBD	como iniciar o SGBD	não

Fonte: Elaborado pelo autor (2026).

A Listagem 4.13 mostra o início do arquivo de conexão do cenário, com os blocos do PostgreSQL e do MongoDB. As reticências marcam os descritores `start`, tratados na Seção 4.2.3.3, e os blocos dos demais SGBDs; o arquivo completo está no Apêndice C.

```
{
  "version": 1,
  "postgres": {
    "connection": {
      "host": "127.0.0.1", "port": 5432,
      "database": "ecommerce",
      "user": "postgres", "password": "postgres"
    },
    "schema": "public",
    ...
  },
  "mongodb": {
    "connection": {
      "uri": "mongodb://127.0.0.1:27017",
      "database": "ecommerce"
    },
    ...
  },
  ...
}
```

Listagem 4.13 – Início do arquivo de conexão do cenário (`dbms_config.json`).

4.2.3.1 Raiz e blocos de SGBD

A raiz exige o campo `version` e aceita um bloco por SGBD; como no arquivo de importação, chaves desconhecidas são rejeitadas. Todo SGBD que aparece no arquivo de importação precisa ter o seu bloco aqui; caso contrário, a importação é recusada antes de começar (Seção 4.2.4). Cada bloco presente deve conter o descritor `connection`.

4.2.3.2 Parâmetros de conexão

Os campos de `connection` dependem do SGBD, pois seguem os parâmetros do *driver* de cada um. A Tabela 6 lista esses campos, os obrigatórios e os valores usados quando um campo opcional é omitido.

No MongoDB e no Neo4j, o endereço é uma URI no formato do respectivo *driver* (`mongodb://...` e `bolt://...`). A URI do MongoDB pode incluir usuário e senha; no Neo4j, as credenciais vão nos campos `user` e `password`. No Cassandra, `hosts` é uma lista, pois um *cluster* pode ter vários nós, e `request_timeout` é o tempo máximo, em segundos, de cada requisição.

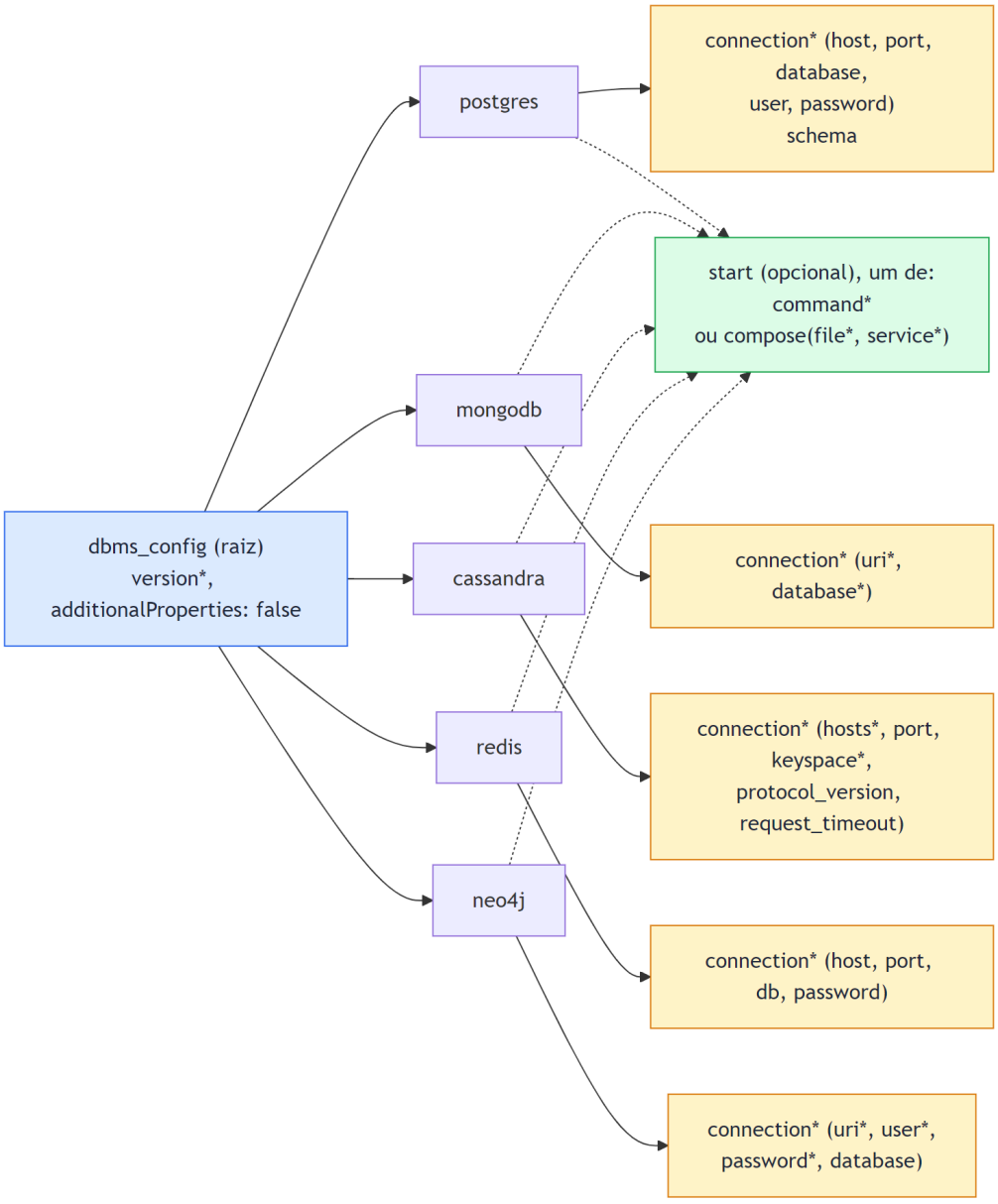


Figura 9 – Estrutura do JSON Schema de conexão (`dbms_config`).
Fonte: Elaborado pelo autor (2026).

Tabela 6 – Campos do descritor `connection` por SGBD.

SGBD	Campos	Obrigatórios	Padrões
PostgreSQL	`host`, `port`, `database`, `user`, `password`	—	`127.0.0.1`, `5432`, `postgres`, `postgres`, senha vazia
MongoDB	`uri`, `database`	`uri`, `database`	—
Cassandra	`hosts`, `port`, `keyspace`, `protocol_version`, `request_timeout`	`hosts`, `keyspace`	porta `9042`; `request_timeout` de 30 s; protocolo negociado pelo *driver*
Redis	`host`, `port`, `db`, `password`	—	`127.0.0.1`, `6379`, `0`, sem senha
Neo4j	`uri`, `user`, `password`, `database`	`uri`, `user`, `password`	`database`: o banco padrão do servidor

Fonte: Elaborado pelo autor (2026).

4.2.3.3 Inicialização dos SGBDs

O descritor `connection` diz *onde* alcançar cada SGBD, mas não o que fazer quando ele não responde. Para esse caso, cada bloco de SGBD aceita um descritor opcional `start`, que registra *como* iniciá-lo, em uma de duas formas mutuamente exclusivas: `command`, uma linha de comando para um SGBD instalado na própria máquina, ou `compose`, que indica um arquivo do Docker Compose e o serviço correspondente. A ferramenta **nunca executa** esse comando: ela o exibe, pronto para ser copiado para um terminal, quando a verificação descrita na Seção 4.3.1 encontra o SGBD fora do ar. A escolha é deliberada: iniciar um serviço do sistema operacional costuma exigir privilégios de administrador, que a ferramenta não tem nem deve pedir. A Listagem 4.14 mostra dois blocos do arquivo de conexão de exemplo para Windows, um em cada forma.

```
"postgres": {
  "connection": { ... },
  "schema": "public",
  "start": { "command": "net start postgresql-x64-16" }
},
"cassandra": {
  "connection": { ... },
  "start": {
    "compose": {
      "file":    "../../docker-compose.yml",
      "service": "cassandra"
    }
  }
}
```

Listagem 4.14 – Descritores `start` nas duas formas (`dbms_config_windows.json`).

Na listagem, o conteúdo dos descritores `connection`, igual ao do arquivo reproduzido no Apêndice C, foi omitido. O PostgreSQL, instalado como serviço do Windows, é iniciado por `net start`; o Apache Cassandra, que não roda nativamente no Windows, é descrito por um serviço do Docker Compose, cujo arquivo tem caminho relativo à pasta do próprio arquivo de conexão. O cenário de exemplo traz três arquivos de conexão com as mesmas conexões e descritores `start` distintos: `dbms_config.json`, todo em Docker Compose; `dbms_config_windows.json`, com serviços do Windows; e `dbms_config_linux.json`, com o `systemctl`. Como os nomes de serviço dependem de cada instalação, os comandos desses arquivos são exemplos a ajustar.

4.2.4 Validação, Consistência entre Arquivos e Fluxo de Carregamento

A validação ocorre em **três camadas** antes de qualquer gravação nos SGBDs. A primeira é a validação por **JSON Schema**: cada arquivo é validado contra o seu schema (sintaxe, tipos e propriedades permitidas) — `import_config.schema.json` para o arquivo de importação e `dbms_config.schema.json` para o arquivo de conexão. A segunda é a **consistência entre arquivos** (`config_parser.merge_configs`): todo SGBD referenciado no arquivo de importação deve estar declarado no arquivo de conexão. A terceira é a **validação cruzada com os dados**: ao carregar as fontes (`sources.load_sources`) e vincular cada entidade à sua fonte (`mapping_resolver`), confere-se se as colunas referenciadas existem, se os SGBDs de modelo plano não usam `columns` aninhados, se as colunas de partição e agrupamento do Cassandra existem, se os filtros são coerentes e se as chaves estrangeiras do PostgreSQL e as arestas do Neo4j referenciam entidades válidas (`validation.validate_backend_entities`).

Cada camada levanta uma exceção específica da hierarquia `BusinessException`, o que torna o diagnóstico preciso: `ConfigError` para erros de schema ou de consistência entre os arquivos; `SourceError` para problemas com os arquivos de fonte (por exemplo, um CSV inexistente ou uma coluna de origem vazia no modo combinado); `MappingError` para incoerências entre o mapeamento e os dados; e `ImportExecutionError` para falhas na conexão ou na gravação em um SGBD. Qualquer falha nas três primeiras camadas **impede** o início da importação. A Figura 10 apresenta o fluxo completo de carregamento, com os pontos de falha de cada camada e a convergência dos dois arquivos em uma configuração única, à qual os dados das fontes se juntam apenas na validação cruzada (camada 3). O diagrama evidencia por que a separação em dois arquivos isola falhas: um `dbms_config.json` inválido é detectado independentemente do `import_config.json`, sem necessidade de ler os CSVs.

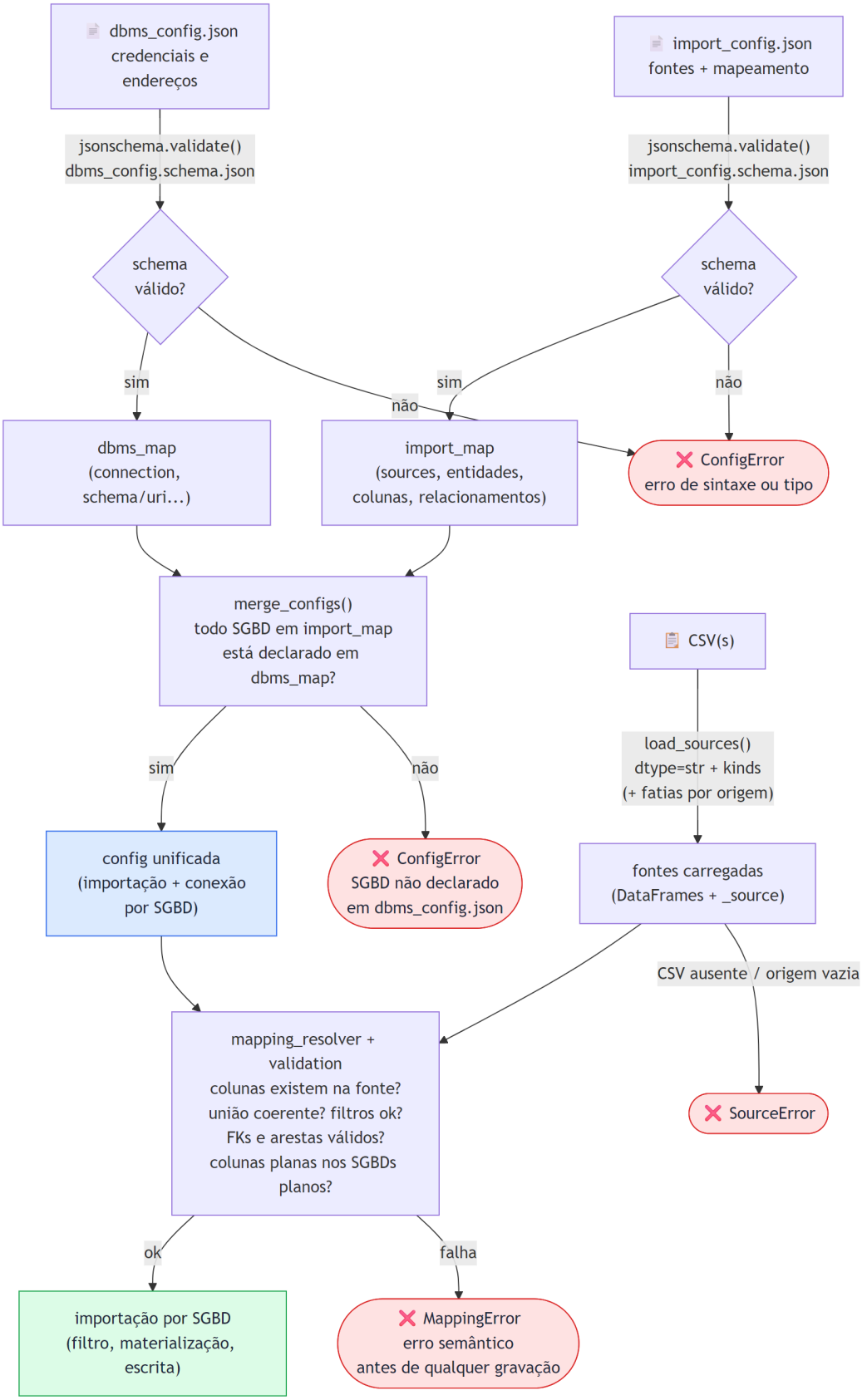


Figura 10 – Fluxo de carregamento e validação das entradas da ferramenta.

Fonte: Elaborado pelo autor (2026).

4.3 ALGORITMO DE EXECUÇÃO DA IMPORTAÇÃO

Definidos o formato de configuração e as validações, esta seção descreve como a ferramenta executa a importação propriamente dita, desde o comando de linha de comando apresentado na seção 4.1 até a persistência em cada SGBD. A Figura 11 apresenta o fluxo completo de execução; cada etapa do diagrama é explicada textualmente na sequência.

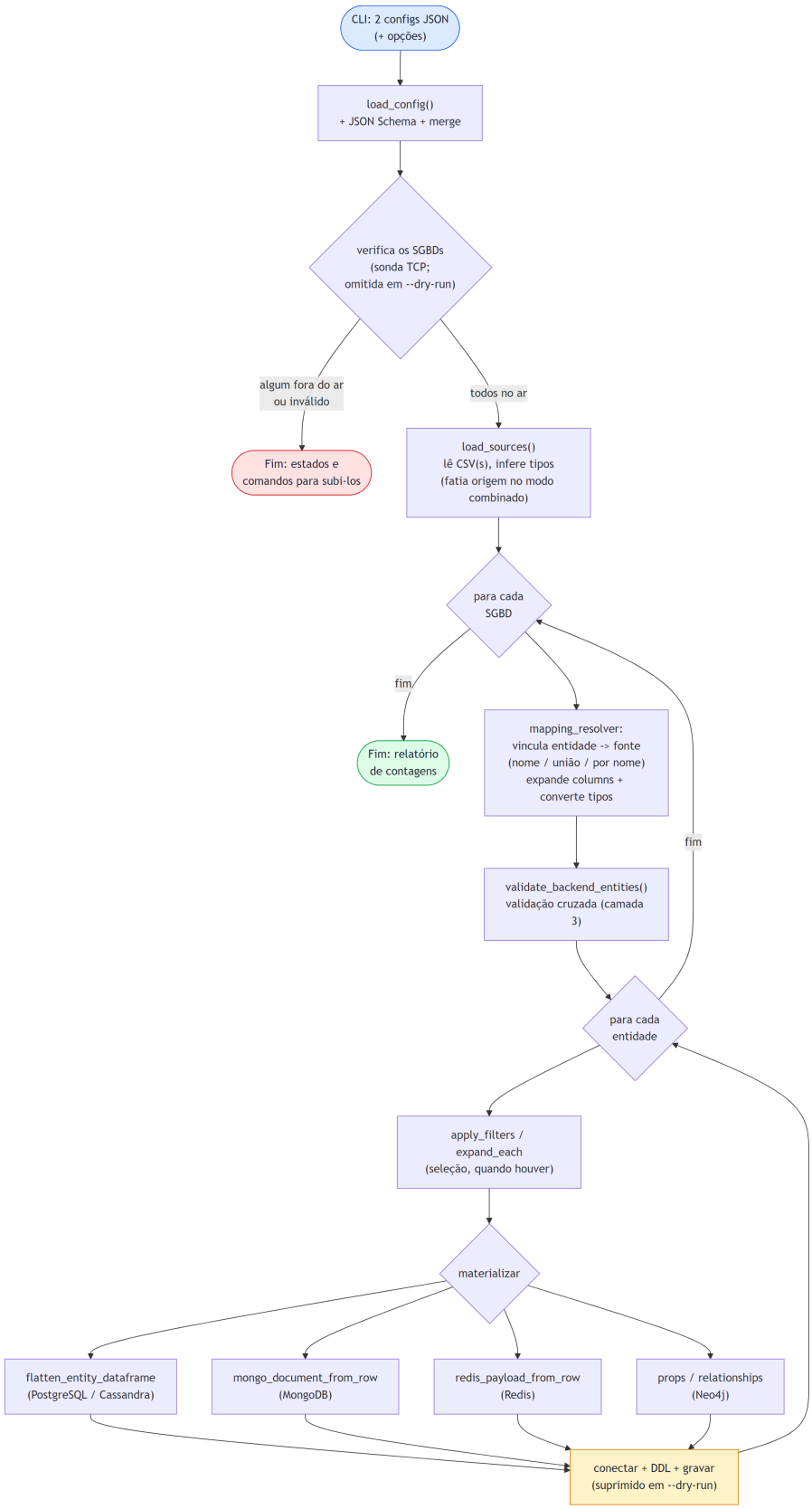


Figura 11 – Algoritmo de execução da importação PolyglotImportCSV.
Fonte: Elaborado pelo autor (2026).

A execução parte da **entrada** fornecida pelo usuário na linha de comando:

os caminhos dos dois arquivos JSON de configuração (importação e conexão), eventuais sobrescritas de fonte (`--source`) e as opções de execução `--dry-run`, `--create-schema`/`--no-create-schema` e `--only`, descritas na seção 4.1.

Na etapa de **carregamento da configuração**, os dois arquivos JSON são lidos e validados contra os seus respectivos JSON Schemas (camada 1 da seção 4.2.4); em seguida, verifica-se a consistência entre os arquivos — todo SGBD referenciado no arquivo de importação deve estar declarado no arquivo de conexão (camada 2) — e as configurações são mescladas em uma estrutura única de execução.

Em uma importação real, segue-se a **verificação dos SGBDs** de destino, detalhada na Seção 4.3.1: se algum deles não responder, a execução termina antes da leitura de qualquer CSV, com a indicação de como iniciá-lo.

Na etapa de **carregamento das fontes**, cada CSV declarado no bloco `sources` é lido uma única vez. Todos os valores são lidos inicialmente como texto, preservando o conteúdo original do arquivo e evitando conversões prematuras de tipo (por exemplo, em datas com fuso horário). Quando a fonte é combinada, a sua primeira coluna é interpretada como a origem e cada valor distinto dá origem a uma fonte derivada; a pseudo-coluna `_source` é preenchida em ambos os modos. Em seguida, a ferramenta infere o tipo de cada coluna de cada fonte, informação que alimenta tanto a conversão dos valores para tipos nativos quanto a geração de DDL para colunas auto-mapeadas.

Validadas as entradas, inicia-se o núcleo do algoritmo, que percorre os SGBDs presentes na configuração (excluídos os não selecionados por `--only`) e, para cada um, **vincula** as suas entidades às respectivas fontes (`mapping_resolver`): resolve o `source` de cada entidade — nome único, união de fontes ou vínculo pelo nome —, expande o mapeamento de colunas (manual, automático ou híbrido) e converte os valores da fonte para tipos nativos. Realiza-se então a validação cruzada da camada 3 (seção 4.2.4). Qualquer falha, em qualquer camada, encerra a execução com uma mensagem de erro antes que qualquer dado seja gravado nos SGBDs.

Para cada entidade vinculada, a ferramenta **aplica os filtros** declarados na configuração, quando houver — operação que corresponde à *seleção* da álgebra relacional: apenas as linhas que satisfazem os predicados participam da entidade. Havendo filtro com o operador `each`, a entidade é adicionalmente **particionada** pelos valores distintos da coluna indicada — um particionamento horizontal em que cada valor dá origem a um destino próprio, identificado por sufixo.

Em seguida, na **materialização**, cada linha selecionada é convertida na estrutura do modelo de dados de destino, segundo os conceitos revisados no capítulo 2. Para os SGBDs de organização tabular (o relacional PostgreSQL e o colunar Cassandra), a conversão combina três operações também descritas pela álgebra relacional:

projeção (somente as colunas mapeadas em `columns` são mantidas), *renomeação* (aplicação de `schema_column`) e *eliminação de duplicatas* com base nas colunas de chave (`is_key`), de modo que cada tabela receba apenas tuplas únicas. Para o modelo de documentos (MongoDB), os valores da linha são agregados em um documento cuja hierarquia reproduz o aninhamento declarado em `columns`, materializando a representação desnormalizada característica desse modelo. Para o modelo chave-valor (Redis), cada linha dá origem a um par em que a chave é o valor da coluna marcada com `is_key` e o valor é a representação textual (JSON) dos demais campos mapeados. Para o modelo de grafo (Neo4j), cada linha dá origem a um nó com as propriedades mapeadas; na sequência, as arestas declaradas em `relationships` conectam os nós criados, com semântica de mesclagem — a aresta só é criada se ainda não existir, evitando duplicação quando a mesma relação aparece em várias linhas.

Por fim, para cada entidade materializada, a ferramenta **conecta-se** ao SGBD (etapa suprimida quando `--dry-run` está ativo), **cria as estruturas de armazenamento** quando `--create-schema` está ativo (tabelas no PostgreSQL; *keyspace* e tabelas no Cassandra) e **grava** os registros por meio da operação de escrita própria de cada tecnologia. A **saída** da execução é um relatório com as contagens de registros gravados por entidade e por SGBD, que permite conferir o resultado da carga.

4.3.1 Verificação dos SGBDs

Antes de ler as fontes, toda importação real verifica se os SGBDs de destino — os presentes no arquivo de importação, filtrados por `--only` — respondem. A verificação abre uma conexão TCP com cada endereço declarado no arquivo de conexão e a fecha em seguida, sem autenticar. As sondas correm em paralelo, com limite de dois segundos cada, de modo que o tempo total não cresce com o número de SGBDs. Cada SGBD termina em um de três estados: **no ar**, quando algum de seus endereços responde (no Cassandra, basta um nó); **fora do ar**, quando nenhum responde; e **conexão inválida**, quando o próprio *driver* rejeita a configuração, como em uma URI com esquema desconhecido. Um único SGBD fora do ar ou com conexão inválida **interrompe** a execução antes da leitura dos CSVs, o que evita uma carga parcial: sem a verificação, os SGBDs anteriores na ordem de importação receberiam dados antes que a falha do seguinte fosse descoberta.

Os endereços não são extraídos por um analisador próprio. Para o MongoDB e o Neo4j, cuja conexão é descrita por uma URI, a ferramenta constrói o cliente do próprio *driver* sem conectar e obtém dele os endereços; assim, uma URI aceita pelo *driver* nunca é recusada pela verificação, e uma recusada nunca passa. Para o PostgreSQL, seguem-se as regras da biblioteca cliente `libpq`, inclusive listas de hosts e *sockets* Unix.

Quando algum SGBD não responde, a ferramenta mostra, abaixo da tabela de estados, o que fazer: para cada descritor `command`, o próprio comando; para os descritores `compose` de um mesmo arquivo, um único `docker compose up`; e, para um SGBD sem `start`, uma orientação genérica. A opção `--check-dbms` executa apenas essa etapa e encerra, com código de saída zero quando todos os SGBDs respondem. A Tabela 7 e a Listagem 4.15 reproduzem uma verificação feita com o arquivo de conexão de exemplo para Windows, com o MongoDB e o Neo4j fora do ar.

Tabela 7 – Estados reportados por `--check-dbms` com dois SGBDs fora do ar.

SGBD	Endereço	Estado
postgres	127.0.0.1:5432	up
mongodb	127.0.0.1:27017	down
cassandra	127.0.0.1:9042	up
redis	127.0.0.1:6379	up
neo4j	127.0.0.1:7687	down

Fonte: Elaborado pelo autor (2026).

A Listagem 4.15 mostra os comandos que a ferramenta lista abaixo da tabela para iniciar os dois SGBDs fora do ar, retirados dos descritores `start` do arquivo de conexão.

```
To start the DBMS that are down, run in a terminal:
  net start MongoDB
  net start neo4j
Service commands (net start, systemctl) may need an administrator
   terminal or sudo.
```

Listagem 4.15 – Comandos de inicialização mostrados por `--check-dbms`.

4.3.2 O Cenário de Exemplo e os Arquivos de Dados

Para concretizar o algoritmo, esta subseção e a seguinte acompanham as linhas do cenário de e-commerce apresentado na seção 4.2 (Tabela 1) pelo núcleo do algoritmo; a execução completa do cenário é demonstrada na seção 4.3.4. O exemplo adota o modo multifonte, em que cada tipo de evento tem o seu próprio CSV.

A Tabela 8 apresenta as oito linhas da fonte `stock` (`ecommerce_stock.csv`), que servem de fio condutor do exemplo. Para caber na página, a tabela resume as colunas de identificação, cujos valores seguem padrões regulares no arquivo: o produto `Product`N tem `product_id` N e pertence à categoria `Categoria`N (`category_id` N); o usuário `user`N é identificado por um `user_id` alfanumérico (por exemplo, `7fd7eee7-...` para `user1`), tem e-mail `user`N`@example.com` e um endereço próprio, detalhado no

Apêndice B; a descrição e a imagem do produto seguem padrões análogos. Todos os eventos de estoque ocorrem em 02/11/2023, e a coluna `last_restock_date` repete o instante do próprio evento.

Tabela 8 – Linhas da fonte `stock` (`ecommerce_stock.csv`, colunas de identificação resumidas).

Hora	Usuário	Produto	Variante	Marca	Categoria	Qtd.	Preço
03:30	user1	Product2	blue	Microsoft	Categoria2	2	50.5
06:45	user2	Product3	green	Google	Categoria3	1	52.07
09:15	user2	Product9	black	Pepsi	Categoria9	67	32.78
12:00	user8	Product8	white	Google	Categoria8	60	46.79
14:30	user9	Product4	yellow	Apple	Categoria4	82	55.94
17:15	user9	Product5	red	Samsung	Categoria5	57	34.21
20:00	user9	Product6	orange	Coca-Cola	Categoria6	64	51.98
22:30	user9	Product7	grey	Audi	Categoria7	1	100000

Fonte: Elaborado pelo autor (2026), a partir de `data/ecommerce/ecommerce_stock.csv`.

4.3.3 A Importação em Cada SGBD

Apresentados os arquivos de dados, esta subseção acompanha o percurso dessas linhas pelo núcleo do algoritmo, SGBD a SGBD. Em cada caso estão em jogo as entradas da ferramenta: as linhas da fonte pertinente, o trecho correspondente do arquivo de importação e o arquivo de conexão — trechos reproduzidos nas listagens da seção 4.2 e, na íntegra, no Apêndice C. O que varia de um SGBD para outro é a estrutura de dados produzida como saída, apresentada a seguir.

PostgreSQL. O arquivo de importação declara quatro entidades (Listagem 4.8). A entidade `inventory` lê da fonte `stock` e projeta quatro colunas, com o `product_id` como chave: o resultado é a relação da Tabela 9, criada e gravada no banco `ecommerce` declarado no arquivo de conexão (Listagem 4.13). As entidades `categories` e `products` também leem de `stock`, e a eliminação de duplicatas pela chave reduz o resultado aos oito produtos e oito categorias distintos; a entidade `orders` lê da fonte `purchase` (as oito compras). As chaves estrangeiras declaradas em `relationships` ligam produtos a categorias e pedidos a produtos.

Tabela 9 – Relação `inventory` materializada no PostgreSQL a partir da fonte `stock`.

product_id	quantity_available	last_restock_date	price
2	2	02/11/2023 03:30	50.5
3	1	02/11/2023 06:45	52.07
9	67	02/11/2023 09:15	32.78
8	60	02/11/2023 12:00	46.79
4	82	02/11/2023 14:30	55.94
5	57	02/11/2023 17:15	34.21
6	64	02/11/2023 20:00	51.98
7	1	02/11/2023 22:30	100000

Fonte: Elaborado pelo autor (2026).

MongoDB. A entidade `product_catalog` (Listagem 4.9) também lê da fonte `stock`, mas materializa cada linha como um documento cuja hierarquia reproduz o aninhamento declarado em `columns`. A Listagem 4.16 mostra o documento gerado a partir da primeira linha de estoque. Os valores preservam a forma textual lida do CSV, coerente com a política de carregamento descrita no início desta seção; os subdocumentos `category` e `stock` agrupam, sem repetição de linhas, o que no PostgreSQL ficou distribuído em três tabelas ligadas por chaves estrangeiras — a representação desnormalizada característica do modelo de documentos.

```
{
  "product_id": "2",
  "product_name": "Product2",
  "product_variant": "blue",
  "product_brand": "Microsoft",
  "product_description": "Description1",
  "product_image": "https://example.com/image1.jpg",
  "price": "50.5",
  "category": {
    "category_id": "2",
    "category_name": "Categoria2"
  },
  "stock": {
    "quantity_available": "2",
    "last_restock_date": "2023-11-02 03:30:00Z"
  }
}
```

Listagem 4.16 – Documento gerado na coleção `product_catalog` para a primeira linha de estoque.

Cassandra. A entidade `user_activity_log` (Listagem 4.10) une as quatro fontes: os 32 eventos alimentam a tabela, com `timestamp` renomeado para `event_time`

e a pseudo-coluna `_source` mapeada para `event_type`. A chave primária composta — partição por `user_id`, agrupamento por `timestamp` — faz com que todos os eventos de um mesmo usuário fiquem na mesma partição, ordenados no tempo. A Tabela 10 mostra a partição do usuário `user1`: os seus quatro eventos, um de cada fonte, com as colunas de referência preenchidas conforme a origem — cada fonte preenche apenas o subconjunto de colunas pertinente ao seu evento, agora a serviço de uma consulta típica do modelo colunar ("o histórico de atividades de um usuário").

Tabela 10 – Partição do usuário `user1` na tabela `user_activity_log` (Cassandra).

event_time	event_type	product_id	order_number	selected_product_id	shopping_cart_id
02/11 03:30	stock	2	—	—	—
15/11 15:09	select_product	—	—	11	—
15/11 15:17	add_to_cart	—	—	—	user:7fd7...:cart
15/11 15:46	purchase	2	ORD1	—	—

Fonte: Elaborado pelo autor (2026).

Redis. As duas entidades desse SGBD (Listagem 4.11) leem de fontes distintas: `shopping_cart` da fonte `add_to_cart` e `user_session` da fonte `select_product`. Cada linha torna-se um par chave-valor: a chave é o valor da coluna marcada com `is_key` e o valor é a representação textual (JSON) dos demais campos mapeados. A Listagem 4.17 mostra os dois pares gerados a partir dos eventos de `user1`; ao todo, são oito carrinhos e oito sessões. Como `shopping_cart` usa mapeamento automático, o seu valor reúne todas as demais colunas da fonte `add_to_cart`; já `user_session` mapeia apenas os campos declarados. Note-se que o padrão `user:...:cart` da chave do carrinho vem do próprio CSV (coluna `shopping_cart_id`), e não de alguma convenção da ferramenta.

```
chave: user:7fd7eee7-4675-4f56-ad81-108da6cecebc:cart
valor: {"timestamp": "2023-11-15 15:17:30Z", "user_id": "7fd7...",
        "user_name": "user1", "user_email": "user1@example.com",
        "cart_product_id": "3", "cart_quantity": "1"}
chave: 7fd7eee7-4675-4f56-ad81-108da6cecebc
valor: {"user_name": "user1", "user_email": "user1@example.com",
        "last_seen": "2023-11-15 15:09:30Z"}
```

Listagem 4.17 – Pares chave-valor gerados no Redis a partir dos eventos de `user1`.

Neo4j. As entidades e a aresta da Listagem 4.12 produzem um grafo de propriedades: os oito produtos da fonte `stock` dão origem aos nós `Product`; os oito com-

pradores da fonte `purchase` dão origem aos nós `User` e às oito arestas `PURCHASED`, identificadas pelo número do pedido e decoradas com quantidade, preço e nota da avaliação. A semântica de mesclagem garante que nós e arestas não se dupliquem quando a mesma chave reaparece. A Listagem 4.18 representa, na notação usual de grafos de propriedades, o fragmento do grafo derivado da primeira compra (`ORD1`).

```
(:User {user_id: "7fd7eee7-...", user_name: "user1",
        user_email: "user1@example.com"})
    -[:PURCHASED {order_number: "ORD1", quantity: "1",
                  price: "50.5", rating: "2"}]->
(:Product {product_id: "2", product_name: "Product2",
           product_brand: "Microsoft"})
```

Listagem 4.18 – Fragmento do grafo materializado no Neo4j para o pedido `ORD1`.

A Tabela 11 consolida o percurso, tomando por referência a primeira linha da fonte `stock` (Tabela 8, o produto `Product2`): cada célula da coluna “Saída materializada” decorre diretamente das entidades, do vínculo com a fonte e das `columns` declaradas no arquivo de importação.

Tabela 11 – Materialização de uma linha da fonte `stock` nos diferentes SGBDs.

SGBD	**Entidade**	**Fonte**	**Saída materializada**
PostgreSQL	`inventory` (a linha também alimenta `products` e `categories`)	`stock`	Linha da Tabela 9: `product_id`, `quantity_available`, `last_restock_date`, `price`.
MongoDB	`product_catalog`	`stock`	Documento da Listagem 4.16, com subdocumentos `category` e `stock` aninhados.
Cassandra	`user_activity_log`	`stock` ∪ demais	Registro com `timestamp` e `_source` renomeados para `event_time` e `event_type`; chave primária composta `(user_id, timestamp)`.
Redis	—	—	As entidades Redis leem de outras fontes; esta linha não alimenta o Redis.
Neo4j	nó `Product`	`stock`	Nó `:Product` da Listagem 4.18, identificado por `product_id`.

Fonte: Elaborado pelo autor (2026).

Assim, **fontes heterogêneas** alimentam destinos igualmente heterogêneos: tabelas relacionais, documentos aninhados, *log* colunar com renomeação de colunas e união de fontes, pares chave-valor e um grafo de propriedades — materializando a persistência poliglota descrita no capítulo 2. As contagens observadas neste percurso (oito registros por entidade de estoque e 32 no registro de atividades) reaparecem na verificação prévia da execução, apresentada a seguir.

4.3.4 Exemplo de Uso e Evidência de Execução (Cenário E-commerce)

Esta seção demonstra o uso da ferramenta em uma execução completa do cenário de e-commerce que acompanha o código-fonte. Todos os arquivos de entrada encontram-se no diretório `data/ecommerce/` do repositório da ferramenta: os

quatro CSVs de fonte (`ecommerce_stock.csv`, `ecommerce_purchase.csv`, `ecommerce_select_product.csv` e `ecommerce_add_to_cart.csv`), com oito linhas cada; o arquivo de importação `import_config.json`, cujos fragmentos por SGBD foram apresentados nas Listagens 4.8 a 4.12; e o arquivo de conexão `dbms_config.json`, cujos fragmentos acompanham as mesmas subseções — ambos reproduzidos na íntegra no Apêndice C. Além deles, o arquivo `docker-compose.yml`, na raiz do repositório, define os contêineres dos cinco SGBDs usados no exemplo. A execução usa o comando apresentado na seção 4.1, instanciado com esses arquivos:

```
python -m polyglotimportcsv \
  --config data/ecommerce/import_config.json \
  --dbms-config data/ecommerce/dbms_config.json \
  --dry-run
```

Com a *flag* `--dry-run`, a ferramenta executa todas as validações e a materialização, mas não abre conexão com nenhum SGBD — o *driver* Apache Cassandra, por exemplo, sequer é importado, o que permite o *dry-run* em ambientes onde a sua extensão C não está disponível. O resultado são as contagens de registros por destino mostradas na Tabela 12, úteis para revisar o mapeamento antes da carga real.

Tabela 12 – Contagens reportadas pelo modo `--dry-run` no cenário e-commerce.

SGBD	**Destino**	**Registros (após projeção/deduplicação)**
PostgreSQL	`categories`	8
PostgreSQL	`products`	8
PostgreSQL	`inventory`	8
PostgreSQL	`orders`	8
MongoDB	`product_catalog`	8 documentos
Cassandra	`user_activity_log`	32 linhas (união das 4 fontes)
Redis	`shopping_cart`	8
Redis	`user_session`	8
Neo4j	nó `User`	8
Neo4j	nó `Product`	8
Neo4j	aresta `PURCHASED`	8 (uma por linha da fonte `purchase`)

Fonte: Elaborado pelo autor (2026).

Para a importação real, basta subir os contêineres com `docker compose up -d` e repetir o mesmo comando sem a *flag* `--dry-run`: os mesmos volumes da Tabela 12 são então persistidos nos cinco SGBDs configurados, materializando o cenário poliglota da Seção 2.1. Alternativamente, o script `run_example.sh`, na raiz do repositório, automatiza

o fluxo completo: lê o `dbms_config.json`, sobe apenas os contêineres dos SGBDs ali declarados e, em seguida, executa o *dry-run* e a importação real com `--create-schema`.

Complementarmente à evidência de execução, testes automatizados (`pytest`) cobrem a validação por schema, a separação e mesclagem dos dois arquivos de configuração, o carregamento das fontes (multifonte e combinada), o mapeamento de colunas (`csv_column`, `schema_column`, aninhamento, mapeamento automático), a união de fontes, a inferência e conversão de tipos, os filtros e *smoke tests* de validação com *dry-run*. Um teste dedicado verifica ainda a **equivalência entre os modos** multifonte e combinado, assegurando que ambos produzem a mesma carga.

4.4 OBSERVABILIDADE E MÉTRICAS

Importar dados para cinco SGBDs em uma só execução exige que o usuário consiga **acompanhar** o que ocorre e **diagnosticar** falhas sem recorrer ao código-fonte. Para isso, a ferramenta incorpora uma camada de observabilidade, construída sobre a biblioteca `rich`, e um subsistema de métricas de desempenho. Além de servir ao uso cotidiano, esse subsistema é o **instrumento de medição** empregado na avaliação de desempenho do Capítulo 5.

Relato da execução. Cada execução emite, no terminal, um relato estruturado: um cabeçalho com o modo (importação ou *dry-run*), as fontes carregadas com as suas contagens de linhas e colunas, uma seção por SGBD com o progresso de cada entidade e um resumo final. O nível de detalhe das mensagens é ajustável por `--log-level` (`DEBUG`, `INFO`, `WARNING` ou `ERROR`; padrão `INFO`). Independentemente do nível escolhido para o terminal, a ferramenta grava um **arquivo de *log* de sessão** sempre em nível `DEBUG`, em `logs/polyglotimportcsv_<carimbo de tempo>.log`, preservando o rastro completo de cada execução para diagnóstico posterior; esse comportamento pode ser desativado ou redirecionado por variáveis de ambiente. Para inspeção dos dados, a ferramenta exibe as primeiras linhas de cada entidade — 50, por padrão, número ajustável por `--sample N` —, acompanhadas da contagem total; `--show-data` exibe todas as linhas e `--no-data` suprime a exibição. A amostra vale nos dois modelos de execução da seção 4.5.2: no modelo em fluxo, as linhas são retidas à medida que os lotes são gravados, até o tamanho da amostra, de modo que a exibição não compromete o consumo limitado de memória.

Métricas de desempenho. Ao longo da execução, a ferramenta coleta métricas por **fase** de processamento de cada entidade, distinguindo quatro fases: **leitura** (`read`, o carregamento das fontes), **seleção** (`filter`, a aplicação de filtros), **mapeamento** (`map`, o vínculo com a fonte e a conversão de tipos) e **escrita** (`write`, a persistência no SGBD). Para cada fase registram-se o número de linhas processadas e o tempo

decorrido, do que se deriva a vazão em linhas por segundo. Ao final, um quadro-resumo apresenta essas métricas no terminal. Quando a *flag* `--benchmark` está ativa, a ferramenta grava as métricas em disco — um arquivo JSON com carimbo de tempo e uma linha acrescentada a um CSV histórico, no diretório `benchmarks/` —, acompanhadas de metadados do ambiente de execução (versão do Python, sistema operacional e tamanho de cada fonte). Além do tempo por fase, o instrumento registra o **pico de memória** consumido durante a execução, coletado com a biblioteca `tracemalloc`; essa medida é o que permite comparar, no Capítulo 5, o custo de memória dos dois modelos de execução apresentados na seção seguinte. Essa gravação estruturada é o que viabiliza a coleta sistemática de resultados reproduzíveis, explorada no Capítulo 5.

4.5 ESTRATÉGIA DE ESCRITA E MODELOS DE EXECUÇÃO

O desempenho de uma importação depende de dois aspectos do projeto da ferramenta. O primeiro é a **estratégia de escrita**: a forma como os valores lidos são **convertidos** aos tipos nativos e **gravados** no SGBD, linha a linha ou em lote. O segundo é o **modelo de execução** (`--execution`), que decide se o arquivo é carregado por inteiro na memória antes de ser gravado ou se é processado em fluxo, com consumo de memória limitado. Ambos são os fatores controlados na avaliação de desempenho do Capítulo 5; esta seção descreve o que cada um significa em termos de projeto e implementação.

4.5.1 Evolução da Estratégia de Escrita

A primeira implementação da ferramenta, desenvolvida na primeira etapa deste Trabalho de Conclusão de Curso (TCC1), adotava a estratégia aqui chamada de **ingênua** (*naive*): os registros eram percorridos **um a um**, cada célula era convertida ao tipo nativo por uma chamada individual e cada linha era gravada no SGBD por um comando próprio. Esse caminho é simples, mas paga, por linha, o custo de uma conversão isolada e de uma ida e volta à rede (e, quando aplicável, de uma confirmação de transação).

No TCC2, a ferramenta evoluiu para a estratégia **em lote** (*batch*), que amortiza esses dois custos processando os dados em blocos. A conversão de tipos passou a ser **vetorizada**: realizada uma vez por coluna, sobre todo o conjunto de valores, em vez de célula a célula, produzindo exatamente os mesmos valores e tipos que a conversão individual. A gravação passou a ser feita em **lotes**: um único comando carrega muitos registros de uma vez, reduzindo drasticamente o número de idas e vindas à rede. A estratégia em lote é a única da ferramenta; não há opção para escolher outra.

Cada SGBD implementa o lote com o recurso idiomático do seu respectivo

controlador (*driver*): no **PostgreSQL**, a inserção em lote de `execute_values`; no **MongoDB**, a inserção múltipla de `insert_many`; no **Cassandra**, a execução concorrente de comandos preparados (com grau de concorrência 64 dentro de cada lote); no **Redis**, o enfileiramento de comandos em um *pipeline* despachado de uma só vez; e no **Neo4j**, a cláusula `UNWIND` aplicada a uma lista de registros, seguida de `MERGE`, sobre um rótulo com restrição de unicidade.

A estratégia ingênua permanece descrita neste trabalho porque é a *linha de base* contra a qual o ganho da estratégia em lote é medido no Capítulo 5: as duas gravam os mesmos dados, e a comparação isola o custo de cada uma.

4.5.2 Modelos de Execução: Materialização e *Streaming*

A opção `--execution` controla **quando** os dados ocupam a memória. No modo de **materialização** (`materialize`), cada fonte é carregada integralmente em memória — em uma estrutura tabular (*DataFrame*) — e só então percorre as quatro fases de processamento (leitura, seleção, mapeamento e escrita) descritas na seção 4.4. Esse modelo torna as fases nitidamente separáveis, razão pela qual é ele que sustenta a decomposição por fase medida na avaliação; em contrapartida, o seu **pico de memória cresce com o tamanho do arquivo**, o que o inviabiliza para entradas maiores que a memória disponível.

No modo de **fluxo** (`stream`, o padrão para importações reais), a fonte é lida em **blocos** de tamanho fixo (8192 linhas por bloco) e cada bloco é processado e gravado antes de o próximo ser lido. Assim, o pico de memória permanece próximo de **um único bloco** — somado aos poucos registros ainda acumulados à espera de completar um lote de gravação —, **constante em relação ao tamanho total do arquivo**. Isso habilita a importação de arquivos maiores que a memória da máquina, ao custo de um processamento mais entrelaçado: as quatro fases deixam de ser etapas sequenciais distintas e passam a se repetir a cada bloco.

A arquitetura que viabiliza esse modo segue o padrão **portas e adaptadores** (*ports and adapters*). Um **orquestrador** agnóstico de SGBD coordena o fluxo, mas não conhece nenhum banco em particular: ele não emite SQL nem Cypher e não remove duplicatas. Toda a especificidade de cada SGBD é isolada atrás de uma **porta** de escrita comum, o `DbmsSink`, para a qual cada banco fornece um **adaptador** (`PostgresSink`, `MongoSink`, `CassandraSink`, `RedisSink` e `Neo4jSink`). Cada adaptador reaproveita as mesmas rotinas de formatação de linha e de gravação em lote já usadas na importação por materialização, de modo que a lógica de cada SGBD não é duplicada entre os dois modelos de execução.

O processamento de cada bloco reproduz, em pequena escala, o pipeline de

importação. Ao ver o primeiro bloco de uma entidade, o orquestrador realiza um **vínculo tardio**: infere os tipos das colunas a partir daquela amostra — prevalecendo sempre um tipo declarado explicitamente na configuração sobre o inferido — e reutiliza esse vínculo para os blocos seguintes. A cada bloco, aplica os filtros de seleção, converte os tipos (de forma vetorizada) e roteia as linhas para as suas partições de destino, acumulando-as em áreas de espera; quando uma partição acumula registros suficientes, um lote é descarregado ao adaptador correspondente. Duas particularidades merecem nota: a gravação de **relacionamentos** no Neo4j exige um **segundo passe** sobre as fontes (as arestas só podem ser criadas depois que todos os nós de destino existem), realizado também em fluxo e com memória limitada; e o modo de fluxo suporta **fontes-união**, em que uma mesma entidade é alimentada por mais de um CSV.

Por fim, a escolha do modelo de execução não altera o resultado. A equivalência dos dados gravados entre os dois modelos de execução — mesmas partições, mesmas contagens e mesmas conversões — é assegurada por testes automatizados, de modo que a escolha entre materialização e fluxo é uma decisão de **custo**, não de **correção**. É justamente esse custo — tempo de importação e, sobretudo, pico de memória — que a avaliação do Capítulo 5 quantifica, varrendo sistematicamente os dois aspectos aqui descritos.

4.6 INTERFACE GRÁFICA

Todas as capacidades descritas até aqui são acionadas por uma interface de linha de comando governada por doze opções, que precisam ser digitadas corretamente a cada execução e cujos erros de sintaxe só se manifestam depois que o comando é submetido. Para reduzir essa barreira — sem esconder a ferramenta de quem deseja aprendê-la — foi construída uma **interface gráfica** *desktop* sobre a CLI já existente. O princípio que orienta o seu projeto é deliberadamente didático: **o comando exibido é o comando executado**. Cada opção de uso cotidiano da CLI corresponde a um controle gráfico, e o comando resultante permanece visível o tempo todo, de modo que a interface **ensina** a linha de comando em vez de substituí-la.

A interface é um **invólucro fino**: ela não reimplementa nenhuma etapa da importação. O seu núcleo traduz o estado do formulário na lista de argumentos da CLI e a executa como **processo filho**, capturando a saída do processo e apresentando-a em um console integrado. Duas consequências decorrem dessa escolha. A primeira é de correção: como quem importa é a própria CLI, não existe risco de a interface e a linha de comando divergirem em comportamento, e toda a validação por *JSON Schema* descrita na Seção 4.2 continua sendo exercida pelo mesmo código. A segunda é de arquitetura: os módulos que montam o comando, validam as entradas, resolvem o executável e traduzem a saída não dependem da biblioteca gráfica, podendo ser

testados sem instanciar uma aplicação de janelas.

A janela divide-se em duas regiões, separadas por um divisor ajustável, como mostra a Figura 12. Na metade superior estão três cartões de formulário. O primeiro seleciona, por diálogos de arquivo, os dois arquivos de configuração. O segundo reúne as opções de execução, com um controle para cada *flag*: os SGBDs de destino, o modelo de execução discutido na Seção 4.5.2, os modificadores de simulação e de criação de esquema, o nível de *log* e a exibição de dados — uma amostra das primeiras linhas de cada entidade, em quantidade escolhida pelo usuário, todos os dados ou nenhum. Ao lado de cada opção cujo nome não se explica sozinho há um **ícone de ajuda** que, ao receber o cursor ou um clique, descreve o efeito da opção; o de `stream`, por exemplo, explica que a memória usada não cresce com o tamanho do arquivo. Uma opção da CLI não aparece no formulário: o registro de métricas de *benchmark*, instrumento da avaliação de desempenho do Capítulo 5, e não do uso cotidiano da ferramenta, que continua disponível na linha de comando. O terceiro cartão contém a tabela de sobrescrita de caminhos das fontes CSV. Na metade inferior está o console integrado, que exibe o comando montado e, abaixo dele, a saída da execução.

O cartão de fontes adapta-se ao arquivo de importação escolhido. Sem uma configuração de importação válida, ele não aceita fontes, pois uma sobrescrita só faz sentido diante das fontes que a configuração declara. Com uma configuração **multifonte**, aceita vários arquivos — escolhidos um a um, de uma pasta inteira ou arrastados sobre a tabela —, e o nome de cada fonte é deduzido comparando o arquivo com o bloco `sources`, de modo que a sobrescrita gerada se refira a uma fonte que de fato existe. Com uma configuração **combinada**, aceita um único arquivo, associado à única fonte declarada. Trocar de configuração não descarta as linhas já escolhidas: as que deixam de caber na nova configuração são apontadas como erro até que o usuário as remova.

Antes de executar, a interface faz uma **validação prévia** com o próprio código da CLI. Os dois arquivos de configuração passam pelo mesmo *JSON Schema* e pela mesma verificação de consistência entre si descritos na Seção 4.2.4; em seguida, cada CSV envolvido tem apenas o seu cabeçalho lido, e as entidades são vinculadas a fontes sem linhas pelo mesmo mecanismo empregado na importação. Uma coluna referenciada na configuração e ausente do arquivo produz, assim, exatamente a mensagem que a CLI emitiria, só que antes da execução, e o botão de execução permanece desabilitado enquanto houver pendências. A validação prévia tem um limite deliberado: nada que exija conexão com um SGBD ou a leitura das linhas de dados é verificado nessa etapa, porque os arquivos podem ser maiores que a memória disponível. Essas verificações continuam a cargo da CLI, durante a execução.

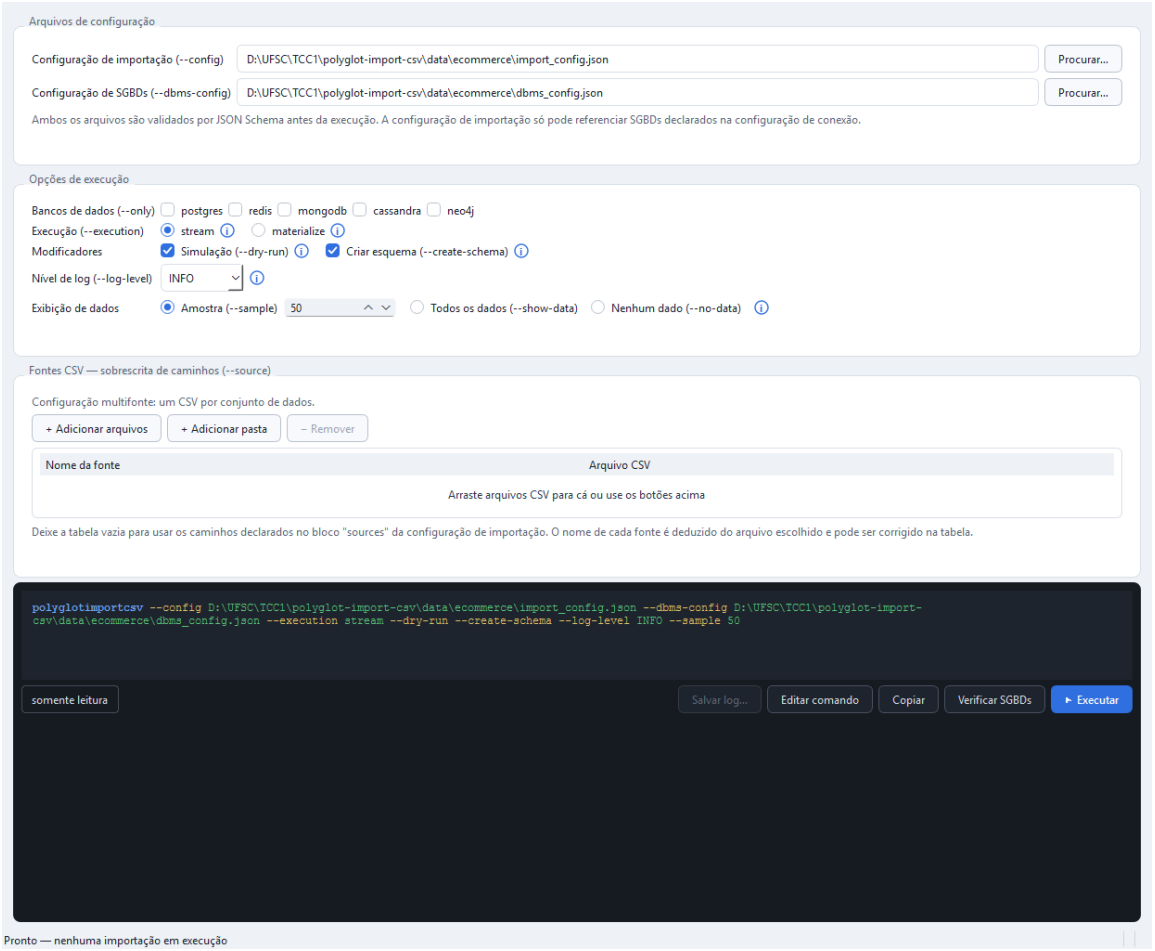


Figura 12 – Interface gráfica no estado ocioso: configuração multifonte escolhida, ícones de ajuda e comando montado, colorido, em modo somente leitura.

Fonte: Elaborado pelo autor (2026).

O comando montado é, por padrão, **somente leitura**, e **explicita todas as opções**, inclusive aquelas mantidas no valor padrão da CLI. A alternativa — omitir o que coincide com o padrão, de modo a permanecer tão curto quanto aquilo que uma pessoa realmente digitaria — contraria o propósito didático da interface: um controle deixado no padrão não produziria alteração visível no texto, e o clique pareceria ter sido ignorado. Escrever a opção por extenso custa alguns caracteres e devolve ao comando a propriedade que justifica exibi-lo — a de que cada controle do formulário tem um correspondente literal na linha de comando. Para facilitar a leitura, o comando é colorido: o nome do programa, as opções e os seus valores recebem cores distintas. Um **modo de edição** permite editar o texto diretamente; ao entrá-lo, o formulário é **bloqueado**, sinalizando que o texto passou a ser a fonte da verdade. Essa decisão é intencional e tem uma razão precisa: **não existe analisador de comando** na ferramenta. O caminho do formulário para o texto é de mão única, e manter os dois editáveis ao mesmo tempo exigiria reconstruir o estado do formulário a partir de uma cadeia de caracteres arbitrária — precisamente o tipo de acoplamento que a interface se propõe a evitar. Ao sair do

modo de edição, o texto digitado é descartado mediante confirmação e o comando é remontado a partir dos controles.

Durante a execução, o formulário permanece desabilitado, o botão principal passa a oferecer a interrupção do processo e a saída da CLI chega ao console de forma incremental, com as mesmas cores que teria em um terminal, como ilustra a Figura 13. Ao término, a barra de estado informa o desfecho: a Figura 14 mostra uma execução concluída, com o tempo total e o caminho do arquivo de *log* da sessão. Um clique nesse caminho abre a pasta em que o arquivo está, e o botão **Salvar log** copia-o para um local escolhido pelo usuário. O arquivo salvo é o *log* completo da sessão, em nível `DEBUG`, e não apenas o texto exibido no console.

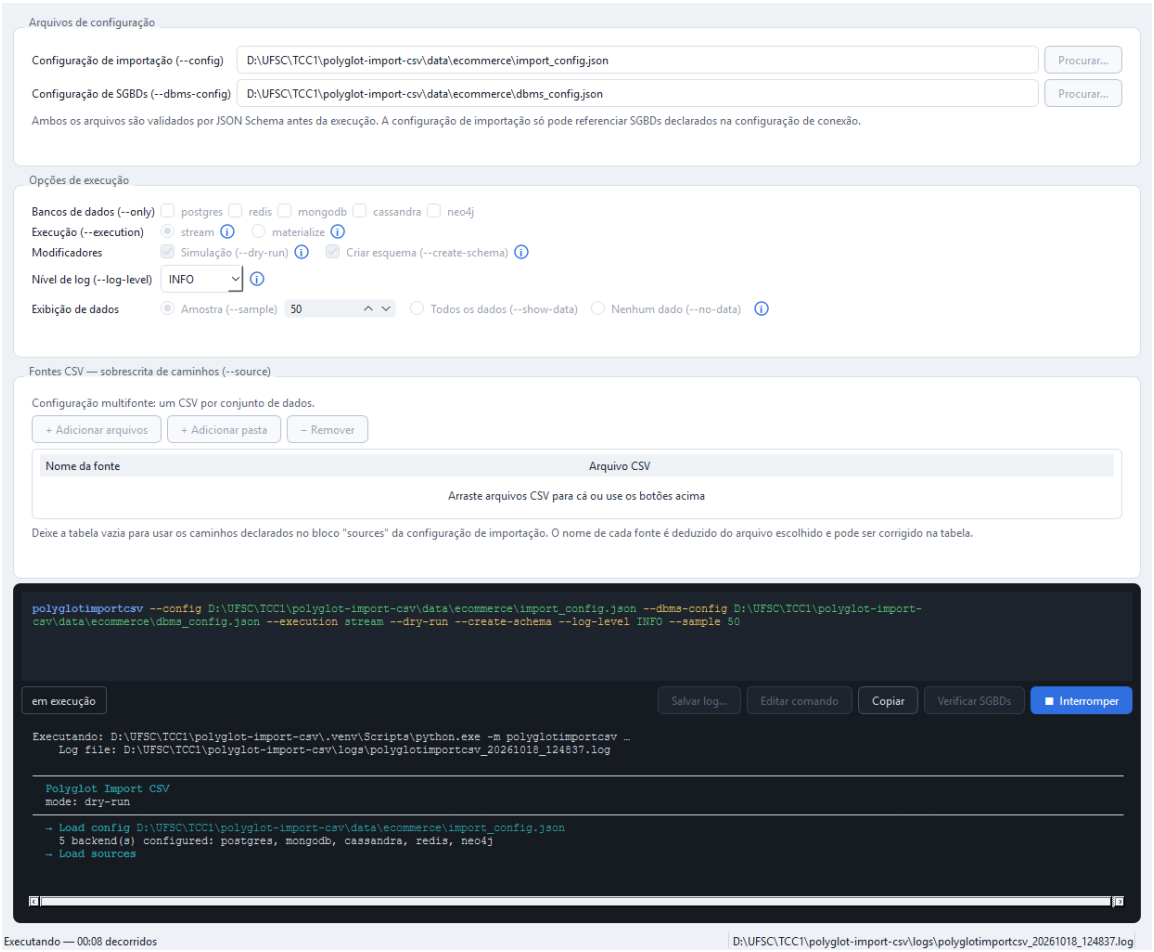


Figura 13 – Execução em andamento: formulário bloqueado, botão de interrupção e saída colorida da CLI chegando ao console em tempo real.

Fonte: Elaborado pelo autor (2026).

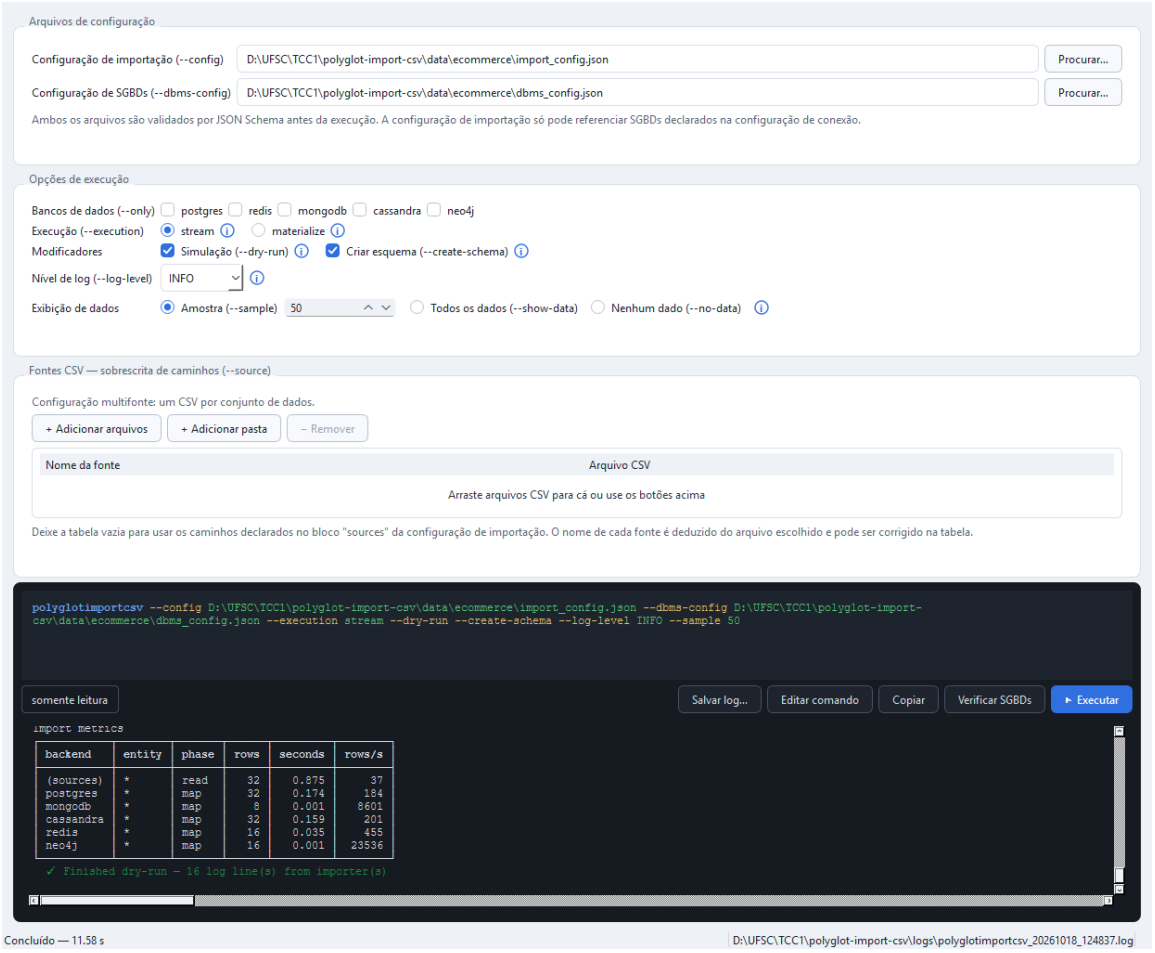


Figura 14 – Execução concluída com sucesso, com o tempo total e o arquivo de *log* da sessão na barra de estado.

Fonte: Elaborado pelo autor (2026).

Os erros aparecem em dois momentos. Os que a validação prévia detecta são apresentados no próprio formulário, junto ao cartão a que se referem, como mostra a Figura 15: a mensagem é a mesma que a CLI produziria, e a execução fica bloqueada. Quando o usuário corrige o arquivo em um editor externo e retorna à janela, a validação é refeita automaticamente. Os erros que só surgem durante a importação, como a falha de gravação em um SGBD, aparecem no console exatamente como apareceriam no terminal, e a barra de estado informa o código de saída do processo. Como a interface executa a CLI real, não há tradução intermediária que pudesse mascarar a mensagem.

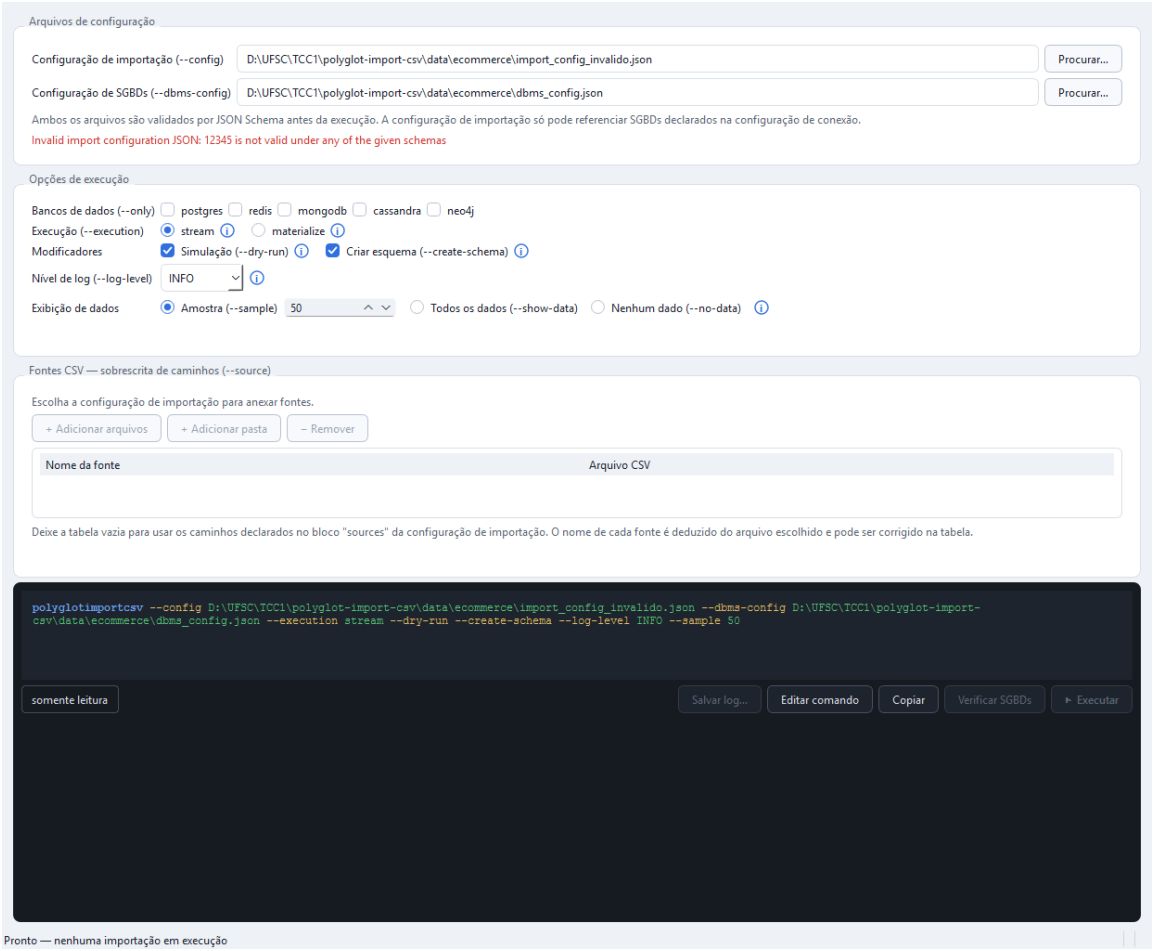


Figura 15 – Erro detectado antes da execução: a configuração de importação viola o *JSON Schema*, a mensagem da CLI aparece no cartão de configuração e o botão de execução permanece desabilitado.

Fonte: Elaborado pelo autor (2026).

A disponibilidade dos SGBDs, que a validação prévia deliberadamente não examina, tem um botão próprio: **Verificar SGBDs**, ao lado do botão de execução. Ele executa a CLI com `--check-dbms`, com os arquivos de configuração e a seleção de SGBDs do formulário, e depende apenas de que esses dois arquivos sejam válidos — pendências nas fontes CSV não o bloqueiam, porque a verificação não lê dados. A saída é a mesma do terminal, inclusive os comandos que iniciam cada SGBD fora do ar, que o usuário copia para um terminal: o console integrado executa somente a própria ferramenta. A Figura 16 mostra uma verificação com dois SGBDs fora do ar.

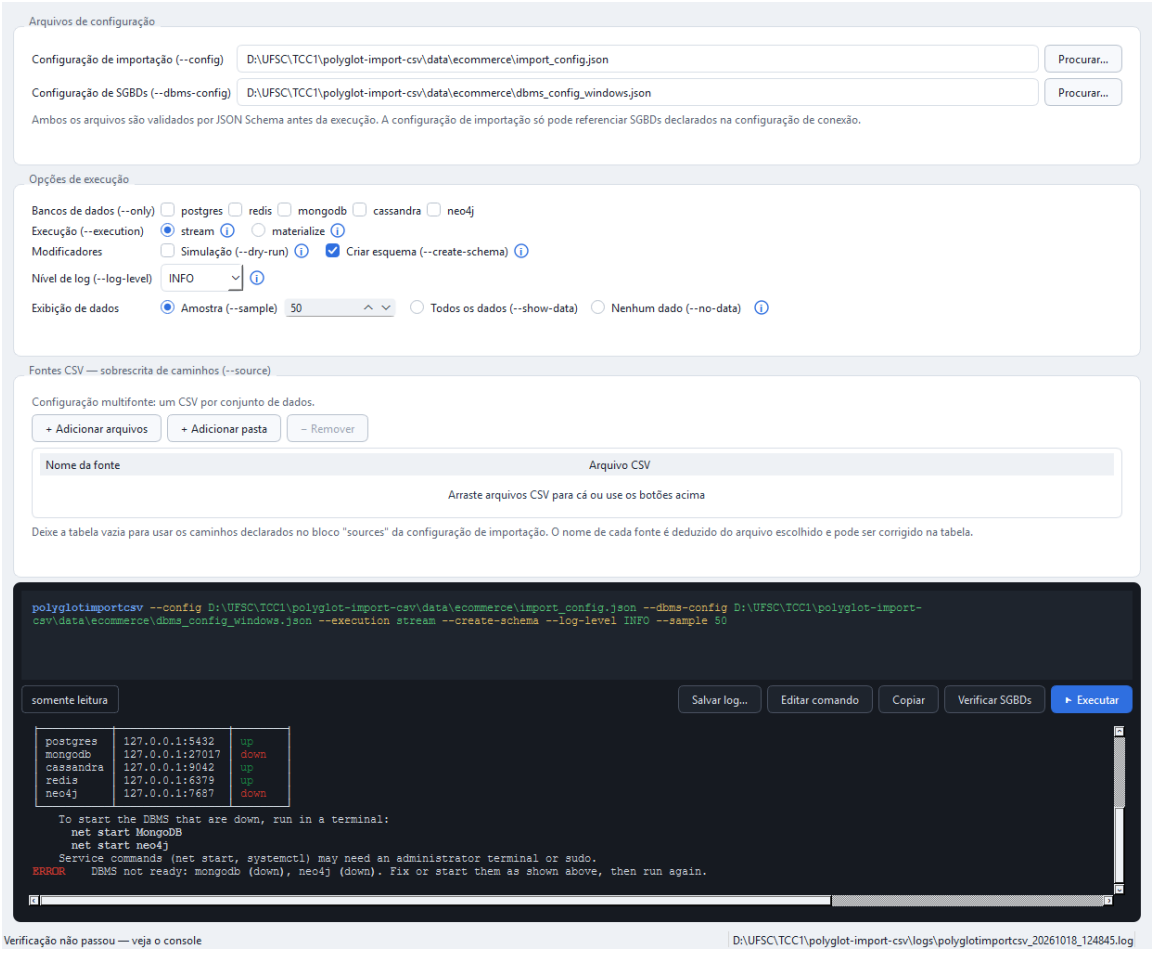


Figura 16 – Verificação dos SGBDs pela interface gráfica: MongoDB e Neo4j fora do ar e, abaixo da tabela de estados, os comandos que os iniciam.

Fonte: Elaborado pelo autor (2026).

Uma limitação de escopo deve ser registrada: a interface **monta, valida e executa** o comando, mas não **edita** os arquivos JSON de configuração, que continuam a ser mantidos em um editor de texto externo. Ampliar a interface para editar a configuração de forma assistida é uma das atividades futuras discutidas no Capítulo 6.

5 AVALIAÇÃO DE DESEMPENHO

Este capítulo avalia empiricamente o desempenho da PolyglotImportCSV. A avaliação foi listada como objetivo específico no Capítulo 1 e apoia-se diretamente no instrumento de medição descrito na seção 4.4 — o subsistema de métricas, que cronometra cada fase da importação e registra o pico de memória por execução — e nos dois eixos de configuração descritos na seção 4.5: a estratégia de escrita (ingênua *versus* em lote) e o modelo de execução (`materialize` *versus* `stream`). O objetivo não é eleger o "melhor" SGBD — os cinco atendem a modelos de dados distintos — mas caracterizar o custo de importar um mesmo conjunto de dados para cinco paradigmas e quantificar o ganho da escrita em lote e do modelo de execução em fluxo.

A prática de comparar sistemas de dados sob uma carga sintética controlada e reproduzível segue a tradição consolidada por Cooper et al. (2010) com o *Yahoo! Cloud Serving Benchmark* (YCSB), que estabeleceu o gerador de carga determinístico e as métricas de *throughput* como base para avaliar sistemas de dados heterogêneos. No contexto específico de persistência poliglota, Landuyt et al. (2023) e Ye et al. (2023) conduzem comparações de desempenho entre múltiplos modelos de dados sob uma carga comum — o mesmo princípio que orienta este capítulo, aqui aplicado à etapa de **importação** e não à de consulta.

5.1 METODOLOGIA

A avaliação é organizada em torno de seis **questões de pesquisa** (QP), que os resultados da seção 5.3 respondem:

- **QP1 — Custo por SGBD.** Qual é o *throughput* de escrita de cada SGBD e como o tempo total de uma importação se decompõe entre as fases de leitura, seleção, mapeamento e escrita?
- **QP2 — Modos de importação.** Importar a partir de um CSV por tipo de evento (modo multifonte) ou de um único CSV combinado impõe custos distintos, e que vantagens tem cada modo?
- **QP3 — Escalabilidade.** Como o tempo e a memória evoluem ao multiplicar o volume de dados por dez, de mil a cem mil linhas?
- **QP4 — Estratégia de escrita.** Qual é o ganho de desempenho da estratégia em lote (conversão vetorizada e gravação em lotes) sobre a ingênua (linha a linha)?

- **QP5 — Modelo de execução.** Qual é o custo, em tempo e em pico de memória, do modo de fluxo em relação ao de materialização?
- **QP6 — Memória limitada.** O pico de memória do modo de fluxo permanece constante em relação ao tamanho do arquivo, como prevê o seu projeto?

5.1.1 Geração dos Dados

A carga de trabalho é produzida por um **gerador determinístico** que sintetiza um domínio de comércio eletrônico a partir de um único parâmetro de escala N (o número de produtos). As demais fontes crescem proporcionalmente a N: o estoque tem N linhas, as compras $3N$, as seleções $2N$ e os carrinhos $2N$. Todos os valores derivam de sementes textuais fixas, de modo que a mesma escala N produz **sempre os mesmos arquivos**, byte a byte, preservando a integridade referencial entre as fontes (todo produto referenciado em uma compra existe no estoque). Essa reprodutibilidade é o que permite comparar campanhas medidas em momentos diferentes: como a carga é idêntica, apenas o tempo e a memória variam entre execuções. O uso de um gerador sintético parametrizável, em vez de um conjunto de dados fixo, segue a mesma orientação metodológica do YCSB (COOPER et al., 2010).

5.1.2 Protocolo de Medição

Cada célula experimental é definida por uma combinação de **tamanho** ($N \in \{1000, 10\,000, 100\,000\}$), **modo** (multifonte ou combinado), **estratégia de escrita** (ingênua ou em lote), **modelo de execução** (`materialize` ou `stream`) e é repetida **três vezes**. Antes de cada repetição, os cinco SGBDs são **limpos** (*cold-load*): as estruturas de destino são truncadas, de modo que nenhuma repetição se beneficia de dados ou de caches deixados pela anterior. O valor reportado para cada célula é a **mediana** das três repetições, que é robusta a uma eventual repetição anômala.

O instrumento de medição impõe uma restrição metodológica importante. O tempo por fase é cronometrado com um relógio de alta resolução; já o pico de memória é obtido instrumentando as alocações do interpretador Python. Essa instrumentação de memória, porém, **infla o tempo** das fases de leitura e mapeamento (cerca de $8{,}6\times$ a leitura e $6{,}5\times$ o mapeamento), enquanto quase não afeta a escrita nos bancos — distorcendo, portanto, as fases umas em relação às outras. Por isso, tempo e memória **não podem ser lidos da mesma execução**. Adotou-se um **protocolo pareado**: duas campanhas idênticas em carga e configuração, uma **com** a instrumentação de memória ativada (fonte dos picos de memória) e outra **sem** ela (fonte dos tempos). As duas campanhas produziram contagens de linhas idênticas em todas as células, confirmando que executam a mesma carga. Ao longo do capítulo, **cada tabela e figura declara**

na legenda de qual das duas campanhas os seus números provêm, e valores de tempo e de memória nunca são combinados em uma mesma linha.

Quanto à precisão, duas campanhas de tempo da mesma célula, sobre o mesmo código, chegaram a diferir entre 15% e 38%: as medidas de tempo carregam, portanto, uma incerteza da ordem de 20%, e diferenças menores que isso não sustentam conclusões. O pico de memória, ao contrário, reproduziu-se com diferença inferior a $0{,}1\%$ entre campanhas independentes, sendo a métrica mais estável do estudo.

5.2 CONFIGURAÇÃO EXPERIMENTAL

Todos os experimentos foram executados em uma **única máquina** com sistema operacional Windows 11 (*build* 10.0.26200) e Python 3.14.5. A semente global de geração foi fixada em 42 e cada célula foi repetida três vezes. Os cinco SGBDs foram providos por contêineres Docker, nas versões indicadas na Tabela 13. Os contêineres partilham os recursos do mesmo hospedeiro (8 GB de memória física); em particular, o Cassandra opera com um *heap* de 1 GB dentro de uma máquina virtual Docker de cerca de 3,8 GB — uma restrição de ambiente que se mostrará relevante na análise da fase de escrita.

Tabela 13 – Imagens Docker dos SGBDs utilizadas na avaliação.

SGBD	**Imagem Docker**
PostgreSQL	`postgres:16-alpine`
Redis	`redis:7-alpine`
MongoDB	`mongo:7`
Cassandra	`cassandra:4.1`
Neo4j	`neo4j:5-community`

Fonte: Elaborado pelo autor (2026), a partir de `docker-compose.yml`.

A matriz completa combina os três tamanhos, os dois modos, as duas estratégias e os dois modelos de execução. Nem toda combinação é medida em todos os pontos: a estratégia ingênua — a implementação original da ferramenta, do TCC1 (seção 4.5.1) — serve de linha de base e foi medida apenas no ponto $N = 10\,000$, no modo de materialização, já que o modo de fluxo sempre gravou em lote (seção 4.5.2). A estratégia ingênua permanece disponível até a versão 1.1.0 da ferramenta, a última que a contém; nas versões seguintes, a ferramenta grava sempre em lote. Cada resultado a seguir indica o seu escopo.

5.3 RESULTADOS

5.3.1 Estratégia de Escrita: Em Lote *versus* Ingênua (QP4)

A Tabela 14 contrasta as duas estratégias de escrita no ponto $N = 10\,000$, no modo de materialização. A estratégia em lote importa o mesmo conjunto de dados — as mesmas 27.494 linhas gravadas — entre $8{,}5$ e $9{,}3$ vezes mais rápido que a via ingênua. O ganho concentra-se, como esperado, nas duas fases que a estratégia em lote ataca: o mapeamento (conversão de tipos), acelerado cerca de $42\times$ pela vetorização, e a escrita, acelerada $7{,}7\times$ pela gravação em lote. A leitura e a seleção, que a estratégia não altera, permanecem estáveis.

Tabela 14 – Tempo total e das fases dominantes por estratégia de escrita, em segundos ($N = 10\,000$, materialização). Campanha sem instrumentação de memória.

Modo	Estratégia	Mapeamento	Escrita	Total	Ganho
multifonte	ingênua	38,54	95,36	135,74	—
	em lote	0,91	12,44	14,54	$9{,}3\times$
combinado	ingênua	—	—	138,44	—
	em lote	—	—	16,30	$8{,}5\times$

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/timings-naive/`.

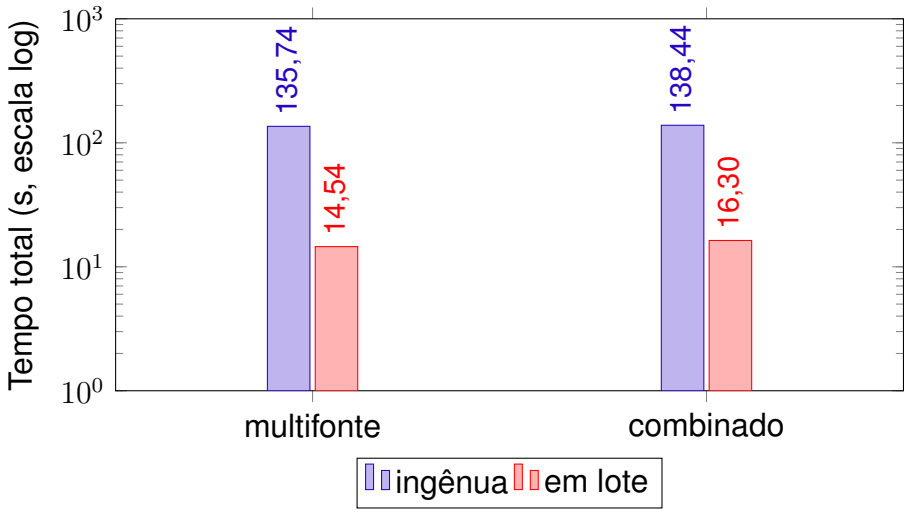


Figura 17 – Tempo total de importação por estratégia de escrita ($N = 10\,000$, materialização; eixo vertical em escala logarítmica). Campanha sem instrumentação de memória.

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/timings-naive/`.

Todos os demais resultados deste capítulo empregam a estratégia em lote, a única da versão atual da ferramenta. A estratégia ingênua cumpre aqui o seu papel exclusivamente metodológico: a linha de base que dá sentido ao número "$9{,}3\times$".

5.3.2 Decomposição por Fase e Escrita por SGBD (QP1)

Com a estratégia em lote, a Tabela 15 decompõe uma importação de $N = 100\,000$ linhas (multifonte, materialização) nas suas quatro fases. A escrita nos bancos domina o custo, respondendo por cerca de 91% do tempo total; a leitura e o mapeamento, juntos, respondem por menos de 10%, e a seleção é desprezível.

Tabela 15 – Tempo por fase de uma importação ($N = 100\,000$, multifonte, materialização), em segundos. Campanha sem instrumentação de memória.

Fase	**Tempo (s)**	**Participação**
Leitura	7,69	5,1%
Seleção	0,00	0,0%
Mapeamento	5,90	3,9%
Escrita	136,34	91,0%
Total	149,93	100%

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/timings/`.

Como a escrita concentra o custo, convém abri-la por SGBD. A Tabela 16 e a Figura 18 apresentam, para a mesma importação, o número de linhas gravadas, o tempo de escrita e o *throughput* resultante em cada um dos cinco bancos. Os valores variam por mais de uma ordem de grandeza: o MongoDB grava a cerca de 14.431 linhas/s, enquanto o Cassandra fica em 912 linhas/s.

Tabela 16 – Escrita por SGBD em uma importação ($N = 100\,000$, multifonte, materialização). Campanha sem instrumentação de memória.

SGBD	**Linhas**	**Tempo (s)**	**Linhas/s**
Cassandra	100.000	109,61	912
Neo4j	51.602	13,23	3.901
PostgreSQL	62.167	8,13	7.644
Redis	49.558	4,56	10.865
MongoDB	11.609	0,80	14.431

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/timings/`.

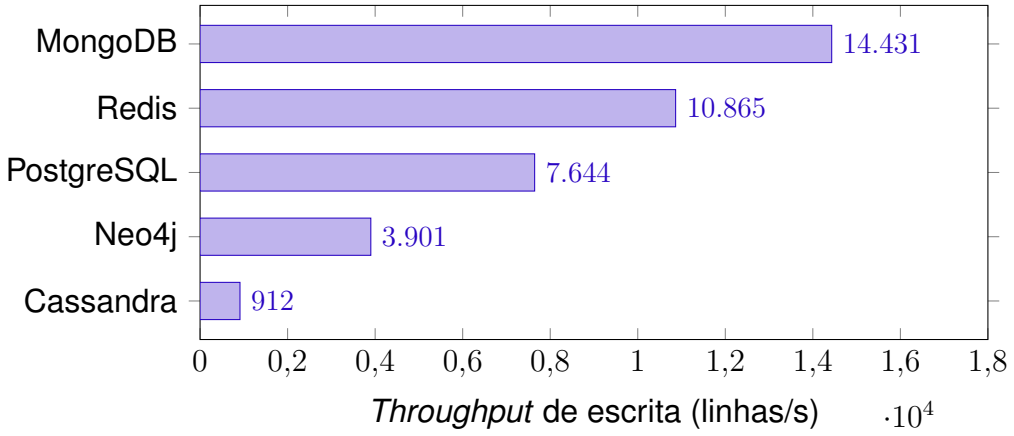


Figura 18 – *Throughput* de escrita por SGBD ($N = 100\,000$, multifonte, materialização, estratégia em lote). Campanha sem instrumentação de memória.

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/timings/`.

O Cassandra sozinho responde por $109{,}6$ s dos $136{,}3$ s da fase de escrita — cerca de 80%. Qualquer afirmação sobre "o tempo de escrita" é, portanto, em larga medida uma afirmação sobre o Cassandra neste ambiente, o que reforça a necessidade de abrir a fase por SGBD em vez de tratá-la como um bloco único.

5.3.3 Modos de Importação: Multifonte *versus* Combinado (QP2)

Os dois modos de importação — um CSV por tipo de evento ou um único CSV combinado — gravam os mesmos dados. A Tabela 17 compara-os no ponto de $100\,000$ linhas quanto ao tamanho dos arquivos, à leitura, ao tempo total e ao pico de memória.

Tabela 17 – Multifonte *versus* combinado ($N = 100\,000$, estratégia em lote). Tempos e picos de memória provêm de campanhas distintas, conforme indicado.

Critério	**Multifonte**	**Combinado**
Tamanho dos arquivos (MB)	23,75 (4 arquivos)	28,55
Colunas por arquivo	20, 33, 6 e 7	39
Células vazias	0%	45%
Fase de leitura, materialização (s)	7,69	15,15
Tempo total, materialização (s)	149,93	161,94
Tempo total, fluxo (s)	165,03	181,42
Pico de memória, materialização (MB)	950,7	$1.009{,}5$
Pico de memória, fluxo (MB)	75,9	79,8

Fonte: Elaborado pelo autor (2026). Tamanho, colunas e células vazias de `data/benchmark/generated/100000/`; tempos de `benchmarks/timings/`; memória de `benchmarks/`.

No **tempo total**, os dois modos empatam: o combinado é cerca de 8% mais lento na materialização e 10% mais lento no fluxo, diferenças abaixo da incerteza de medição do tempo ($\approx 20\%$). O modelo de execução em fluxo, portanto, não favorece um modo em relação ao outro: nos dois, ele reduz o pico de memória cerca de $12{,}5$ vezes e custa de 10 a 12% de tempo. As diferenças que se sustentam estão em outros pontos. O arquivo combinado é cerca de 20% **maior**, porque todos os tipos de evento partilham um único cabeçalho de 39 colunas e cada linha preenche apenas as do seu tipo: 45% das células de dados do arquivo ficam vazias, contra nenhuma nos arquivos do modo multifonte. Por isso, e porque a ferramenta separa o arquivo combinado por tipo de evento durante a leitura, inferindo os tipos das colunas de cada parte, a **fase de leitura** do combinado leva o dobro do tempo, e o seu pico de memória é 5 a 6% maior — diferença real, pois a memória se reproduz com variação inferior a $0{,}1\%$. Como a leitura é uma parcela pequena de uma importação dominada pela escrita (seção 5.3.2), a vantagem não chega ao tempo total.

Além do desempenho, o modo multifonte tem vantagens de **organização**. Cada arquivo traz apenas as colunas do seu tipo de evento, o que o torna mais fácil de ler, de validar e de produzir: cada um pode vir de um sistema diferente e ser atualizado independentemente dos demais. A opção `--source` (seção 4.1) substitui uma fonte isolada — por exemplo, apenas as compras —, enquanto no modo combinado só o arquivo inteiro pode ser substituído. No modelo de execução em fluxo, a amostra usada para inferir os tipos de uma entidade alimentada por mais de uma fonte é lida do primeiro bloco de cada arquivo multifonte, ao passo que o arquivo combinado precisa ser percorrido até que cada tipo de evento apareça. Esse passe adicional de leitura já está incluído nos tempos medidos. A vantagem do modo combinado é a simplicidade de um único arquivo, formato natural quando a origem dos dados já é um registro único de eventos. Em síntese, a escolha entre os dois modos é sobretudo de organização dos dados; em desempenho, o multifonte lê mais rápido e ocupa menos memória, sem diferença significativa no tempo total.

5.3.4 Modelos de Execução: Memória e Tempo (QP5 e QP6)

O resultado central da avaliação está no contraste entre materialização e fluxo. A Tabela 18 e a Figura 19 mostram o pico de memória dos dois modelos em função do tamanho do arquivo, no modo multifonte. A materialização cresce **linearmente** com o volume: de $10{,}5$ MB em mil linhas para $950{,}7$ MB em cem mil — praticamente $90\times$ mais memória para $100\times$ mais dados. O fluxo, ao contrário, **achata**: de $10{,}7$ MB para $75{,}9$ MB no mesmo intervalo. Em cem mil linhas, o fluxo usa $12{,}5\times$ **menos memória** que a materialização, e a vantagem cresce com o tamanho do arquivo. Esse achatamento é a evidência empírica que responde à QP6: o pico do modo de fluxo é governado

pelo tamanho do bloco, não pelo do arquivo, exatamente como prevê o seu projeto (seção 4.5.2).

Tabela 18 – Pico de memória por modelo de execução (N crescente, multifonte, estratégia em lote), em MB. Campanha com instrumentação de memória.

Linhas	Materialização	Fluxo	Razão
1.000	10,5	10,7	$0{,}98\times$
10.000	95,3	47,8	$2{,}0\times$
100.000	950,7	75,9	$\mathbf{12{,}5}\times$

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/` (raiz).

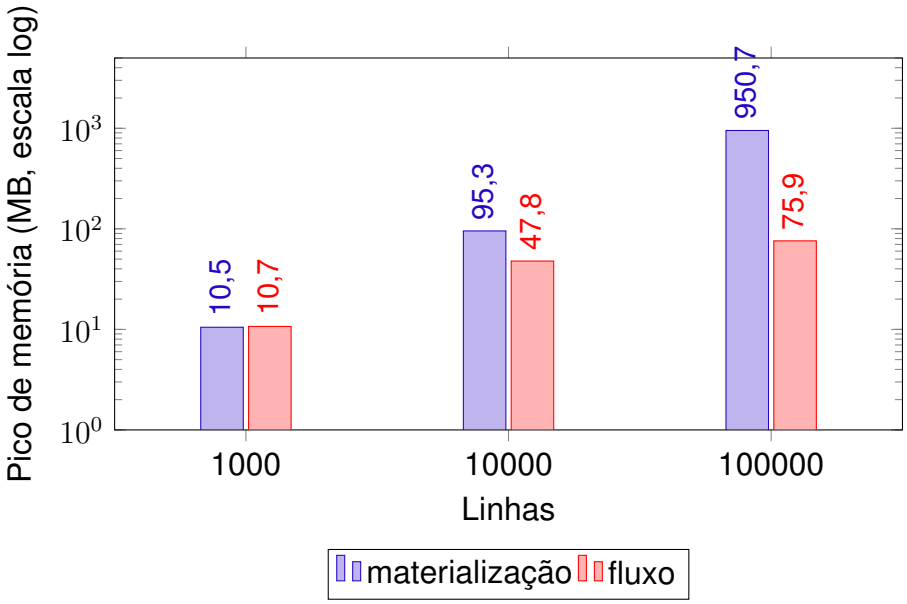


Figura 19 – Pico de memória por modelo de execução em função do volume (multifonte, escala logarítmica): a materialização cresce com o arquivo; o fluxo achata.

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/` (raiz).

Essa economia de memória tem um custo em tempo, mas modesto e **decrescente**. A Tabela 19 e a Figura 20 mostram que o sobrecusto de tempo do fluxo cai de $+61\%$ em mil linhas para apenas $+10\%$ em cem mil. A razão é que, à medida que o volume cresce, a escrita nos bancos — comum aos dois modelos — passa a dominar o tempo total, diluindo o custo fixo do processamento entrelaçado do fluxo. Ou seja: **quanto maior o arquivo, mais barata fica a troca** — a vantagem de memória do fluxo cresce justamente onde o seu custo de tempo encolhe.

Tabela 19 – Tempo total por modelo de execução (N crescente, multifonte, estratégia em lote), em segundos. Campanha sem instrumentação de memória.

Linhas	Materialização	Fluxo	Sobrecusto
1.000	2,87	4,63	+61%
10.000	16,20	20,94	+29%
100.000	149,93	165,03	+10%

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/timings/`.

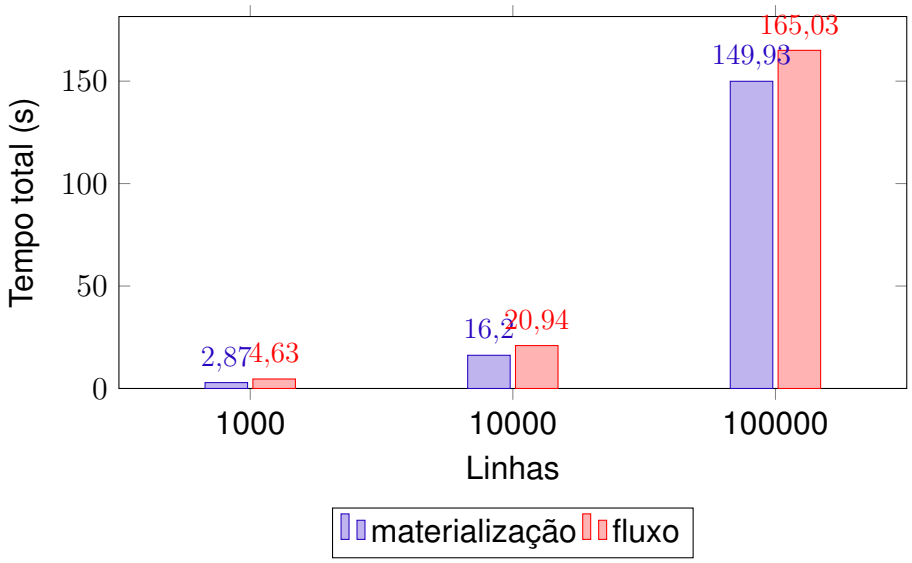


Figura 20 – Tempo total por modelo de execução em função do volume (multifonte): o sobrecusto do fluxo encolhe de +61% para +10% à medida que a escrita passa a dominar.

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/timings/`.

5.3.5 Uma Observação sobre o Pico de Memória da Estratégia em Lote

Um ponto exige cuidado na leitura das medidas de memória. No mesmo ponto de $10\,000$ linhas e modo de materialização, a estratégia **ingênua** exibe um pico **menor** que a estratégia em lote — $14{,}4$ MB contra $95{,}4$ MB no modo multifonte (Tabela 20). O número não é um erro, e reproduz-se entre campanhas. A explicação está na natureza da estratégia em lote: para gravar em lotes, ela **acumula registros em memória** antes de despachá-los. No Neo4j, em particular, a gravação de relacionamentos por `UNWIND` monta a lista completa de arestas de uma entidade em memória antes de submetê-la, ao passo que a via ingênua percorre as arestas uma a uma, sem acumular. A escrita em lote, portanto, **troca memória por vazão**: ela é $7{,}7\times$ mais rápida (Tabela 14), mas ao preço de um pico de memória maior.

Tabela 20 – Pico de memória por estratégia de escrita ($N = 10\,000$, materialização), em MB. Campanha com instrumentação de memória.

Modo	**Ingênua**	**Em lote**
multifonte	14,4	95,4
combinado	21,3	$101{,}6$

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/naive/`.

Vale notar que esse acúmulo é justamente o que o **modo de fluxo** limita: ao processar o arquivo em blocos, ele mantém em memória apenas um bloco e os lotes ainda não descarregados — e é por isso que o fluxo, mesmo gravando em lote, atinge picos muito menores que a materialização (Tabela 18). Os dois eixos, portanto, atuam sobre a memória de formas complementares.

5.4 ANÁLISE E DISCUSSÃO

Os resultados respondem às seis questões de pesquisa. A escrita nos bancos é a fase dominante de uma importação (QP1): mais de 90% do tempo em cem mil linhas, com *throughput* que varia por mais de uma ordem de grandeza entre SGBDs. Essa dispersão reflete o modelo de dados de cada banco e a forma de gravação idiomática correspondente — o MongoDB e o Redis, orientados a inserções em massa simples, lideram; o Cassandra e o Neo4j, que na configuração deste estudo aplicam respectivamente escrita replicada e `MERGE` com verificação de unicidade, ficam atrás. Trata-se de uma manifestação, na etapa de importação, das mesmas diferenças de paradigma que a literatura observa na etapa de consulta (MUNISWAMAIAH; AGERWALA; TAPPERT, 2023; LANDUYT et al., 2023).

Os dois modos de importação empatam no tempo total, nos dois modelos de execução (QP2); o multifonte lê duas vezes mais rápido e ocupa de 5 a 6% menos memória, por ler arquivos menores, sem células vazias, que não precisam ser separados por tipo de evento, e tem vantagens de organização. Quanto à escalabilidade (QP3), o tempo cresce de forma aproximadamente proporcional ao volume, governado pela fase de escrita, enquanto a memória segue o modelo de execução escolhido.

O maior ganho de engenharia está nos dois eixos de desempenho. A estratégia em lote acelera a importação em cerca de $9\times$ (QP4), pela vetorização da conversão de tipos e pela escrita em lote. O modo de fluxo reduz o pico de memória em $12{,}5\times$ em cem mil linhas (QP5), a um custo de tempo de apenas $+10\%$ nesse ponto — e o achatamento do pico com o volume (QP6) confirma que o fluxo importa com memória **constante** no tamanho do arquivo. É esse último resultado que sustenta a principal

afirmação de projeto da ferramenta: a de importar arquivos maiores que a memória disponível, algo impossível no modo de materialização.

5.4.1 Ameaças à Validade

Os resultados devem ser lidos à luz de algumas limitações. Quanto à **validade de construção**, os tempos carregam incerteza da ordem de 20% (seção 5.1.2), de modo que diferenças menores que isso não foram interpretadas como significativas; o pico de memória, muito mais estável ($0{,}1\%$), não sofre dessa limitação. A separação em campanhas pareadas para tempo e memória evita a distorção que a instrumentação de memória introduziria nas fases, mas impede a leitura das duas métricas de uma mesma execução.

Quanto à **validade externa**, todos os experimentos correram em uma única máquina, sob Windows, com os cinco SGBDs em contêineres partilhando 8 GB de memória física. Esse ambiente não representa uma implantação de produção, com bancos dedicados e ajustados. O caso mais sensível é o do Cassandra, cujo *throughput* de escrita (912 linhas/s) reflete um *heap* de 1 GB em uma máquina virtual apertada — uma restrição de **ambiente**, não uma propriedade intrínseca do SGBD; um Cassandra provisionado adequadamente reduziria substancialmente a sua parcela na fase de escrita. Quanto à **validade interna**, o gerador determinístico e a limpeza dos bancos antes de cada repetição isolam o efeito medido, mas o volume avaliado (até cem mil linhas) é moderado, e não se explorou o efeito de caches quentes nem de execuções concorrentes. Aprofundar o caminho de escrita do Cassandra e estender o estudo a volumes maiores figuram entre as atividades futuras (Capítulo 6).

6 ATIVIDADES FUTURAS

Concluídas a implementação da ferramenta e a sua avaliação de desempenho, esta seção reúne as atividades que dão continuidade ao trabalho. Elas dividem-se entre a ampliação da forma de uso, o aprofundamento da avaliação e a extensão das funcionalidades de importação.

- **Edição assistida da configuração na interface gráfica.** A interface gráfica entregue (Seção 4.6) monta e executa o comando de importação, mas não edita os dois arquivos JSON de configuração, que permanecem sob um editor de texto externo. A extensão natural é trazer essa edição para dentro da interface, com validação por *JSON Schema* durante a digitação, pré-visualização do CSV de origem e um editor de mapeamento coluna a coluna, reduzindo ainda mais a barreira de entrada para usuários menos familiarizados com a linha de comando.
- **Aprofundamento da avaliação de desempenho.** A avaliação do Capítulo 5 cobriu volumes de até cem mil linhas em uma única máquina, com os cinco SGBDs em contêineres partilhando os mesmos recursos. Três extensões se destacam: medir volumes maiores, na ordem de milhões de linhas; aprofundar o caminho de escrita do Cassandra — a fase dominante do custo — para além da concorrência já empregada pela estratégia em lote (Seção 4.5.1); e repetir os experimentos em um ambiente com SGBDs dedicados e ajustados, isolando o desempenho da ferramenta das restrições do ambiente de contêineres.
- **Suporte a outros formatos de entrada.** Aceitar arquivos TSV e planilhas Excel, além de CSV, reaproveitando o mesmo mapeamento declarativo já existente.
- **Operadores de filtro adicionais e políticas de coerção configuráveis.** Ampliar o vocabulário de seleção de linhas e permitir que o usuário defina como valores ambíguos ou malformados são convertidos aos tipos nativos de cada SGBD.
- **Testes de integração em ambiente conteinerizado.** A suíte de testes atual é majoritariamente unitária; prevê-se automatizar testes de integração contra os cinco SGBDs em `docker compose`, dentro de um fluxo de integração contínua (CI).
- **Generalização do núcleo de importação.** Estender a arquitetura de portas e adaptadores (Seção 4.5.2) a novos destinos — sistemas de mensageria, *data lakes* e outros SGBDs —, reaproveitando o orquestrador agnóstico de destino sem duplicar a lógica de importação.

7 CONSIDERAÇÕES FINAIS

Este trabalho apresentou a *PolyglotImportCSV*, uma ferramenta de importação declarativa que materializa um mesmo conjunto de dados em formato CSV em cinco SGBDs de paradigmas distintos — PostgreSQL, MongoDB, Cassandra, Redis e Neo4j — a partir de dois arquivos de configuração validados estaticamente. Partindo da fundamentação sobre persistência poliglota e do estado da arte em importação e modelagem de dados heterogêneos, o trabalho evoluiu da proposta a uma implementação funcional, avaliada empiricamente.

As principais contribuições são: (i) um formato de configuração JSON, separado em mapeamento e conexão e validado por JSON Schema antes de qualquer escrita, com dois modos de entrada — um arquivo por entidade (multifonte) e um único CSV combinado; (ii) o mapeamento declarativo de colunas com conversão nativa aos tipos de cada SGBD; (iii) uma camada de observabilidade e um subsistema de métricas que instrumentam cada fase da importação; (iv) dois eixos de desempenho — a estratégia de escrita em lote, com conversão vetorizada e gravação em lotes, e o modelo de execução em fluxo, de memória limitada; e (v) uma avaliação empírica com dados reais que quantifica o custo de importar para os cinco paradigmas e o ganho de cada eixo.

A avaliação (Capítulo 5) confirmou os objetivos traçados. A estratégia em lote acelerou a importação em cerca de nove vezes sobre a linha de base ingênua, e o modo de execução em fluxo reduziu o pico de memória em mais de doze vezes no maior volume medido, mantendo-o constante em relação ao tamanho do arquivo — o que habilita a importação de arquivos maiores que a memória disponível. A equivalência dos dados gravados entre os dois modos de importação e entre os dois modelos de execução, assegurada por testes automatizados, permitiu tratar a escolha entre eles como uma decisão de custo, não de correção.

As limitações reconhecidas dizem respeito sobretudo ao ambiente de avaliação: os experimentos correram em uma única máquina, com os cinco SGBDs em contêineres partilhando os mesmos recursos, o que penalizou em especial a escrita no Cassandra — uma restrição do ambiente, e não uma propriedade da ferramenta. Além disso, a suíte de testes permanece majoritariamente unitária, sem integração contínua contra os SGBDs conteinerizados.

Como trabalho futuro (Capítulo 6), destacam-se a edição assistida da configuração dentro da interface gráfica, o aprofundamento da avaliação em volumes maiores e em ambientes com SGBDs dedicados, e a generalização do núcleo de importação a novos destinos. Consolida-se, assim, a *PolyglotImportCSV* como artefato de pesquisa

e de código aberto alinhado à literatura sobre persistência poliglota e modelagem de dados heterogêneos.

REFERÊNCIAS

CANDEL, C. J. F.; RUIZ, D. S.; GARCíA-MOLINA, J. J. A unified metamodel for NoSQL and relational databases. *Information Systems*, v. 104, p. 101898, 2022. Citado na página 30.

CHILLóN, A. H. et al. A generic schema evolution approach for nosql and relational databases. *IEEE Transactions on Knowledge and Data Engineering*, v. 36, n. 7, p. 2774–2789, 2024. Citado na página 30.

CHILLóN, A. H. et al. Propagating schema changes to code: An approach based on a unified data model. In: *Proceedings of the Workshops of the EDBT/ICDT 2023 Joint Conference (CoMoNoS)*. [s.n.], 2023. (CEUR Workshop Proceedings, v. 3379). Disponível em: <https://ceur-ws.org/Vol-3379/CoMoNoS_2023_id251_Alberto_Hernandez_Chillon.pdf>. Citado 2 vezes nas páginas 14 e 30.

COOPER, B. F. et al. Benchmarking cloud serving systems with YCSB. In: *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*. [S.l.]: ACM, 2010. (SoCC '10), p. 143–154. Citado 2 vezes nas páginas 72 e 73.

DBCROSSBAR: copy tabular data between databases, CSV files, and cloud storage. 2024. <https://www.dbcrossbar.org>. Ferramenta de código aberto em Rust para migração e conversão de esquemas. Citado 2 vezes nas páginas 29 e 30.

DUGGAN, J. et al. The BigDAWG polystore system. *ACM SIGMOD Record*, v. 44, n. 2, p. 11–16, 2015. Citado na página 30.

FORD, N. *The Productive Programmer*. [S.l.]: O'Reilly Media, 2008. Citado na página 18.

FOWLER, M. *PolyglotPersistence*. 2011. Disponível em: <https://martinfowler.com/bliki/PolyglotPersistence.html>. Citado na página 18.

GLAKE, D. et al. *Towards Polyglot Data Stores: Overview and Open Research Questions*. 2022. Citado 3 vezes nas páginas 24, 30 e 31.

Hackolade. *Polyglot Data Modeling*. 2024. <https://hackolade.com/polyglot-data-modeling.html>. Modelagem conceitual com engenharia de esquema para múltiplos SGBDs. Citado na página 30.

HECHT, R.; JABLONSKI, S. Nosql evaluation: A use case oriented survey. In: *Proceedings of the 2011 International Conference on Cloud and Service Computing*. [S.l.: s.n.], 2011. p. 336–341. Citado na página 20.

KAUR, K.; SHARMA, S.; KAHLON, K. S. A middleware for polyglot persistence and data portability of big data PaaS cloud applications. *Computers, Materials & Continua*, v. 65, n. 2, p. 1625–1647, 2020. Citado na página 30.

KHINE, P. P.; WANG, Z. A review of polyglot persistence in the big data world. *Information*, v. 10, n. 4, p. 141, 2019. Citado na página 14.

KIEHN, F. et al. Polyglot data management: State of the art & open challenges. *Proceedings of the VLDB Endowment*, v. 15, n. 12, p. 3750–3753, 2022. Citado 4 vezes nas páginas 14, 24, 30 e 31.

LANDUYT, D. V. et al. A comparative performance evaluation of multi-model NoSQL databases and polyglot persistence. In: *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing (SAC)*. [S.l.]: ACM, 2023. (SAC '23), p. 286–293. Citado 2 vezes nas páginas 72 e 81.

MongoDB, Inc. *mongoimport — Database Tools*. 2026. <https://www.mongodb.com/docs/database-tools/mongoimport/>. Utilitário de importação de CSV/JSON/TSV para o MongoDB. Citado na página 28.

MUNISWAMAIAH, M.; AGERWALA, T.; TAPPERT, C. C. Comparison of SQL, NoSQL, and NewSQL database technologies. In: *2023 IEEE International Conference on Big Data (BigData)*. [S.l.]: IEEE, 2023. p. 6230–6232. Citado na página 81.

Neo4j, Inc. *Import — Operations Manual*. 2026. <https://neo4j.com/docs/operations-manual/current/tools/neo4j-admin/neo4j-admin-import/>. Importação em massa de CSV com `neo4j-admin database import`. Citado na página 29.

PEREIRA, F. d. O.; GUERRA, E.; ROSA, R. R. A framework model for supporting transparent polyglot persistence with a unified API and extensible for different database types. In: *Proceedings of the 27th International Conference on Enterprise Information Systems (ICEIS)*. [S.l.]: SciTePress, 2025. p. 109–120. Citado na página 30.

Redis Ltd. *Bulk Loading*. 2026. <https://redis.io/docs/latest/develop/clients/patterns/bulk-loading/>. Carga em massa via `redis-cli --pipe` usando o protocolo RESP. Citado na página 27.

ROY-HUBARA, N.; SHOVAL, P.; STURM, A. Selecting databases for polyglot persistence applications. *Data & Knowledge Engineering*, v. 137, p. 101950, 2022. Citado 2 vezes nas páginas 24 e 30.

SACHDEVA, S.; VASAVA, J. Plugging and playing with variety of data using multi-model database and polyglot persistence. In: *2024 Third International Conference on Electrical, Electronics, Information and Communication Technologies (ICEEICT)*. [S.l.]: IEEE, 2024. p. 1–8. Citado na página 30.

SADALAGE, P. J.; FOWLER, M. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. [S.l.]: Pearson Education, 2013. Citado 2 vezes nas páginas 14 e 24.

SILVA, H. A. B. d.; BORNIA, L. G.; MELLO, R. d. S. Um estudo sobre modelagem poliglota de dados. In: *Anais da Escola Regional de Banco de Dados (ERBD)*. [S.l.]: Sociedade Brasileira de Computação, 2024. p. 11–20. Citado 2 vezes nas páginas 14 e 30.

TAN, R. et al. Enabling query processing across heterogeneous data models: A survey. In: *2017 IEEE International Conference on Big Data (BigData)*. [S.l.: s.n.], 2017. p. 3211–3220. Citado 2 vezes nas páginas 24 e 30.

The Apache Software Foundation. *sstableloader*. 2026. <https://cassandra.apache.org/doc/latest/cassandra/managing/tools/sstable/sstableloader.html>. Ferramenta de carga em massa de SSTables no Apache Cassandra. Citado na página 28.

The PostgreSQL Global Development Group. *PostgreSQL Documentation: COPY*. 2026. <https://www.postgresql.org/docs/current/sql-copy.html>. Comando `COPY` para importação/exportação de dados, inclusive CSV. Citado na página 26.

YE, F. et al. A benchmark for performance evaluation of a multi-model database vs. polyglot persistence. *Journal of Database Management*, v. 34, n. 3, p. 1–20, 2023. Citado 3 vezes nas páginas 14, 18 e 72.

APÊNDICES

APÊNDICE A – JSON SCHEMAS DE CONFIGURAÇÃO NA FORMA SERIALIZADA

Este apêndice reproduz os dois documentos *JSON Schema* (rascunho 2020-12) tal como embutidos em `src/polyglotimportcsv/schemas/`. A estrutura de alto nível de ambos é descrita nas seções 4.2.1 e 4.2.3.

A.1 SCHEMA DE IMPORTAÇÃO (`IMPORT_CONFIG.SCHEMA.JSON`)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://polyglot-import-csv.local/schemas/import_config.schema.json",
  "title": "PolyglotImportCSV import configuration",
  "description": "Declarative mapping from named CSV sources to entities and
      relationships of one or more DBMSs. Connection settings live in the separate
      DBMS configuration.",
  "type": "object",
  "required": ["sources"],
  "properties": {
    "sources": {
      "type": "object",
      "minProperties": 1,
      "description": "Named data sources: a CSV path per entity, or a combined CSV
          whose column 0 holds each row's origin.",
      "additionalProperties": {
        "oneOf": [
          { "type": "string", "minLength": 1 },
          {
            "type": "object",
            "required": ["file", "origin_column"],
            "properties": {
              "file": { "type": "string", "minLength": 1 },
              "origin_column": { "const": true }
            },
            "additionalProperties": false
          }
        ]
      }
    },
    "postgres": { "$ref": "#/$defs/postgresMapping" },
    "mongodb": { "$ref": "#/$defs/mongoMapping" },
    "cassandra": { "$ref": "#/$defs/cassandraMapping" },
    "redis": { "$ref": "#/$defs/redisMapping" },
    "neo4j": { "$ref": "#/$defs/neo4jMapping" }
  },
  "additionalProperties": false,
  "$defs": {
    "columnSpec": {
      "type": "object",
      "description": "Leaf mapping from a CSV column to a destination field.",
      "properties": {
```

```
      "is_key": {
        "type": "boolean",
        "description": "Primary or merge key for the entity."
      },
      "csv_column": {
        "description": "CSV header name or 1-based data-column index when it
            differs from the JSON key.",
        "oneOf": [
          { "type": "string", "minLength": 1 },
          { "type": "integer", "minimum": 1 }
        ]
      },
      "schema_column": {
        "type": "string",
        "minLength": 1,
        "description": "Destination field name when it differs from the JSON key."
      },
      "db_type": {
        "type": "string",
        "description": "SQL/CQL type for DDL generation (PostgreSQL, Cassandra)."
      }
    },
    "additionalProperties": false
  },
  "columnEntry": {
    "description": "Either a leaf columnSpec or a nested object (MongoDB
        subdocuments).",
    "oneOf": [
      { "$ref": "#/$defs/columnSpec" },
      {
        "type": "object",
        "minProperties": 1,
        "additionalProperties": { "$ref": "#/$defs/columnEntry" }
      }
    ]
  },
  "filter": {
    "type": "object",
    "description": "Row predicate applied to the full CSV before materialization
        .",
    "required": ["column", "operator"],
    "properties": {
      "column": { "type": "string", "minLength": 1 },
      "operator": {
        "type": "string",
        "enum": ["==", "!=", ">", "<", ">=", "<=", "in", "not_in", "each"]
      },
      "value": {},
      "target_suffix": {
        "type": "string",
        "description": "With operator 'each', optional template; default is
            sanitized distinct value"
      }
    },
    "additionalProperties": false
  },
```

```
  "entity": {
    "type": "object",
    "description": "One logical target (table, collection, label, or Redis entity)
        .",
    "properties": {
      "columns": {
        "type": "object",
        "minProperties": 1,
        "additionalProperties": { "$ref": "#/$defs/columnEntry" }
      },
      "filters": {
        "type": "array",
        "items": { "$ref": "#/$defs/filter" }
      },
      "cassandra_partition": {
        "type": "array",
        "items": { "type": "string" },
        "description": "CSV/source column names forming the partition key (
            compound supported)"
      },
      "cassandra_cluster": {
        "type": "array",
        "items": { "type": "string" },
        "description": "Clustering columns (optional)"
      },
      "source": {
        "description": "Source name, or list of source names (union with _source
            per row). Omit to bind by the entity key name.",
        "oneOf": [
          { "type": "string", "minLength": 1 },
          { "type": "array", "minItems": 1, "items": { "type": "string", "
              minLength": 1 } }
        ]
      },
      "csv_columns": {
        "type": "array",
        "minItems": 1,
        "description": "Auto-map restriction: column names, 1-based indices, or
            ranges like \"1-5\". Only valid with auto-mapping.",
        "items": {
          "oneOf": [
            { "type": "string", "minLength": 1 },
            { "type": "integer", "minimum": 1 }
          ]
        }
      },
      "auto_map": {
        "type": "boolean",
        "description": "With 'columns' present, auto-map the whole source and let
            the manual entries override per column."
      }
    },
    "additionalProperties": false
  },
  "postgresMapping": {
    "type": "object",
```

```
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/entity" }
      },
      "relationships": {
        "type": "object",
        "additionalProperties": {
          "type": "object",
          "required": ["from", "to", "foreign_key"],
          "properties": {
            "from": { "type": "string" },
            "to": { "type": "string" },
            "foreign_key": { "type": "string" },
            "references_key": { "type": "string" }
          },
          "additionalProperties": false
        }
      }
    },
    "additionalProperties": false
  },
  "mongoMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/entity" }
      }
    },
    "additionalProperties": false
  },
  "cassandraMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/entity" }
      }
    },
    "additionalProperties": false
  },
  "redisMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/entity" }
      }
    },
    "additionalProperties": false
  },
```

```
    "neo4jMapping": {
      "type": "object",
      "required": ["entities"],
      "properties": {
        "entities": {
          "type": "object",
          "additionalProperties": { "$ref": "#/$defs/entity" }
        },
        "relationships": {
          "type": "object",
          "additionalProperties": {
            "type": "object",
            "required": ["from", "to", "type"],
            "properties": {
              "from": { "type": "string" },
              "to": { "type": "string" },
              "type": { "type": "string" },
              "columns": {
                "type": "object",
                "additionalProperties": { "$ref": "#/$defs/columnSpec" }
              }
            },
            "additionalProperties": false
          }
        }
      },
      "additionalProperties": false
    }
  }
}
```

A.2 SCHEMA DE CONEXÃO (DBMS_CONFIG.SCHEMA.JSON)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://polyglot-import-csv.local/schemas/dbms_config.schema.json",
  "title": "PolyglotImportCSV DBMS connection configuration",
  "description": "Connection settings for each DBMS. Lists which DBMSs are available
      ; the import configuration may only target DBMSs declared here.",
  "type": "object",
  "required": ["version"],
  "properties": {
    "version": {
      "type": "integer",
      "minimum": 1,
      "description": "Configuration format version."
    },
    "postgres": { "$ref": "#/$defs/postgresConnection" },
    "mongodb": { "$ref": "#/$defs/mongoConnection" },
    "cassandra": { "$ref": "#/$defs/cassandraConnection" },
    "redis": { "$ref": "#/$defs/redisConnection" },
    "neo4j": { "$ref": "#/$defs/neo4jConnection" }
  },
  "additionalProperties": false,
  "$defs": {
```

```
"postgresConnection": {
  "type": "object",
  "required": ["connection"],
  "properties": {
    "connection": {
      "type": "object",
      "properties": {
        "host": { "type": "string" },
        "port": { "type": "integer" },
        "database": { "type": "string" },
        "user": { "type": "string" },
        "password": { "type": "string" }
      },
      "additionalProperties": false
    },
    "schema": {
      "type": "string",
      "default": "public",
      "description": "Target schema for created tables."
    },
    "start": { "$ref": "#/$defs/start" }
  },
  "additionalProperties": false
},
"mongoConnection": {
  "type": "object",
  "required": ["connection"],
  "properties": {
    "connection": {
      "type": "object",
      "properties": {
        "uri": { "type": "string" },
        "database": { "type": "string" }
      },
      "required": ["uri", "database"],
      "additionalProperties": false
    },
    "start": { "$ref": "#/$defs/start" }
  },
  "additionalProperties": false
},
"cassandraConnection": {
  "type": "object",
  "required": ["connection"],
  "properties": {
    "connection": {
      "type": "object",
      "properties": {
        "hosts": { "type": "array", "items": { "type": "string" } },
        "port": { "type": "integer" },
        "keyspace": { "type": "string" },
        "protocol_version": { "type": "integer" },
        "request_timeout": { "type": "number", "exclusiveMinimum": 0, "
            description": "Client-side request timeout in seconds (default 30).
            The driver's own default of 10s is short for a bulk load." }
      },
```

```
        "required": ["hosts", "keyspace"],
        "additionalProperties": false
      },
      "start": { "$ref": "#/$defs/start" }
    },
    "additionalProperties": false
  },
  "redisConnection": {
    "type": "object",
    "required": ["connection"],
    "properties": {
      "connection": {
        "type": "object",
        "properties": {
          "host": { "type": "string" },
          "port": { "type": "integer" },
          "db": { "type": "integer" },
          "password": { "type": "string" }
        },
        "additionalProperties": false
      },
      "start": { "$ref": "#/$defs/start" }
    },
    "additionalProperties": false
  },
  "neo4jConnection": {
    "type": "object",
    "required": ["connection"],
    "properties": {
      "connection": {
        "type": "object",
        "properties": {
          "uri": { "type": "string" },
          "user": { "type": "string" },
          "password": { "type": "string" },
          "database": { "type": "string" }
        },
        "required": ["uri", "user", "password"],
        "additionalProperties": false
      },
      "start": { "$ref": "#/$defs/start" }
    },
    "additionalProperties": false
  },
  "start": {
    "description": "How to start this DBMS when it does not answer. Shown to the
        user, never executed.",
    "oneOf": [
      {
        "type": "object",
        "properties": {
          "command": { "type": "string", "minLength": 1, "description": "Command
              line that starts a DBMS installed on this machine." }
        },
        "required": ["command"],
        "additionalProperties": false
```

```
        },
        {
          "type": "object",
          "properties": {
            "compose": {
              "type": "object",
              "properties": {
                "file": { "type": "string", "minLength": 1, "description": "Docker
                    Compose file, relative to this configuration file's folder." },
                "service": { "type": "string", "minLength": 1, "description": "
                    Service of that file that runs this DBMS." }
              },
              "required": ["file", "service"],
              "additionalProperties": false
            }
          },
          "required": ["compose"],
          "additionalProperties": false
        }
      ]
    }
  }
}
```

APÊNDICE B – O ARQUIVO DE DADOS `ECOMMERCE_JOIN.CSV`

Este apêndice reproduz, em forma tabular, o arquivo de dados do cenário e-commerce, `data/ecommerce/ecommerce_join.csv`, utilizado nas seções 4.3 e 4.3.4. O arquivo contém 32 linhas de dados e 39 colunas, e resulta da **junção** (*join*) de todas as coleções de dados da fonte — usuários, endereços, produtos, categorias, estoque, pedidos, seleções de produto e carrinhos de compra — em uma única tabela larga. Por isso ele é **esparso**: cada linha representa um evento da loja virtual, identificado pela coluna `action` (`stock`, `purchase`, `select_product` ou `add_to_cart`), e preenche apenas as colunas pertinentes àquele tipo de evento — as demais permanecem vazias. A Tabela 21 resume quais grupos de colunas cada ação preenche. Este arquivo é a entrada do **modo combinado** de importação; no **modo multifonte**, os mesmos dados são fornecidos como quatro arquivos CSV separados, um por origem (`ecommerce_stock.csv`, `ecommerce_purchase.csv`, `ecommerce_select_product.csv` e `ecommerce_add_to_cart.csv`).

Tabela 21 – Grupos de colunas do `ecommerce_join.csv` preenchidos por ação.

Grupo	**Colunas**	`stock`	`purchase`	`select_product`	`add_to_cart`
Evento	`action`, `timestamp`	•	•	•	•
Usuário	`user_id`, `user_name`, `user_email`	•	•	•	•
Endereço do usuário	`street`, `neighborhood`, `state`, `country`, `zip_code`	•	•	—	—
Pedido	`order_number`, `order_date`, `order_status`	—	•	—	—
Contraparte da venda	`traded_with`, `trader_street`, `trader_neighborhood`, `trader_state`, `trader_country`, `trader_zip_code`	—	•	—	—
Avaliação e pagamento	`comment`, `rating`, `payment_method`, `quantity`	—	•	—	—
Produto	`product_id`, `product_name`, `product_variant`, `product_brand`, `product_description`, `product_image`	•	•	—	—
Categoria	`category_id`, `category_name`	•	•	—	—
Estoque	`quantity_available`, `price`, `last_restock_date`	•	•	—	—
Seleção de produto	`selected_product_id`, `suggested_product_count`	—	—	•	—
Carrinho	`shopping_cart_id`, `cart_product_id`, `cart_quantity`	—	—	—	•

Fonte: Elaborado pelo autor (2026).

Vários valores do arquivo seguem padrões regulares, o que permite apresentá-

los de forma resumida sem perda de informação. O produto `Product`N tem `product_id` N, pertence à categoria `Categoria`N (`category_id` N) e tem descrição e imagem padronizadas (`Description`M e `https://example.com/image`M`.jpg`). Cada usuário `user`N tem e-mail `user`N`@example.com`, rua `Street` N, `42` e um identificador alfanumérico (`user_id`) cujo início é dado na Tabela 22, junto com os demais dados de endereço. Nas tabelas deste apêndice, os usuários são referidos pelo nome (`user`N), tanto na coluna `user_id` quanto em `traded_with`.

Tabela 22 – Usuários do `ecommerce_join.csv` e seus dados de identificação e endereço.

Usuário	**`user_id` (início)**	**Bairro**	**Estado**	**País**	**CEP**
user1	`7fd7eee7-...`	ABC	CA	United States	12345
user2	`f246fb91-...`	DEF	SC	Brasil	84102149
user3	`09b3458a-...`	GHI	RS	Brasil	18057313
user4	`1fd3d09a-...`	JKL	TO	Brasil	19645442
user5	`0331463b-...`	MNO	RJ	Brasil	30097508
user6	`f4857da9-...`	PQR	SP	Brasil	92083256
user7	`a87df805-...`	STU	SP	Brasil	67065799
user8	`c11a1ef2-...`	VWX	SE	Brasil	18439703
user9	`d0416923-...`	YZ	AM	Brasil	18439666

Fonte: Elaborado pelo autor (2026).

Nas seções seguintes, o arquivo é apresentado em quatro blocos, um por ação. Cada tabela exibe as colunas com valores próprios daquele bloco; as colunas cujo valor é constante ou segue os padrões acima são descritas no texto que antecede cada tabela. Os valores e a ordem das linhas são exatamente os do arquivo original.

B.1 LINHAS COM `ACTION == STOCK`

As oito linhas de reposição de estoque preenchem os grupos de usuário, endereço, produto, categoria e estoque (21 das 39 colunas). Todas ocorrem em 02/11/2023 e, em cada uma, `last_restock_date` repete o instante do próprio evento (`timestamp`). A Tabela 23 apresenta os valores.

Tabela 23 – Linhas `action == stock` do `ecommerce_join.csv`.

Hora	Usuário	Produto	Variante	Marca	Categoria	Qtd.	Preço
03:30	user1	Product2	blue	Microsoft	Categoria2	2	50.5
06:45	user2	Product3	green	Google	Categoria3	1	52.07
09:15	user2	Product9	black	Pepsi	Categoria9	67	32.78
12:00	user8	Product8	white	Google	Categoria8	60	46.79
14:30	user9	Product4	yellow	Apple	Categoria4	82	55.94
17:15	user9	Product5	red	Samsung	Categoria5	57	34.21
20:00	user9	Product6	orange	Coca-Cola	Categoria6	64	51.98
22:30	user9	Product7	grey	Audi	Categoria7	1	100000

Fonte: Elaborado pelo autor (2026).

B.2 LINHAS COM `ACTION == PURCHASE`

As oito linhas de compra são as mais completas do arquivo (34 das 39 colunas): acrescentam aos grupos anteriores os dados de pedido, contraparte da venda, avaliação e pagamento. Em todas elas, `order_status` vale `Completed` e `quantity` vale 1; o registro do evento (`timestamp`) ocorre entre 15 e 19/11/2023, posterior à data do pedido (`order_date`), que inclui hora e fuso horário no arquivo original. O pagamento (`payment_method`) é `cash` nos pedidos `ORD1` e `ORD2` e `credit_card` nos demais. As colunas de produto, categoria e estoque trazem os dados cadastrais do produto negociado, tal como nas linhas de estoque (Tabela 23); o endereço do comprador e o do vendedor (colunas `trader_*`) são os da Tabela 22. A Tabela 24 apresenta os valores próprios de cada compra.

Tabela 24 – Linhas `action == purchase` do `ecommerce_join.csv`.

Pedido	Data	Comprador	Vendedor	Produto	Nota	Comentário
ORD1	03/11	user1	user2	Product2	2	Bad
ORD2	03/11	user2	user1	Product3	4	Bom
ORD3	05/11	user3	user9	Product4	3	Médio
ORD4	07/11	user4	user9	Product5	5	Ótimo produto
ORD5	09/11	user5	user9	Product6	1	Não gostei
ORD6	10/11	user6	user9	Product7	5	—
ORD7	11/11	user7	user8	Product8	1	Reprovado
ORD8	14/11	user8	user2	Product9	4	Bom, bonito e barato

Fonte: Elaborado pelo autor (2026).

B.3 LINHAS COM `ACTION == SELECT_PRODUCT`

As oito linhas de seleção de produto preenchem apenas a identificação do evento, o usuário e o grupo de seleção (7 das 39 colunas): o produto visualizado (`selected_product_id`) e a quantidade de sugestões exibidas (`suggested_product_count`), que vale 3 em todas as linhas. A Tabela 25 apresenta os valores.

Tabela 25 – Linhas `action == select_product` do `ecommerce_join.csv`.

Momento	Usuário	Produto selecionado	Sugestões
15/11 15:09	user1	11	3
16/11 03:11	user2	12	3
16/11 17:50	user3	22	3
17/11 06:05	user4	14	3
17/11 21:56	user5	14	3
18/11 10:09	user6	21	3
18/11 23:20	user7	33	3
19/11 15:18	user8	17	3

Fonte: Elaborado pelo autor (2026).

B.4 LINHAS COM `ACTION == ADD_TO_CART`

As oito linhas de adição ao carrinho preenchem a identificação do evento, o usuário e o grupo de carrinho (8 das 39 colunas). O identificador do carrinho (`shopping_cart_id`) segue o padrão `user:⟨user_id⟩:cart` — por exemplo, `user:7fd7eee7-. . . :cart` para `user1`. A Tabela 26 apresenta os demais valores.

Tabela 26 – Linhas `action == add_to_cart` do `ecommerce_join.csv`.

Momento	Usuário	Produto adicionado	Quantidade
15/11 15:17	user1	3	1
16/11 03:27	user2	2	1
16/11 18:08	user3	2	1
17/11 06:14	user4	2	1
17/11 22:11	user5	2	1
18/11 10:21	user6	2	1
18/11 23:34	user7	2	1
19/11 15:29	user8	2	1

Fonte: Elaborado pelo autor (2026).

APÊNDICE C – ARQUIVOS DE CONFIGURAÇÃO DO CENÁRIO E-COMMERCE

Este apêndice reproduz os arquivos de configuração do cenário de e-commerce utilizado no capítulo 4, tal como se encontram no diretório `data/ecommerce/` do repositório da ferramenta: o arquivo de importação no modo multifonte `import_config.json` (Listagem C.1), cujos fragmentos por SGBD ilustraram a seção 4.2; a sua variante para o modo combinado `import_config_combined.json` (Listagem C.2); e o arquivo de conexão `dbms_config.json` (Listagem C.3). Junto com o arquivo de dados reproduzido no Apêndice B, eles constituem as entradas completas do exemplo, e neles podem ser conferidas as entidades, fontes e chaves que fundamentam as saídas materializadas da seção 4.3.3 e as contagens da Tabela 12.

C.1 ARQUIVO DE IMPORTAÇÃO (`IMPORT_CONFIG.JSON`)

```
{
  "sources": {
    "stock": "ecommerce_stock.csv",
    "purchase": "ecommerce_purchase.csv",
    "select_product": "ecommerce_select_product.csv",
    "add_to_cart": "ecommerce_add_to_cart.csv"
  },

  "postgres": {
    "entities": {
      "categories": {
        "source": "stock",
        "columns": {
          "category_id": { "is_key": true, "db_type": "BIGINT" },
          "category_name": { "db_type": "TEXT" }
        }
      },
      "products": {
        "source": "stock",
        "columns": {
          "product_id": { "is_key": true, "db_type": "BIGINT" },
          "product_name": { "db_type": "TEXT" },
          "product_variant": { "db_type": "TEXT" },
          "product_brand": { "db_type": "TEXT" },
          "category_id": { "db_type": "BIGINT" },
          "price": { "db_type": "NUMERIC(14,2)" }
        }
      },
      "inventory": {
        "source": "stock",
        "columns": {
          "product_id": { "is_key": true, "db_type": "BIGINT" },
          "quantity_available": { "db_type": "BIGINT" },
          "last_restock_date": { "db_type": "TIMESTAMPTZ" },
          "price": { "db_type": "NUMERIC(14,2)" }
```

```
      }
    },
    "orders": {
      "source": "purchase",
      "columns": {
        "order_number": { "is_key": true, "db_type": "TEXT" },
        "user_id": { "db_type": "TEXT" },
        "order_date": { "db_type": "TIMESTAMPTZ" },
        "order_status": { "db_type": "TEXT" },
        "payment_method": { "db_type": "TEXT" },
        "quantity": { "db_type": "BIGINT" },
        "product_id": { "db_type": "BIGINT" },
        "price": { "db_type": "NUMERIC(14,2)" },
        "comment": { "db_type": "TEXT" },
        "rating": { "db_type": "BIGINT" }
      }
    }
  },
  "relationships": {
    "product_category": {
      "from": "products",
      "to": "categories",
      "foreign_key": "category_id",
      "references_key": "category_id"
    },
    "order_product": {
      "from": "orders",
      "to": "products",
      "foreign_key": "product_id",
      "references_key": "product_id"
    }
  }
},

"mongodb": {
  "entities": {
    "product_catalog": {
      "source": "stock",
      "columns": {
        "product_id": {},
        "product_name": {},
        "product_variant": {},
        "product_brand": {},
        "product_description": {},
        "product_image": {},
        "price": {},
        "category": {
          "category_id": {},
          "category_name": {}
        },
        "stock": {
          "quantity_available": {},
          "last_restock_date": {}
        }
      }
    }
```

```
    }
  },

  "cassandra": {
    "entities": {
      "user_activity_log": {
        "source": ["stock", "purchase", "select_product", "add_to_cart"],
        "columns": {
          "user_id": {},
          "timestamp": { "schema_column": "event_time" },
          "_source": { "schema_column": "event_type" },
          "product_id": {},
          "order_number": {},
          "selected_product_id": {},
          "shopping_cart_id": {}
        },
        "cassandra_partition": ["user_id"],
        "cassandra_cluster": ["timestamp"]
      }
    }
  },

  "redis": {
    "entities": {
      "shopping_cart": {
        "source": "add_to_cart",
        "auto_map": true,
        "columns": {
          "shopping_cart_id": { "is_key": true }
        }
      },
      "user_session": {
        "source": "select_product",
        "columns": {
          "user_id": { "is_key": true },
          "user_name": {},
          "user_email": {},
          "timestamp": { "schema_column": "last_seen" }
        }
      }
    }
  },

  "neo4j": {
    "entities": {
      "User": {
        "source": "purchase",
        "columns": {
          "user_id": { "is_key": true },
          "user_name": {},
          "user_email": {}
        }
      },
      "Product": {
        "source": "stock",
        "columns": {
```

```
          "product_id": { "is_key": true },
          "product_name": {},
          "product_brand": {}
        }
      }
    },
    "relationships": {
      "PURCHASED": {
        "from": "User",
        "to": "Product",
        "type": "PURCHASED",
        "columns": {
          "order_number": { "is_key": true },
          "quantity": {},
          "price": {},
          "rating": {}
        }
      }
    }
  }
}
```

Listagem C.1 – Arquivo de importação completo do cenário e-commerce (`import_config.json`).

C.2 ARQUIVO DE IMPORTAÇÃO NO MODO COMBINADO (`IMPORT_CONFIG_COMBINED.JSON`)

No modo combinado, uma única fonte declara o arquivo largo `ecommerce_join.csv` com a marca `origin_column`, que instrui a ferramenta a tratar a primeira coluna de cada linha como a sua origem. O restante do arquivo — os mapeamentos de `postgres`, `mongodb`, `cassandra`, `redis` e `neo4j` — é **idêntico** ao do arquivo multifonte da Listagem C.1: as entidades continuam a referir-se às origens pelos mesmos nomes (`stock`, `purchase`, `select_product` e `add_to_cart`). A Listagem C.2 reproduz apenas o bloco `sources` que distingue os dois arquivos; as reticências indicam o trecho de mapeamento omitido por ser igual ao já apresentado.

```
{
  "sources": {
    "ecommerce": { "file": "ecommerce_join.csv", "origin_column": true }
  },

  "postgres":  { ... },
  "mongodb":   { ... },
  "cassandra": { ... },
  "redis":     { ... },
  "neo4j":     { ... }
}
```

Listagem C.2 – Bloco `sources` do arquivo de importação no modo combinado (`import_config_combined.json`).

C.3 ARQUIVO DE CONEXÃO (DBMS_CONFIG.JSON)

```
{
  "version": 1,

  "postgres": {
    "connection": {
      "host": "127.0.0.1",
      "port": 5432,
      "database": "ecommerce",
      "user": "postgres",
      "password": "postgres"
    },
    "schema": "public",
    "start": { "compose": { "file": "../../docker-compose.yml", "service": "postgres
        " } }
  },

  "mongodb": {
    "connection": {
      "uri": "mongodb://127.0.0.1:27017",
      "database": "ecommerce"
    },
    "start": { "compose": { "file": "../../docker-compose.yml", "service": "mongodb"
         } }
  },

  "cassandra": {
    "connection": {
      "hosts": ["127.0.0.1"],
      "port": 9042,
      "keyspace": "ecommerce"
    },
    "start": { "compose": { "file": "../../docker-compose.yml", "service": "
        cassandra" } }
  },

  "redis": {
    "connection": { "host": "127.0.0.1", "port": 6379, "db": 0 },
    "start": { "compose": { "file": "../../docker-compose.yml", "service": "redis" }
         }
  },

  "neo4j": {
    "connection": {
      "uri": "bolt://127.0.0.1:7687",
      "user": "neo4j",
      "password": "password",
      "database": "neo4j"
    },
    "start": { "compose": { "file": "../../docker-compose.yml", "service": "neo4j" }
         }
  }
}
```

Listagem C.3 – Arquivo de conexão completo do cenário e-commerce (dbms_config.json).